



SWINBURNE UNIVERSITY OF TECHNOLOGY

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 3

Cloud Application Deployment & Infrastructure Scaling on AWS

DECLARED TARGET LEVEL: HIGH DISTINCTION (Task 3.4)

Field	Detail
Student Name	Karamjot Singh
Student ID	104809887
Unit Code & Title	SWE40006 - Software Deployment and Evolution
Semester	Semester 2, 2026
Assessment	Deployment Portfolio Task 3
Submission Date	09/09/2026
Declared Target Level	High Distinction - Tasks 3.1, 3.2, 3.3 and 3.4 all completed
AWS Region	Asia Pacific (Sydney) - ap-southeast-2
Source Repository	https://github.com/Karam1008/swe40006-task3-aws.git

Table of Contents

TABLE OF CONTENTS	2
1. INTRODUCTION AND SCOPE	4
1.1 PURPOSE OF THE TASK	4
1.2 NAMING CONVENTION	5
1.3 LIVE VERIFICATION ENDPOINTS	5
2. ARCHITECTURE OVERVIEW	6
2.1 FINAL NETWORK TOPOLOGY	6
3. TASK 3.1 - PASS LEVEL: MONOLITHIC DEPLOYMENT	7
3.1 ACCOUNT AND REGION CONFIGURATION	7
3.2 SSH KEY PAIR GENERATION	8
3.3 COMPUTE PROVISIONING	8
3.4 LAMP STACK INSTALLATION	10
3.5 LOCAL DATABASE AND WORDPRESS DEPLOYMENT	11
3.6 VERIFICATION	13
3.7 WEAKNESSES OF THE PASS-LEVEL DESIGN	15
4. TASK 3.2 - CREDIT LEVEL: DATABASE DECOUPLING AND S3 BACKUP	16
4.1 SECURITY GROUP DESIGN PRECEDING THE DATABASE	16
4.2 RDS INSTANCE PROVISIONING	17
4.3 DATABASE EXPORT	18
4.4 ENCRYPTED CONNECTIVITY TO RDS	19
4.5 APPLICATION USER AND LEAST PRIVILEGE	20
4.6 IMPORT AND VERIFICATION	21
4.7 APPLICATION CUTOVER	22
4.8 PROVING THE DECOUPLING	23
4.9 S3 BUCKET CONFIGURATION	24
4.10 IAM ROLE RATHER THAN ACCESS KEYS	26
4.11 BACKUP EXECUTION	27
4.12 RESTORE DEMONSTRATION	28
5. TASK 3.3 - DISTINCTION LEVEL: LOAD BALANCING AND AUTO SCALING	31
5.1 THREE-TIER SECURITY GROUP REFACTOR	31
5.2 TARGET GROUP AND HEALTH CHECK DESIGN	32
5.3 APPLICATION LOAD BALANCER	33
5.4 PREPARING THE GOLDEN AMI	34
5.5 LAUNCH TEMPLATE	36
5.6 AUTO SCALING GROUP	38
5.7 VERIFICATION	40
6. TASK 3.4 - HIGH DISTINCTION LEVEL: MONITORING AND SECURE MANAGEMENT	45
6.1 SNS TOPIC AND SUBSCRIPTION	45
6.2 CLOUDWATCH ALARM DESIGN	47
6.3 ALARM VALIDATION UNDER SYNTHETIC LOAD	49
6.4 SYSTEMS MANAGER SESSION MANAGER	53
6.5 ELIMINATION OF PUBLIC SSH ACCESS	54
6.6 SECURITY POSTURE ACROSS THE FOUR LEVELS	55

7. SELF-TROUBLESHOOTING LOG AND ANALYSIS	56
7.1 MySQL Error 2005 - UNKNOWN SERVER HOST	56
7.2 MySQL Error 3159 - INSECURE TRANSPORT PROHIBITED	56
7.3 MySQL Error 1045 - ACCESS DENIED	57
7.4 SHELL METACHARACTER MISINTERPRETATION	57
7.5 SECURITY GROUP CHANGE CAUSING LOSS OF ACCESS	57
7.6 WORDPRESS CANONICAL URL REDIRECTION	58
7.7 STRESS-NG TEMPORARY PATH FAILURE	58
8. LIMITATIONS AND FUTURE WORK	59
8.1 UPLOADED MEDIA IS NOT SHARED BETWEEN INSTANCES	59
8.2 No HTTPS TERMINATION	59
8.3 FIXED CAPACITY RATHER THAN ELASTIC SCALING	59
8.4 SINGLE-AZ DATABASE	59
8.5 MANUAL BACKUPS AND MANUAL DEPLOYMENT	59
8.6 INFRASTRUCTURE DEFINED THROUGH THE CONSOLE	60
8.7 MONITORING COVERS COMPUTE ONLY	60
9. EXTERNAL CODE AND SCRIPT REPOSITORY	61
10. GENERATIVE AI DECLARATION	62
10.1 TOOL USED AND SCOPE OF USE	62
10.2 WHAT WAS PERFORMED BY ME	62
10.3 VERIFICATION AND INDEPENDENT JUDGEMENT	63
10.4 STATEMENT	63
11. REFERENCES	64

1. Introduction and Scope

This report documents the design, provisioning, deployment and operational hardening of a production-style WordPress web application on Amazon Web Services, completed for Deployment Portfolio Task 3 of SWE40006. All four grade bands were attempted and completed in sequence: 3.1 (Pass), 3.2 (Credit), 3.3 (Distinction) and 3.4 (High Distinction).

The work was carried out in a personal AWS account on the AWS Free Plan, in the Asia Pacific (Sydney) region (ap-southeast-2). Every resource was created and configured by me through the AWS Management Console and the AWS CLI running on the EC2 instances themselves. No pre-built templates, third-party automation or classroom sandbox was used.

1.1 Purpose of the Task

The task is framed as a progressive migration exercise. Each grade band deliberately removes a weakness introduced by the previous one:

- Pass (3.1) produces a working but entirely monolithic deployment - web server, application and database all on a single EC2 instance, with the instance itself directly exposed to the internet.
- Credit (3.2) removes the database from the instance, making the compute layer stateless and therefore replaceable, and introduces an off-instance backup and restore capability.
- Distinction (3.3) exploits that statelessness by placing instances behind a load balancer in an Auto Scaling Group, so that instance failure no longer means application failure.
- High Distinction (3.4) closes the remaining operational gaps: the deployment previously had no monitoring and no alerting, and administrative access still depended on an open SSH port and a private key file.

1.2 Naming Convention

A consistent prefix was applied to every resource created for this task so that all artefacts could be identified unambiguously during verification and teardown.

Resource	Identifier
Key pair	swe40006-pass-key
Original EC2 instance	SWE40006-WebServer-Pass
Restore-target EC2 instance	SWE40006-WebServer-Restore
RDS instance	swe40006-rds
S3 bucket	swe40006-backup-104809887
IAM role / policy	swe40006-ec2-s3-role
Security groups	swe40006-alb-sg, swe40006-web-sg, swe40006-rds-sg
AMI	swe40006-wordpress-ami
Launch template	swe40006-launch-template
Target group / ALB	swe40006-tg / swe40006-alb
Auto Scaling Group	swe40006-asg
SNS topic	swe40006-alerts
CloudWatch alarm	swe40006-high-cpu-alarm

1.3 Live Verification Endpoints

The following endpoints were live at the time of submission and may be used for grading verification.

Endpoint	Value
Application Load Balancer DNS (primary)	swe40006-alb-447325225.ap-southeast-2.elb.amazonaws.com
Health check endpoint	swe40006-alb-447325225.ap-southeast-2.elb.amazonaws.com/health.html
Instance identity endpoint	swe40006-alb-447325225.ap-southeast-2.elb.amazonaws.com/instance.html
RDS endpoint (private, not internet reachable)	swe40006-rds.chm0e2oc24zi.ap-southeast-2.rds.amazonaws.com
Source repository	https://github.com/Karam1008/swe40006-task3-aws.git

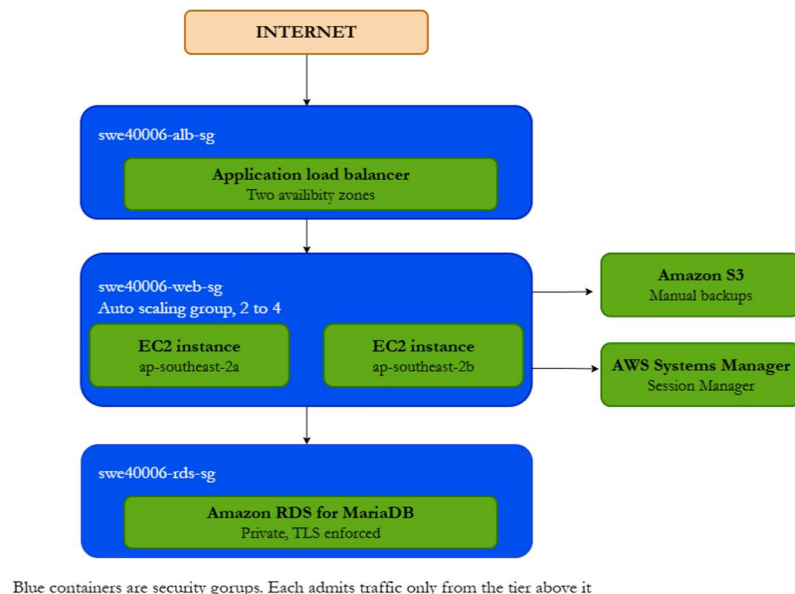
2. Architecture Overview

The architecture evolved across four distinct states. The table below summarises that evolution and is expanded in the sections that follow.

Level	Compute	Data	Access & Operations
3.1 Pass	Single EC2 instance, public IP, directly internet-facing	MariaDB local to the instance	SSH on port 22 open to 0.0.0.0/0
3.2 Credit	Single EC2 instance; second instance used to prove restore	Amazon RDS for MariaDB, private, TLS enforced; backups in S3	SSH open; IAM instance role replaces static credentials
3.3 Distinction	Auto Scaling Group, 2–4 instances across two Availability Zones, behind an ALB	Shared RDS instance; golden AMI holds application code	Instances no longer publicly reachable; SSH restricted to a single source IP
3.4 High Distinction	Unchanged from 3.3	Unchanged from 3.3	Port 22 removed entirely; access via SSM Session Manager; CloudWatch alarm with SNS email

2.1 Final Network Topology

The final design is a conventional three-tier layout inside the default VPC, using the public subnets of two Availability Zones. Traffic flows in one direction only, and each tier is authorised by referencing the security group of the tier above it rather than by IP range:



Two properties of this topology (they are the outcome of deliberate choices rather than defaults):

- No inbound path exists from the internet to any EC2 instance. The only internet-facing listener in the account is the ALB on port 80.
- No inbound path exists from the internet to the database. RDS was created with "Publicly accessible: No" and its security group accepts port 3306 only from the web tier security group.

3. Task 3.1 - Pass Level: Monolithic Deployment

Objective: create an AWS account, generate an SSH key pair, launch an EC2 web server in a security group permitting HTTP, HTTPS and SSH, deploy a working WordPress/LAMP stack, and verify public accessibility via the instance public IPv4 address.

3.1 Account and Region Configuration

An AWS account was created under the AWS Free Plan. Asia Pacific (Sydney), ap-southeast-2, was selected as the working region and held constant for every subsequent resource. Region consistency matters more than it first appears: security groups, AMIs, launch templates, target groups and RDS instances are all regional resources, and a resource created in the wrong region cannot be referenced by resources in another.

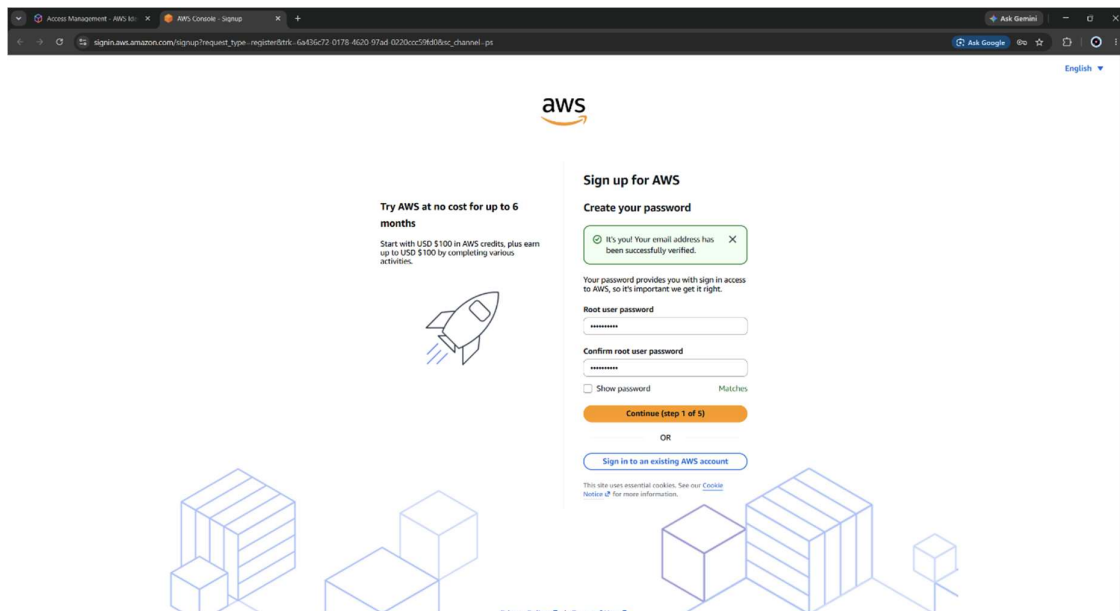


Figure 1: AWS account setup; email verified and account created (Free tier)

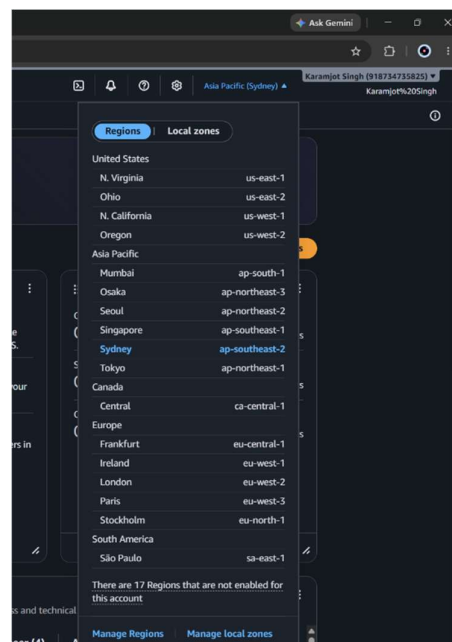


Figure 2: Region selector confirming Asia Pacific (Sydney), ap-southeast-2, as the working region.

3.2 SSH Key Pair Generation

An RSA key pair named swe40006-pass-key was generated through EC2 → Network & Security → Key Pairs, in .ppk format. AWS retains only the public half; the private key is downloaded once and cannot be retrieved again, which is the reason key pair loss requires instance replacement rather than recovery.

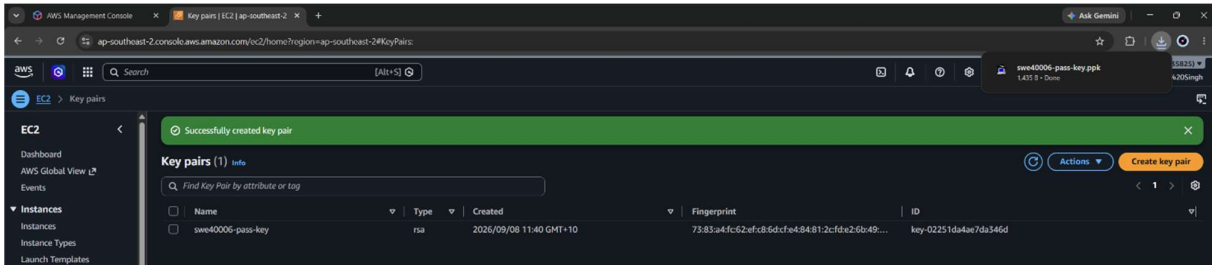


Figure 3: Created key pair

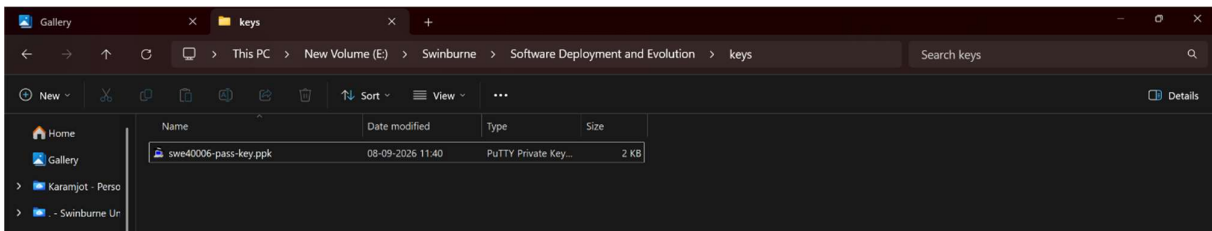


Figure 4: Key pair downloaded

3.3 Compute Provisioning

An EC2 instance named SWE40006-WebServer-Pass was launched with the following configuration:

Setting	Value and rationale
AMI	Amazon Linux 2023 - free tier eligible, ships with dnf, systemd and a preinstalled AWS CLI v2 and SSM Agent, both of which are relied on at Credit and High Distinction levels
Instance type	t3.micro - free tier eligible burstable instance, 2 vCPU / 1 GiB
Key pair	swe40006-pass-key
Storage	Default gp3 root volume, within the 30 GB free tier allowance
Auto-assign public IP	Enabled - required at this level because the instance is the public entry point
Security group	launch-wizard-1 : SSH 22, HTTP 80 and HTTPS 443, all from 0.0.0.0/0

The choice of Amazon Linux 2023 over Ubuntu was made deliberately. The default login user is ec2-user rather than ubuntu, the package manager is dnf rather than apt, and the Apache service unit is named httpd rather than apache2. Amazon Linux 2023 was preferred because the SSM Agent required at High Distinction level is preinstalled and enabled by default, avoiding an installation step on every instance launched by the Auto Scaling Group later in the task.

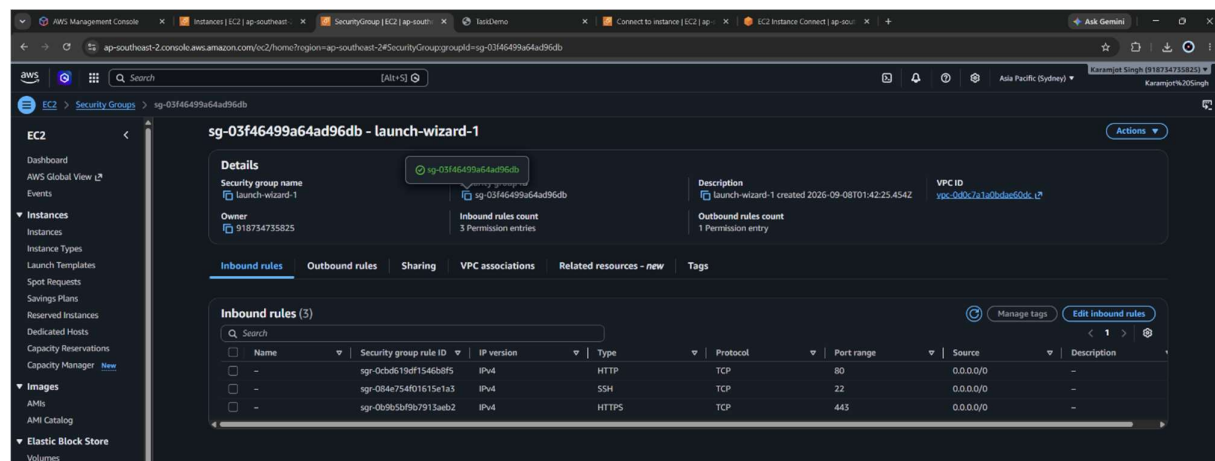


Figure 5: Launch wizard Security Group inbound rules

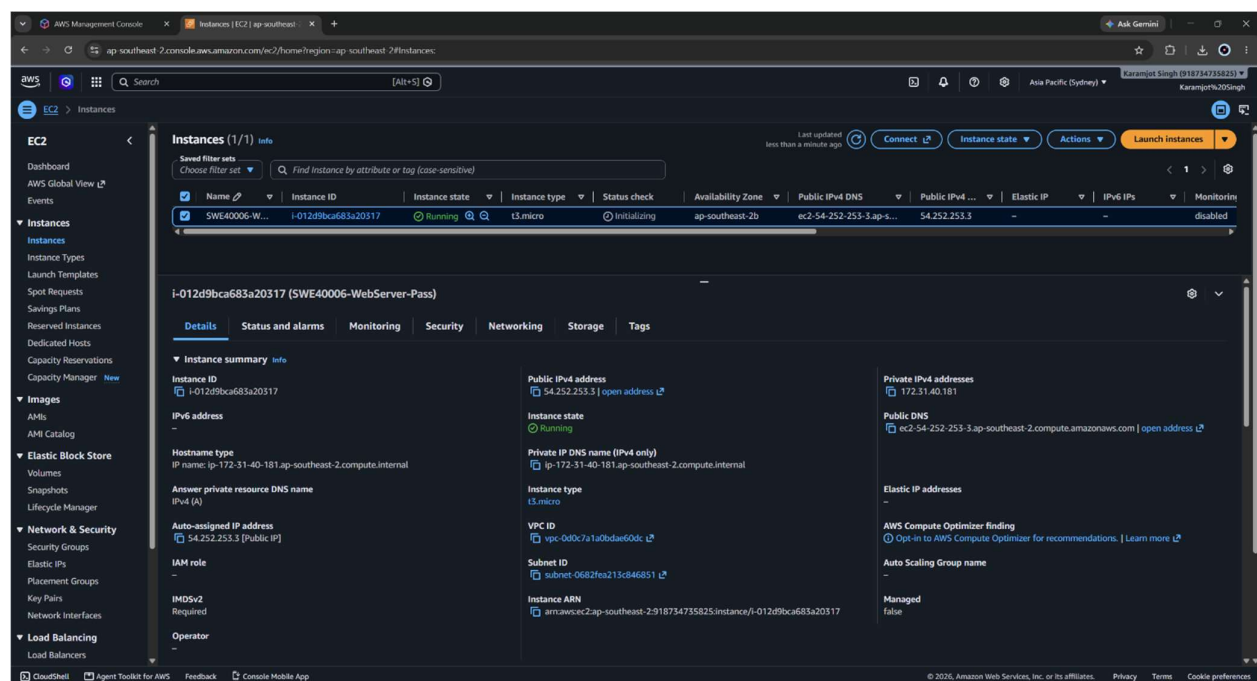


Figure 6: EC2 Instances console showing SWE40006-WebServer-Pass with its instance ID, type and Availability Zone

3.4 LAMP Stack Installation

The instance was accessed over SSH and the LAMP stack installed.

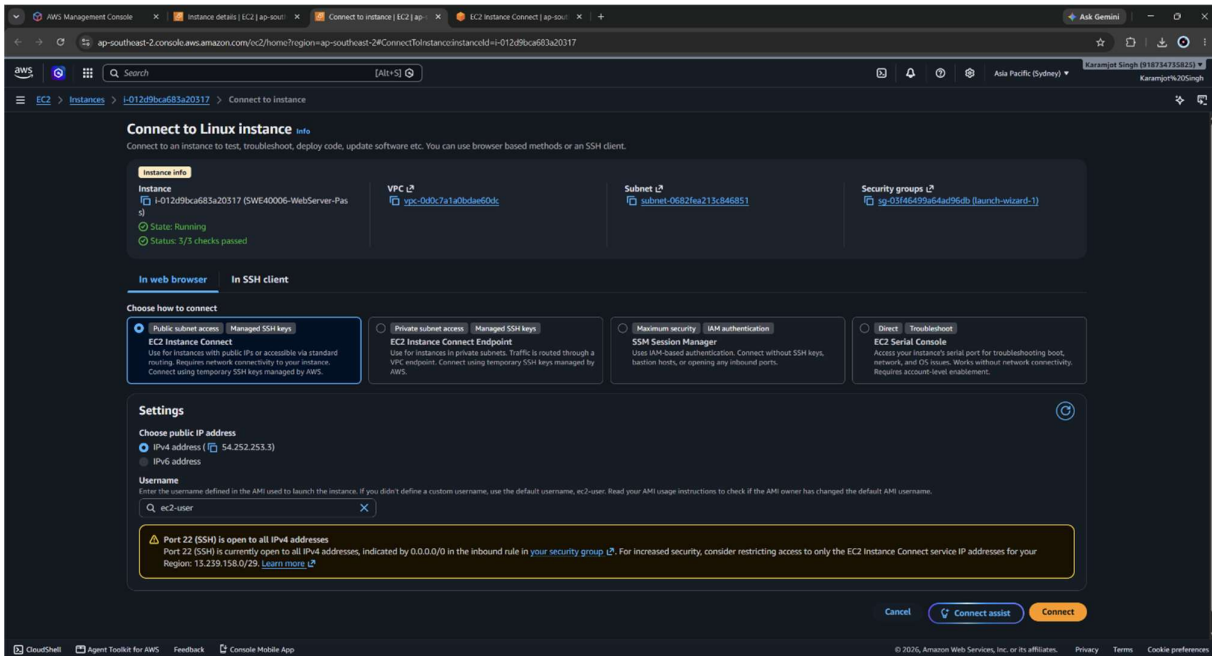


Figure 7: Instance connection to SSH

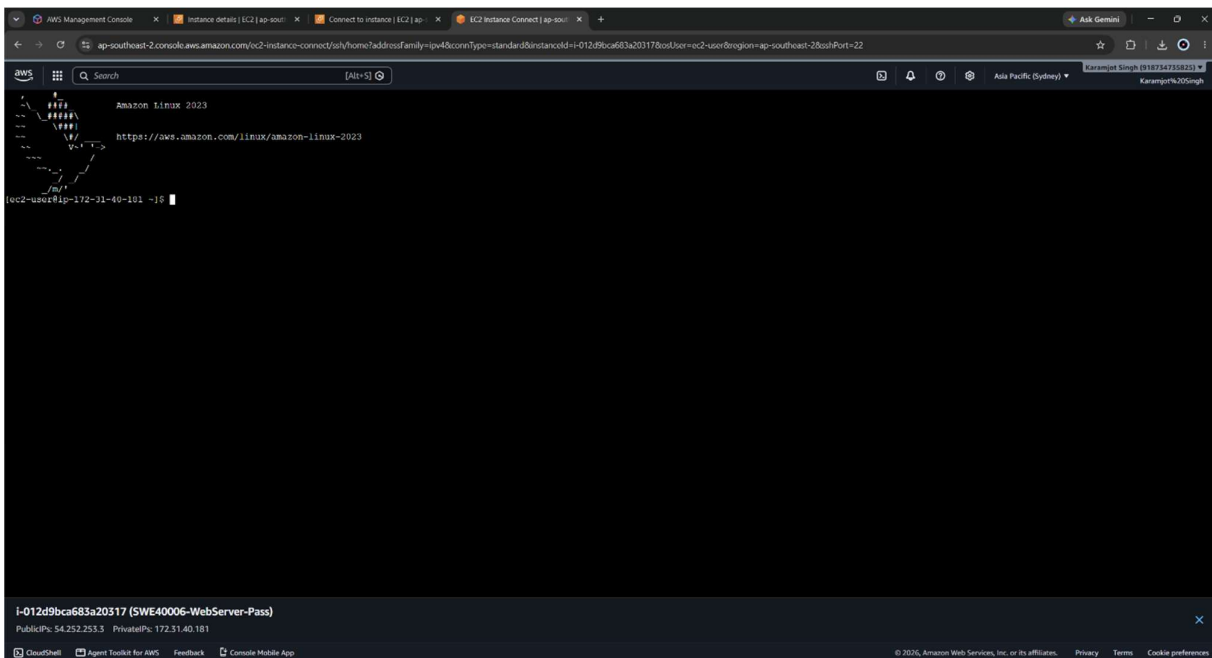


Figure 8: Successful connection (EC2 Instance connect console)

Commands were executed in the following sequence:

```
# Update the package index
sudo dnf update -y

# Install Apache, MariaDB server, PHP and the extensions WordPress requires
sudo dnf install -y httpd mariadb105-server php php-mysqlnd php-gd php-xml

# Enable both services at boot and start them now
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl start mariadb
sudo systemctl enable mariadb
```

The distinction between “systemctl start” and “systemctl enable” is significant for this task. start affects only the current boot; enable creates the symbolic link in the systemd multi-user target that causes the service to launch automatically on subsequent boots. Without enable, an instance launched from an AMI of this machine would boot with Apache stopped - which would cause every Auto Scaling Group instance to fail its load balancer health check at Distinction level.

```
Verifying : php8.5-cli-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-common-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-fpm-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-gd-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-mbstring-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-mysqlnd-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-openssl-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-sodium-8.5.9-1.amzn2023.0.1.x86_64
Verifying : php8.5-xml-8.5.9-1.amzn2023.0.1.x86_64
Verifying : python0.43.4-1.amzn2023.0.4.x86_64
Verifying : xml-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64
fonts-filesystem-1.2.0-12.amzn2023.0.2.noarch
generic-logos-httpd-18.0-12.amzn2023.0.3.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64
jbigkit-libs-2.1-21.amzn2023.0.2.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXpm-3.5.17-3.amzn2023.0.2.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
liblfr-4.4.0-4.amzn2023.0.27.x86_64
maillog-2.1.49-3.amzn2023.0.3.noarch
mariadb105-3110.5.29-1.amzn2023.0.1.x86_64
mariadb105-cracklib-password-check-3110.5.29-1.amzn2023.0.1.x86_64
mariadb105-server-3110.5.29-1.amzn2023.0.1.x86_64
mod_ssl-2.4.68-1.amzn2023.0.1.x86_64
perl-DBD-MariaDB-1.22-1.amzn2023.0.5.x86_64
perl-DBD-Mysql-1.21.9998.39-2.amzn2023.0.2.noarch
perl-Module-Load-1.0.36-2.amzn2023.0.2.noarch
php8.5-cli-8.5.9-1.amzn2023.0.1.x86_64
php8.5-gd-8.5.9-1.amzn2023.0.1.x86_64
php8.5-openssl-8.5.9-1.amzn2023.0.1.x86_64
php8.5-sodium-8.5.9-1.amzn2023.0.1.x86_64
php8.5-xml-8.5.9-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-172-31-40-101 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-40-101 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-172-31-40-101 ~]$ sudo systemctl start mariadb
[ec2-user@ip-172-31-40-101 ~]$ sudo systemctl enable mariadb
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[ec2-user@ip-172-31-40-101 ~]$
```

Figure 9: Console output of the dnf install command showing Apache, MariaDB and PHP packages resolving and completing

3.5 Local Database and WordPress Deployment

A database and application user were created on the instance-local MariaDB server:

```
sudo mysql -u root

CREATE DATABASE wordpress_db;
CREATE USER 'wp_user'@'localhost' IDENTIFIED BY '<password>';
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

The local root account authenticates over the Unix socket rather than by password, which is why sudo mysql -u root requires no credentials. This works only for a process running as root on the machine itself and is a

meaningful security property: the local database was never reachable over the network at all. That property is lost when the database moves to RDS, which is precisely why RDS enforces TLS and network-level restriction instead.

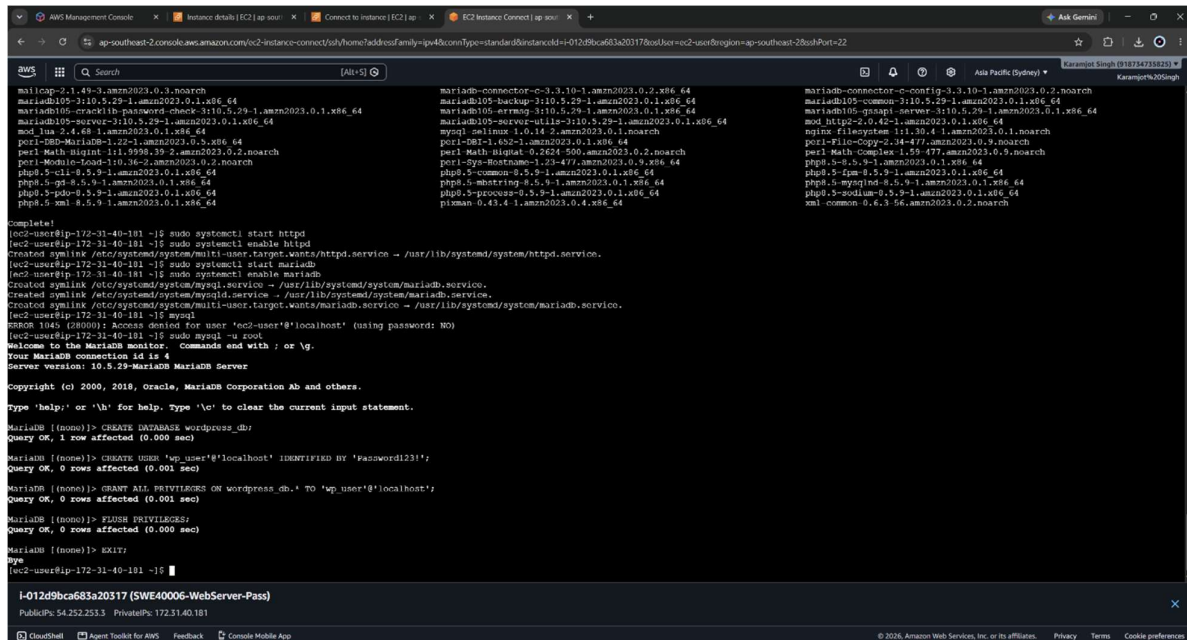


Figure 10: MariaDB monitor showing the CREATE DATABASE, CREATE USER and GRANT statements executing successfully on the local server.

WordPress was then downloaded and deployed into the Apache document root:

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
sudo cp -r wordpress/* /var/www/html/

sudo chown -R apache:apache /var/www/html
sudo chmod -R 755 /var/www/html/
```

The ownership change is required because the Apache worker processes run as the apache user; without it WordPress cannot write wp-config.php during the installation wizard and the setup fails with a permissions error.

```

[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl start mariadb
[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl enable mariadb
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysql.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[ec2-user@ip-172-31-40-181 ~]$ mysql
mysql [(none)]> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.000 sec)

mysql [(none)]> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'Password123!';
Query OK, 0 rows affected (0.001 sec)

mysql [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

mysql [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

mysql [(none)]> EXIT;
mysql>
[ec2-user@ip-172-31-40-181 ~]$ cd /tmp
[ec2-user@ip-172-31-40-181 tmp]$ wget https://wordpress.org/latest.tar.gz
--2026-09-08 01:56:14-- https://wordpress.org/latest.tar.gz
Connecting to wordpress.org (wordpress.org) [66.6.42.252]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35356041 (34M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz
100%[=====] 35,356,041 33.72M 10.0MB/s in 3.3s

2026-09-08 01:56:18 (10.2 MB/s) - 'latest.tar.gz' saved [35356041/35356041]

[ec2-user@ip-172-31-40-181 tmp]$ tar -xzf latest.tar.gz
[ec2-user@ip-172-31-40-181 tmp]$ cd - && mv wordpress/* /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$ sudo chown -R apache:apache /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$ sudo chmod -R 755 /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$

```

i-012d9bca683a20317 (SWE40006-WebServer-Pass)

PublicIP: 54.252.253.3 PrivateIP: 172.31.40.181

Figure 11: WordPress installation wizard reached at <http://<public-ip>>, confirming Apache and PHP are serving the application.

3.6 Verification

The installation wizard was completed against the local database (Database Host: localhost) and an identifiable post was published, titled "Post created on local MariaDB", to provide a verifiable data marker for the migration performed in Task 3.2. Without such a marker, a successful migration is indistinguishable from a fresh empty installation.

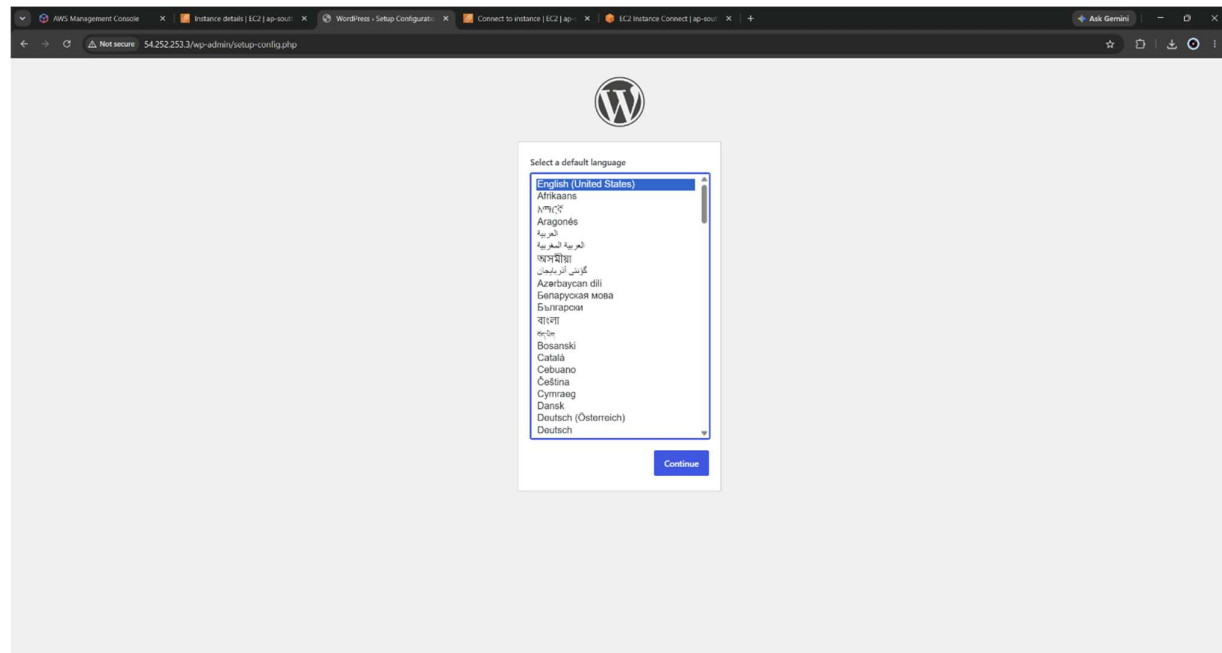
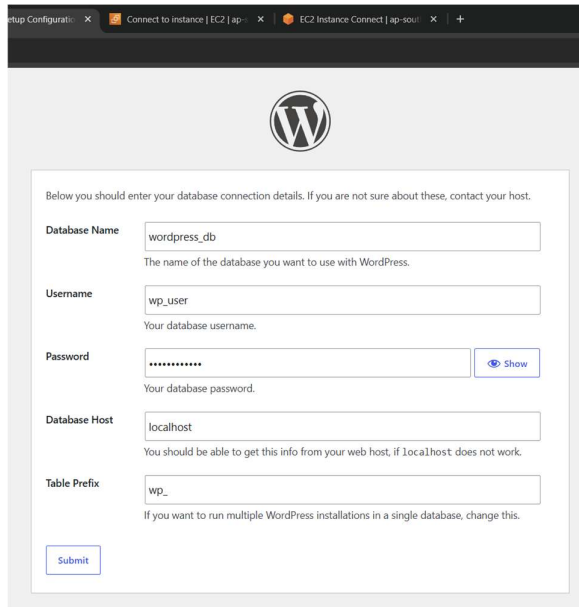


Figure 12: Wordpress admin setup interface (instance ip)



Below you should enter your database connection details. If you are not sure about these, contact your host.

Database Name
wordpress_db
The name of the database you want to use with WordPress.

Username
wp_user
Your database username.

Password

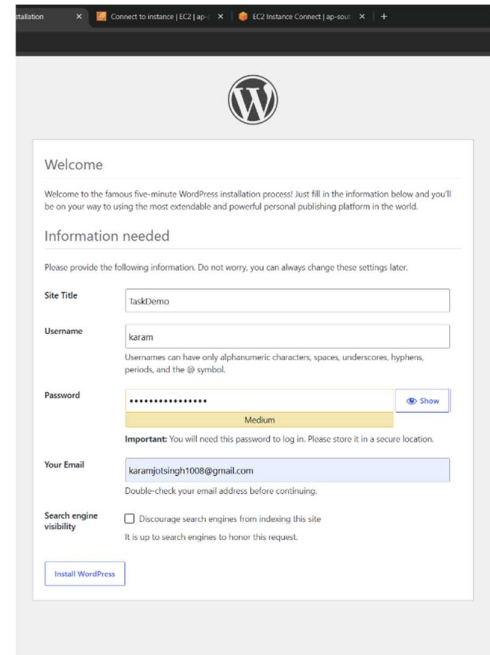
Your database password. [Show](#)

Database Host
localhost
You should be able to get this info from your web host, if localhost does not work.

Table Prefix
wp_
If you want to run multiple WordPress installations in a single database, change this.

[Submit](#)

Figure 13: Wordpress database creation



Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title
TaskDemo

Username
karam
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password

Medium [Show](#)

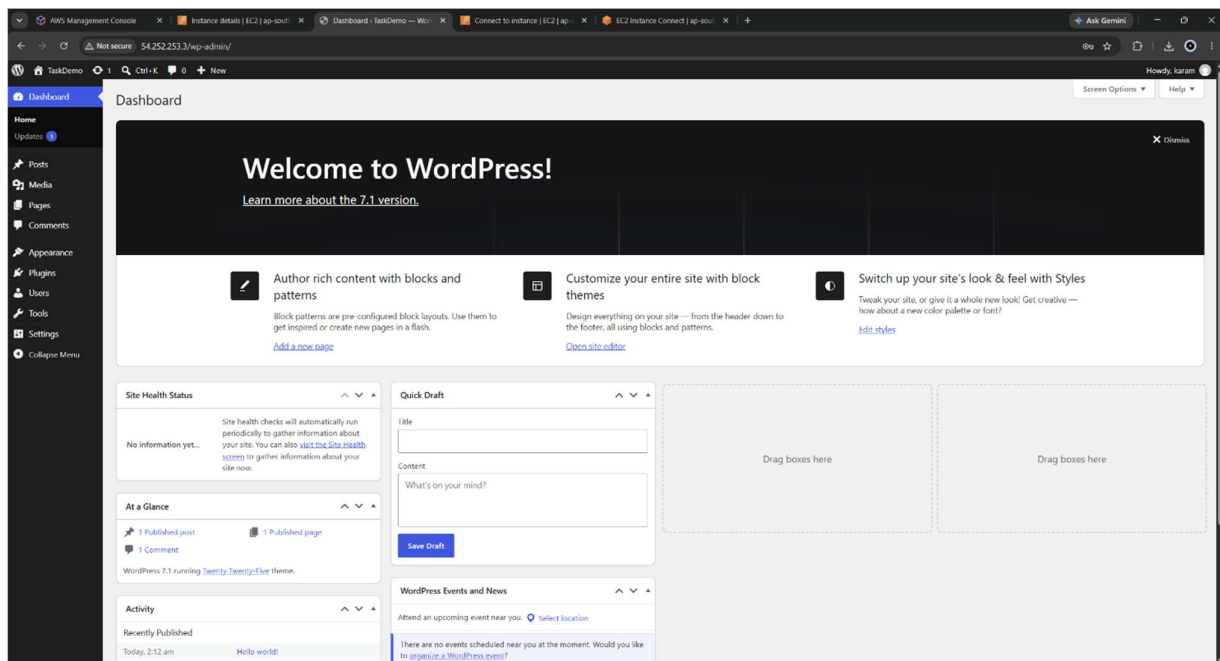
Important: You will need this password to log in. Please store it in a secure location.

Your Email
karamjotsingh1008@gmail.com
Double-check your email address before continuing.

Search engine visibility
☐ Discourage search engines from indexing this site
It is up to search engines to honor this request.

[Install WordPress](#)

Figure 14: Wordpress admin account creation



Dashboard

Welcome to WordPress!

[Learn more about the 7.1 version.](#)

Author rich content with blocks and patterns
Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.
[Add a new page](#)

Customize your entire site with block themes
Design everything on your site — from the header down to the footer, all using blocks and patterns.
[Open site editor](#)

Switch up your site's look & feel with Styles
Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?
[Edit styles](#)

Site Health Status
Site health checks will automatically run periodically to gather information about your site. You can also [visit the Site Health screen](#) to get more information about your site now.

At a Glance
1 Published post 1 Published page
1 Comment
WordPress 7.1 running Twenty Twenty-Three theme.

Activity
Recently Published
Today, 2:12 am Hello world!

Quick Draft
Title
Content
What's on your mind?
[Save Draft](#)

WordPress Events and News
Attend an upcoming event near you. [Select location](#)
There are no events scheduled near you at the moment. Would you like to [organize a WordPress event](#)?

Drag boxes here

Figure 15: Wordpress admin dashboard accessed

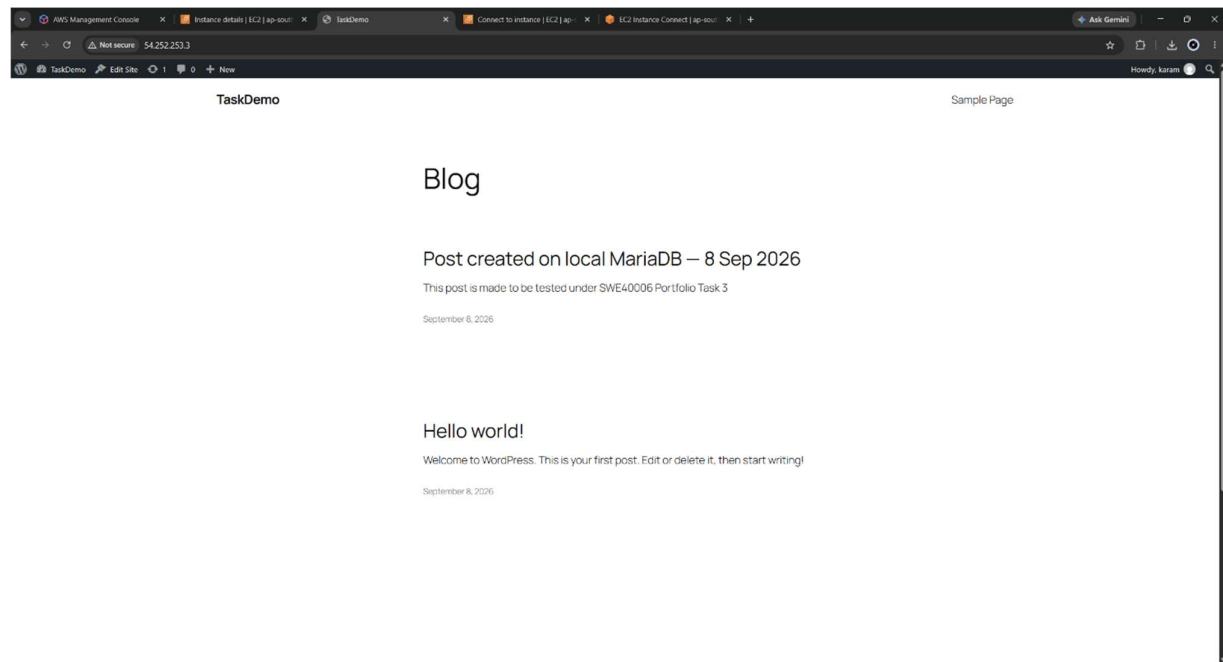


Figure 16: WordPress site rendering the identifiable post, served from the EC2 public IPv4 address (visible in the address bar)

3.7 Weaknesses of the Pass-Level Design

The deployment is functional but has four defects, each addressed at a later level:

- All application state lives on a single instance. Terminating it destroys the database, so the instance cannot be treated as disposable - which rules out auto scaling entirely.
- There is no backup. An accidental deletion or a failed update is unrecoverable.
- The instance is a single point of failure and is directly exposed to the internet on three ports.
- There is no monitoring. A failure would be discovered only by a user attempting to load the site.

4. Task 3.2 - Credit Level: Database Decoupling and S3 Backup

Objective: migrate the application database from the EC2 instance to Amazon RDS and reconfigure the application to use the RDS endpoint; create an S3 bucket, upload a manual backup of the application files, and demonstrate restoring those files to an EC2 instance.

4.1 Security Group Design Preceding the Database

The RDS security group was created before the database instance so that it could be attached during creation rather than retrofitted. A dedicated group, swe40006-rds-sg, was configured with a single inbound rule:

Type	Port	Source	Purpose
MySQL/Aurora	TCP 3306	sg- (launch-wizard-1)	Permit database access from the web tier only

The source is a security group identifier, not a CIDR range. This is the important design decision in this section. Referencing a security group means the rule authorises membership rather than a network location: any instance that belongs to the web tier group is permitted, and any instance that does not - including other instances in the same VPC and subnet - is denied. The authorisation therefore survives instance replacement, IP reassignment and Auto Scaling activity without the rule ever needing to be edited. A CIDR-based rule would have had to be updated every time the Auto Scaling Group replaced an instance in Task 3.3.

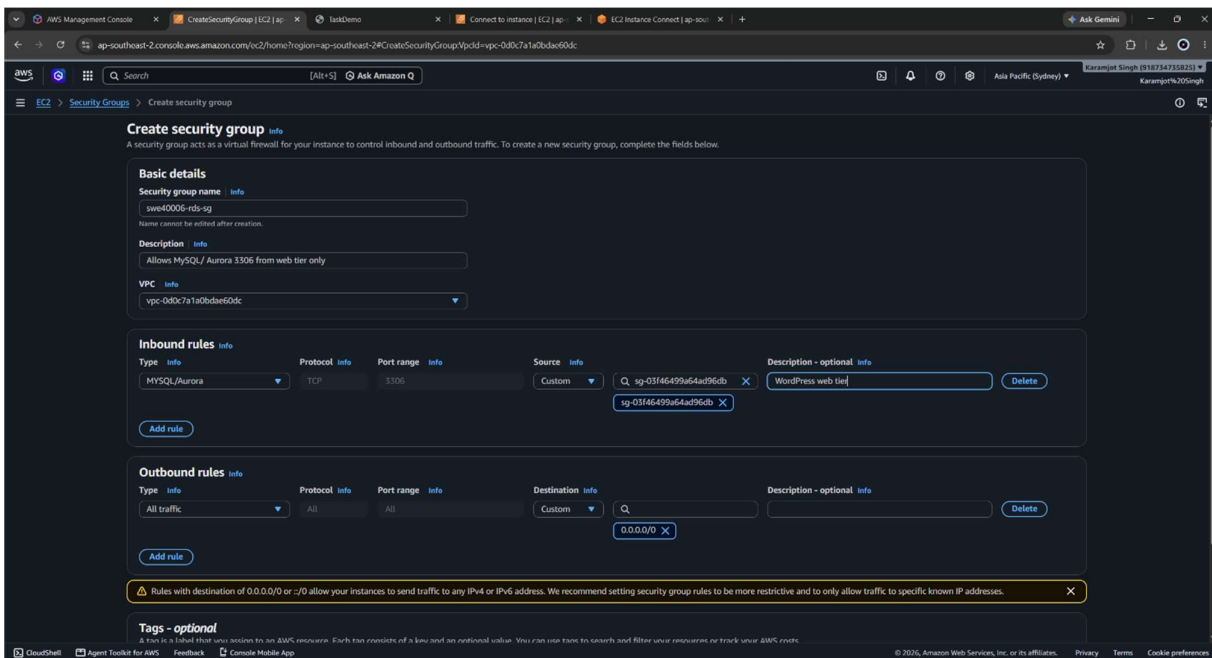


Figure 17: Creating RDS Security Group - inbound rules

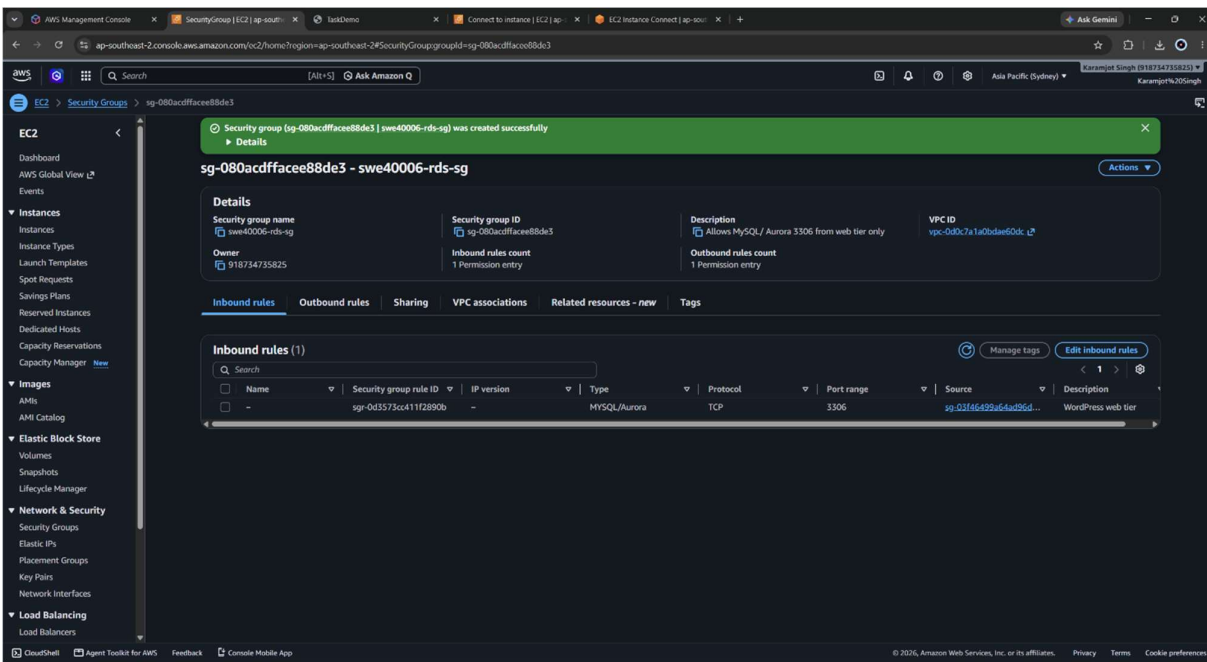


Figure 18: Created RDS Security group

4.2 RDS Instance Provisioning

An RDS instance was created with the configuration below. Several settings were chosen specifically to constrain cost within the AWS Free Plan and are annotated as such.

Setting	Value	Rationale
Engine	MariaDB	Matches the MariaDB 10.5 source engine, eliminating dump compatibility risk
Template	Free tier	Constrains the console to free-tier-eligible options
Instance class	db.t4g.micro	Free tier eligible burstable class
Storage	20 GiB gp3, autoscaling disabled	Autoscaling disabled deliberately - it can silently grow storage beyond the free allowance
Multi-AZ	Single instance	Multi-AZ is not free-tier eligible; accepted as a documented limitation
Publicly accessible	No	The database has no public IP and cannot be reached from outside the VPC
Security group	swe40006-rds-sg	Default group explicitly removed
Initial database name	wordpress_db	Avoids a separate CREATE DATABASE step
Encryption at rest	Enabled	Free, and pairs with the TLS in transit enforced by the engine
Enhanced Monitoring / Performance Insights	Disabled	Both are billable beyond minimal usage
Automated backups	Enabled, 1 da	Provides point-in-time recovery independent of the manual S3 backups

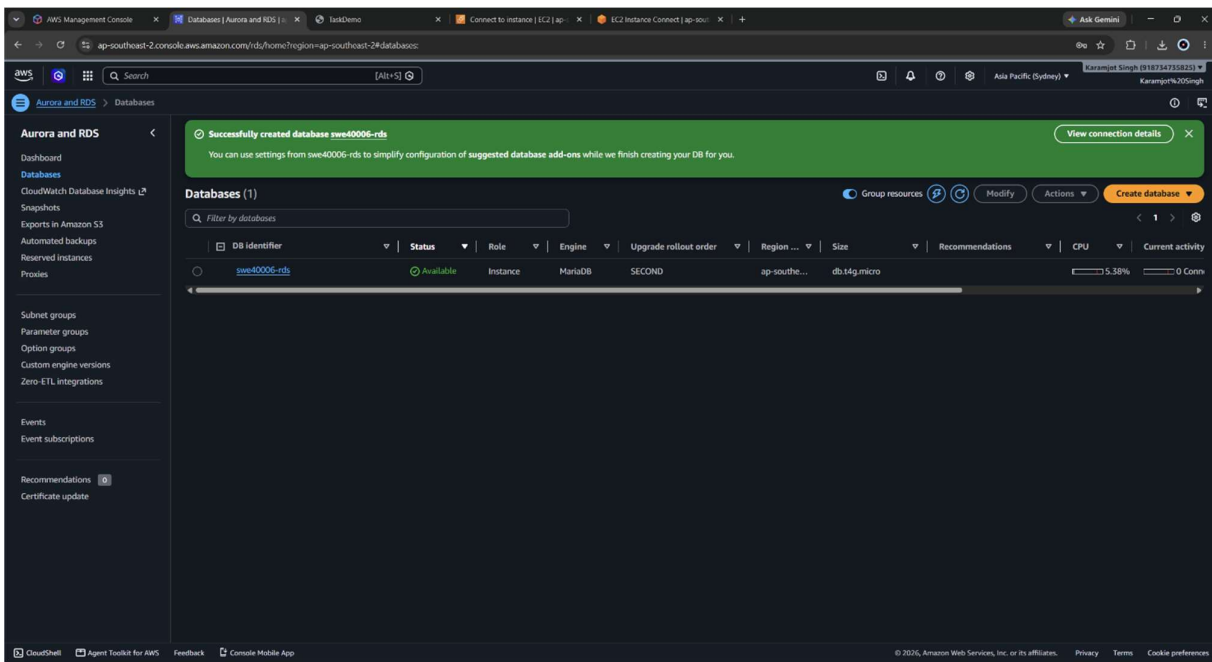


Figure 19: RDS Databases console showing swe40006-rds with status Available.

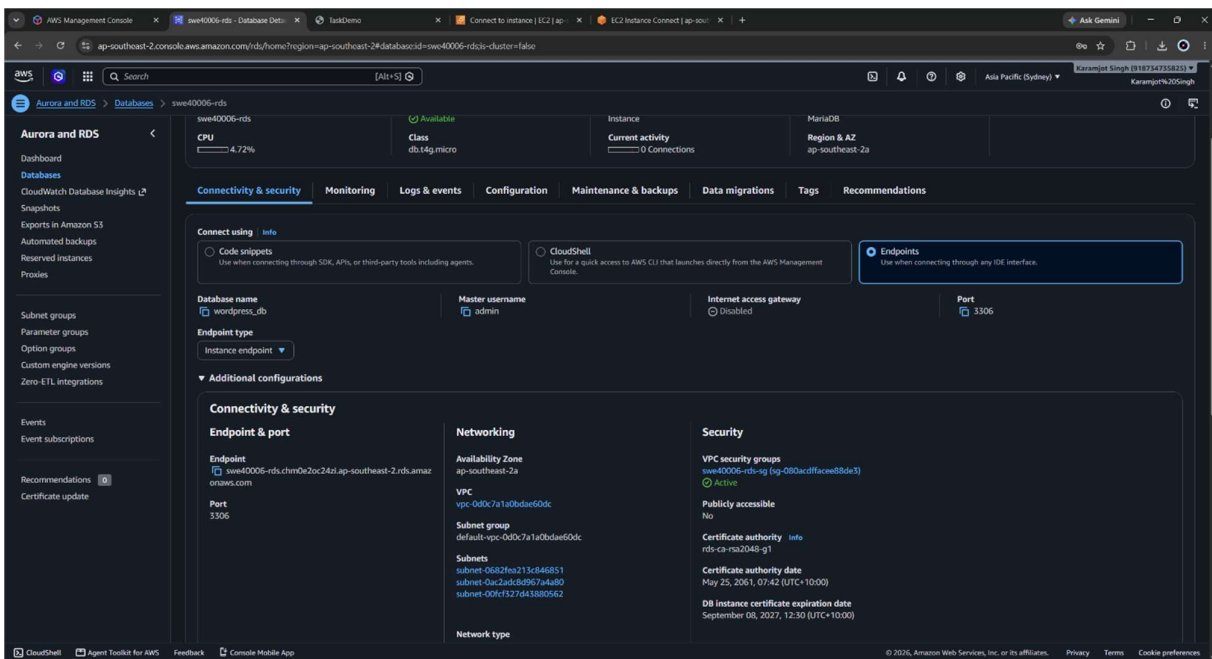


Figure 20: RDS Connectivity & security panel showing the endpoint, port 3306, Publicly accessible: No, and the attached security group.

4.3 Database Export

The local database was exported using mysqldump before any change was made to the application:

```
sudo mysqldump -u root \
  --single-transaction --routines --triggers \
  wordpress_db > /home/ec2-user/wordpress_db_backup.sql

sudo chown ec2-user:ec2-user /home/ec2-user/wordpress_db_backup.sql
ls -lh /home/ec2-user/wordpress_db_backup.sql
grep -c "CREATE TABLE" /home/ec2-user/wordpress_db_backup.sql
```

The flags were chosen deliberately. --single-transaction takes a consistent snapshot inside a transaction rather than locking tables, so the running site is not interrupted. --routines and --triggers include stored programs that a default dump omits. The --databases flag was deliberately not used: it would embed CREATE DATABASE and USE statements in the dump, which conflicts with the database RDS had already created, and it removes the ability to choose the target schema at import time.

The dump was verified before proceeding, by file size and by counting CREATE TABLE statements. Because the shell redirection operator creates the output file whether or not mysqldump succeeds, a zero-byte file is a realistic failure mode that produces no error message; checking the size is the only reliable confirmation.

```

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [(none)]> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'Password123!';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Type
[ec2-user@ip-172-31-40-181 ~]$ cd /tmp
[ec2-user@ip-172-31-40-181 tmp]$ wget https://wordpress.org/latest.tar.gz
2026-09-08 01:56:18 -- https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 66.4.42.252, 54.231.109.100a1:4206:2afc
Connecting to wordpress.org (wordpress.org) [66.4.42.252]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35356041 (34M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz                                     100%[----->] 33.72M  10.8MB/s   in 3.3s

2026-09-08 01:56:18 (10.2 MB/s) - 'latest.tar.gz' saved [35356041/35356041]

[ec2-user@ip-172-31-40-181 tmp]$ tar -xzf latest.tar.gz
[ec2-user@ip-172-31-40-181 tmp]$ sudo cp -r wordpress/* /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$ sudo chown -R apache:apache /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$ sudo chmod -R 755 /var/www/html/
[ec2-user@ip-172-31-40-181 tmp]$ sudo grep -R "rm .*/bin/daemon .*/bin/daemon" /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress_db' );
define( 'DB_USER', 'wp_user' );
define( 'DB_HOST', 'localhost' );
[ec2-user@ip-172-31-40-181 tmp]$ RDS=swe40006-rds.abcl23xyz.ap-southeast-2.rds.amazonaws.com
[ec2-user@ip-172-31-40-181 tmp]$ sudo mysqldump -u root --single-transaction --routines --triggers wordpress_db > /home/ec2-user/wordpress_db_backup.sql
[ec2-user@ip-172-31-40-181 tmp]$ ls -lh /home/ec2-user/wordpress_db_backup.sql
-rw-r--r-- 1 ec2-user ec2-user 1.6M Sep  8 02:40 /home/ec2-user/wordpress_db_backup.sql
[ec2-user@ip-172-31-40-181 tmp]$ head -5 /home/ec2-user/wordpress_db_backup.sql
/*!999999 - enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
--
Host: localhost    Database: wordpress_db

[ec2-user@ip-172-31-40-181 tmp]$

```

Figure 21: Terminal showing the mysqldump command, the resulting file size, and the CREATE TABLE count confirming a complete export.

4.4 Encrypted Connectivity to RDS

The first connection attempt to RDS was rejected by the engine. This is documented in full in Section 7.2; in summary, current RDS parameter groups set require_secure_transport = ON, and unencrypted client connections are refused. Rather than disable the parameter, the correct client configuration was applied: the regional certificate authority bundle was downloaded and supplied to every client connection.

```

# Download the ap-southeast-2 RDS certificate authority bundle
curl -o /home/ec2-user/rds-ca.pem \
  https://truststore.pki.rds.amazonaws.com/ap-southeast-2/ap-southeast-2-bundle.pem

# Session variables reused by every RDS command
RDS= swe40006-rds.chm0e2oc24zi.ap-southeast-2.rds.amazonaws.com
SSL="--ssl-ca=/home/ec2-user/rds-ca.pem --ssl-verify-server-cert"

mysql -h $RDS -P 3306 -u admin -p $SSL

```

Encryption was then confirmed from inside the session rather than assumed:

```

SHOW STATUS LIKE 'Ssl_cipher';
SELECT @@hostname, @@version, @@version_comment;
SHOW DATABASES;

```

A non-empty `Ssl_cipher` value confirms the session is encrypted. The `@@hostname` query was included as a deliberate control: it returns an RDS-internal hostname rather than the EC2 instance hostname, proving the session terminates on the remote database server and not on the local MariaDB instance still running at that point.

```

-- Table structure for table 'wp_comments'
(ec2-user@ip-172-31-40-181 tmp)$ grep -c "CREATE TABLE" /home/ec2-user/wordpress_db_backup.sql
12
(ec2-user@ip-172-31-40-181 tmp)$ mysql -h $RDS -P 3306 -u admin -p $SSL
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 69
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/
Copyright (c) 2009, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\o' to clear the current input statement.

MariaDB [(none)]> SHOW STATUS LIKE 'ssl_cipher';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Ssl_cipher    | TLS_AES_256_GCM_SHA384 |
+-----+-----+
1 row in set (0.004 sec)

MariaDB [(none)]> SELECT @@hostname, @@version, @@version_comment;
+-----+-----+-----+
| @@hostname | @@version | @@version_comment |
+-----+-----+-----+
| ip-10-6-0-49 | 11.8.8-MariaDB-log | managed by https://aws.amazon.com/rds/ |
+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
| wordpress_db |
+-----+
6 rows in set (0.008 sec)

MariaDB [(none)]>

```

i-012d9bca683a20317 (SWE40006-WebServer-Pass)
PublicIP: 54.252.253.3 PrivateIP: 172.31.40.181

Figure 22: MariaDB session against the RDS endpoint showing a populated `Ssl_cipher` value, the RDS hostname and engine version, and the database list.

4.5 Application User and Least Privilege

```

CREATE USER 'wp_user'@'%' IDENTIFIED BY '<password>';
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'%';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'wp_user'@'%';

```

Two decisions are embedded in these statements. The host component changed from `'localhost'` to `'%'` because the client is now a different machine from the database server; a `localhost`-scoped account could never authenticate remotely. Restricting the source network is deliberately left to the security group, which is the correct layer for network authorisation and is enforced before the connection reaches the database engine at all.

The grant is scoped to `wordpress_db.*` rather than `*.*`. The credentials are stored in plain text in `wp-config.php` on an internet-facing web server, so they must be treated as compromisable. Scoping them to a single schema limits the blast radius of a web-tier compromise to the WordPress data alone.

```

AWS Management Console | ssw40006-edt - Database Det... | taskDemo | Connect to instance | EC2 | ap-... | EC2 Instance Connect | ap-sou-...
ap-south-east-2.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressfamily=ipv4&connectType=standard&instanceId=i-012d9bca683a20317&hostUser=ec2-user&region=ap-southeast-2&sshPort=22

MariaDB [(none)]> SELECT @@hostname, @@version, @@version_comment;
+-----+-----+-----+
| @@hostname | @@version | @@version_comment |
+-----+-----+-----+
| ip-10-6-0-49 | 11.8.8-MariaDB-log | managed by https://aws.amazon.com/rds/ |
+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
| wordpress_db |
+-----+
6 rows in set (0.008 sec)

MariaDB [(none)]> CREATE USER 'wp_user'@'%' IDENTIFIED BY 'Wp8da20261';
Query OK, 0 rows affected (0.028 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'%;
Query OK, 0 rows affected (0.006 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.009 sec)

MariaDB [(none)]> SELECT User, Host FROM mysql.user WHERE User = 'wp_user';
+-----+-----+
| User | Host |
+-----+-----+
| wp_user | % |
+-----+-----+
1 row in set (0.002 sec)

MariaDB [(none)]> SHOW GRANTS FOR 'wp_user'@'%';
+-----+
| Grants for wp_user@% |
+-----+
| GRANT USAGE ON *.* TO 'wp_user'@'%' IDENTIFIED BY PASSWORD '**91E1F156C8B095B023BA21E4B814773A8B897DA' |
| GRANT ALL PRIVILEGES ON 'wordpress_db'.* TO 'wp_user'@'%' |
+-----+
2 rows in set (0.001 sec)

MariaDB [(none)]> EXIT;

```

Figure 23: Output of `SHOW GRANTS` confirming `wp_user` privileges are scoped to `wordpress_db` only, not to all schemas.

4.6 Import and Verification

```
mysql -h $RDS -u admin -p $SSL wordpress_db < /home/ec2-user/wordpress_db_backup.sql
echo "Exit code: $?"
```

```
# Verified as the application user, not the master user
mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SHOW TABLES;"
mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; \
  SELECT ID, post_title FROM wp_posts WHERE post_status='publish'; \
  SELECT option_value FROM wp_options WHERE option_name='siteurl';"
```

Verification was performed as `wp_user` rather than the admin master user. This is not a cosmetic difference: it exercises the exact credential path the application will use, so a privilege or host-matching error surfaces here rather than as an opaque "Error establishing a database connection" after the cutover. The `siteurl` value was also captured at this point, because it is stored in the database and later causes a redirect behaviour that is analysed in Section 7.5.

```

[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SHOW TABLES;"
Enter password:
ERROR 1045 (28000): Access denied for user 'wp_user'@'172.31.40.181' (using password: YES)
[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SHOW TABLES;"
Enter password:
+-----+
| Tables in wordpress_db |
+-----+
| wp_comments            |
| wp_links               |
| wp_options             |
| wp_posts               |
| wp_term_relationships  |
| wp_term_taxonomy       |
| wp_termmeta            |
| wp_users               |
+-----+

[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_status FROM wp_posts WHERE post_status='publish'; SELECT COUNT(*) AS total_posts FROM wp_posts; SELECT option_value AS site_url FROM wp_options WHERE option_name='siteurl';"
Enter password:
ERROR 1045 (28000): Access denied for user 'wp_user'@'172.31.40.181' (using password: YES)
[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_status FROM wp_posts WHERE post_status='publish'; SELECT COUNT(*) AS total_posts FROM wp_posts; SELECT option_value AS site_url FROM wp_options WHERE option_name='siteurl';"
Enter password:
+----+-----+-----+
| ID | post_title | post_status |
+----+-----+-----+
| 1 | Hello world! | publish |
| 2 | Sample Page | publish |
| 4 | Navigation | publish |
| 7 | Post created on local Mariain - 8 Sep 2026 | publish |
| 8 | Custom Styles | publish |
+----+-----+-----+
+-----+
| total_posts |
+-----+
| 5 |
+-----+
+-----+
| site_url |
+-----+
| http://54.252.253.3 |
+-----+

[ec2-user@ip-172-31-40-181 ~]$

```

i-012d9bca683a20317 (SWE40006-WebServer-Pass)
PublicIP: 54.252.253.3 PrivateIP: 172.31.40.181

Figure 24: Query results from the RDS endpoint showing the migrated `wp_` tables and the identifiable post title, confirming the data transferred intact.

4.7 Application Cutover

`wp-config.php` was backed up before modification, providing an immediate rollback path, and then edited to point at RDS:

```

sudo cp /var/www/html/wp-config.php /home/ec2-user/wp-config.php.bak
sudo nano /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress_db' );
define( 'DB_USER', 'wp_user' );
define( 'DB_PASSWORD', '██████████' );
define( 'DB_HOST', 'swe40006-rds.chm0e2oc24zi.ap-southeast-2.rds.amazonaws.com' );
define( 'MYSQL_CLIENT_FLAGS', MYSQLI_CLIENT_SSL );

```

The `MYSQL_CLIENT_FLAGS` constant is required and is not part of a default WordPress configuration.

WordPress connects through PHP's `mysqli` extension, which does not negotiate TLS by default; without this constant the application is rejected by `require_secure_transport` with the same error encountered on the command line, presented to the user as a generic database connection failure. Adding it to the configuration file also means the setting is inherited by every instance later launched from the golden AMI, with no per-instance configuration required.

```

wp_posts
wp_term_relationships
wp_term_taxonomy
wp_termmeta
wp_terms
wp_usermeta
wp_users

[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_status FROM wp_posts WHERE post_status='publish'; SELECT COUNT(*) AS total_posts FROM wp_posts; SELECT option_value AS site_url FROM wp_options WHERE option_name='siteurl';"
Enter password:
ERROR 1045 (28000): Access denied for user 'wp_user'@'172.31.40.181' (using password: YES)
[ec2-user@ip-172-31-40-181 ~]$ mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_status FROM wp_posts WHERE post_status='publish'; SELECT COUNT(*) AS total_posts FROM wp_posts; SELECT option_value AS site_url FROM wp_options WHERE option_name='siteurl';"
Enter password:

+----+-----+-----+
| ID | post_title | post_status |
+----+-----+-----+
| 1 | Hello world! | publish |
| 2 | Sample Page | publish |
| 4 | Navigation | publish |
| 7 | Post created on local MariaDB - 8 Sep 2026 | publish |
| 8 | Custom Styles | publish |
+----+-----+-----+

+-----+
| total_posts |
+-----+
| 5 |
+-----+

+-----+
| site_url |
+-----+
| http://54.252.253.3 |
+-----+

[ec2-user@ip-172-31-40-181 ~]$ sudo cp /var/www/html/wp-config.php /home/ec2-user/wp-config.php.bak
[ec2-user@ip-172-31-40-181 ~]$ sudo cp /var/www/html/wp-config.php /home/ec2-user/wp-config.php.bak
[ec2-user@ip-172-31-40-181 ~]$ sudo nano /var/www/html/wp-config.php
[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl restart httpd
[ec2-user@ip-172-31-40-181 ~]$

i-012d9bca683a20317 [SWE40006-WebServer-Pass]
PublicIP: 54.252.253.3 PrivateIP: 172.31.40.181

```

Figure 25: wp-config.php showing DB_HOST set to the RDS endpoint and the MYSQL_CLIENT_FLAGS constant present. (Password redacted)

4.8 Proving the Decoupling

A configuration change alone does not prove the database was decoupled - the application could still be reading from the local server. The local MariaDB service was therefore stopped and disabled, and the site retested:

```

sudo systemctl stop mariadb
sudo systemctl disable mariadb
sudo systemctl status mariadb --no-pager

```

The site continued to serve normally with the local database service inactive. This is the definitive evidence for Credit level: the application is demonstrably no longer dependent on any database process running on the instance. The ordering was deliberate - the local service was stopped only after the RDS connection had been confirmed working, so that a rollback remained available throughout.

```

[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl restart httpd
[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl stop mariadb
[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl disable mariadb
Removed /etc/systemd/system/mariadb.service.
Removed /etc/systemd/system/mariadb.service.
Removed /etc/systemd/system/mariadb.service.
[ec2-user@ip-172-31-40-181 ~]$ sudo systemctl status mariadb --no-pager
mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/etc/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/

Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: Please report any problems at https://mariadb.org/
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: The latest information about MariaDB is available at https://mariadb.org/.
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: Consider joining MariaDB's strong and vibrant community:
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: https://mariadb.org/get-involved/
Sep 08 01:53:33 ip-172-31-40-181.ap-southeast-2.compute.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
Sep 08 03:11:12 ip-172-31-40-181.ap-southeast-2.compute.internal systemd[1]: Stopping mariadb.service - MariaDB 10.5 database server...
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal systemd[1]: mariadb.service: Deactivated successfully.
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal systemd[1]: Stopped mariadb.service - MariaDB 10.5 database server.
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal systemd[1]: mariadb.service: Consumed 1.851s CPU time.
[ec2-user@ip-172-31-40-181 ~]$

i-012d9bca683a20317 [SWE40006-WebServer-Pass]
PublicIP: 54.252.253.3 PrivateIP: 172.31.40.181

```

Figure 26: systemctl status showing mariadb as inactive (dead) alongside the WordPress site loading successfully in the browser.

A second post was then published through the WordPress admin interface and queried directly from the RDS endpoint, confirming that write operations, not only reads, are being served by RDS:

```

mysql -h $RDS -u wp_user -p $SSL -e "USE wordpress_db; \
SELECT ID, post_title, post_date FROM wp_posts ORDER BY ID DESC LIMIT 3;"

```

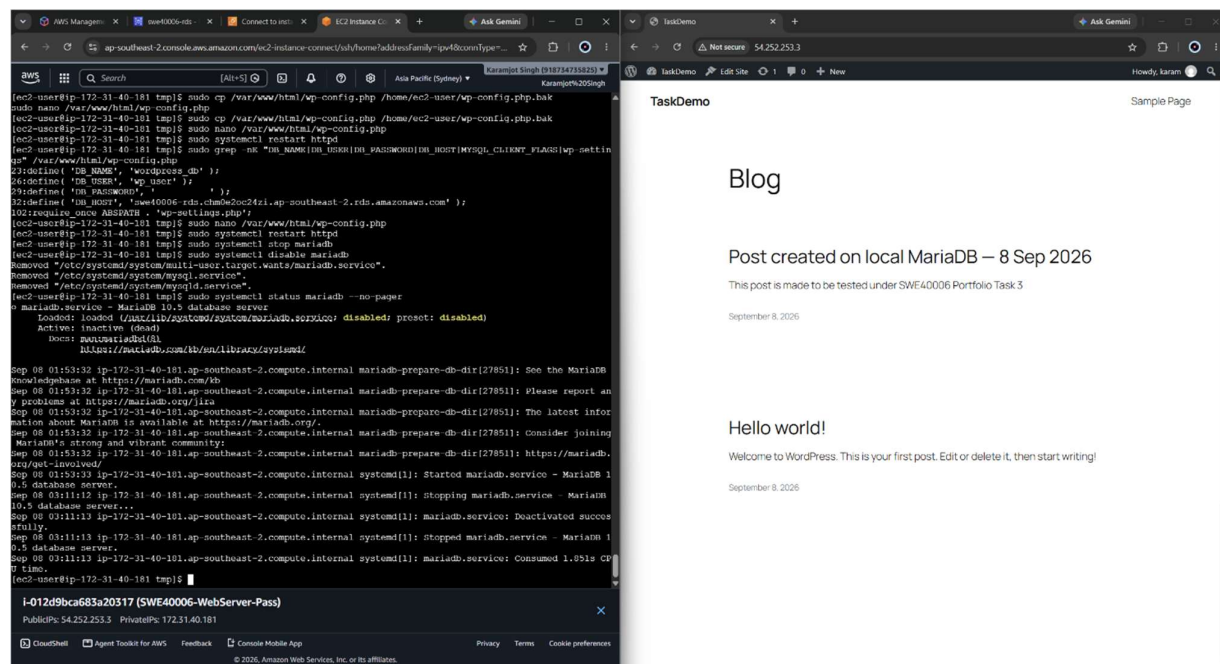


Figure 27: New Post created

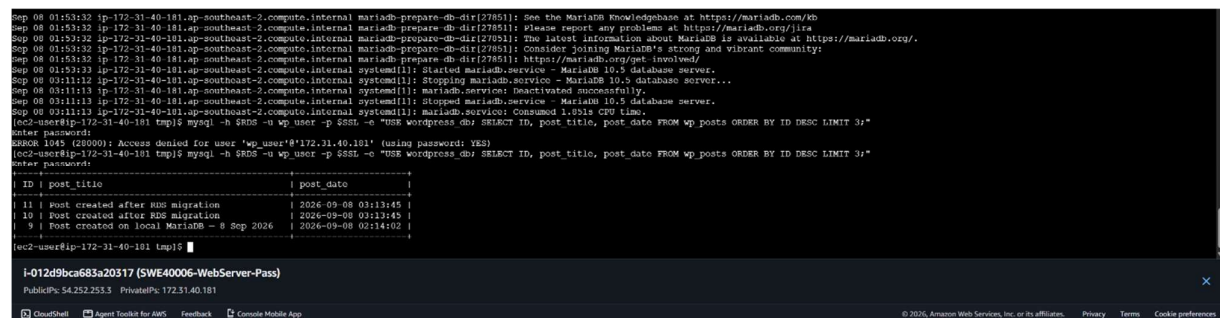


Figure 28: The post created after migration appearing in a query run directly against the RDS endpoint, confirming writes are persisted to RDS.

4.9 S3 Bucket Configuration

An S3 bucket, swe40006-backup-104809887, was created in the same region as the compute resources. Two non-default settings were applied:

- Block all public access remained enabled. The bucket holds a database dump containing WordPress user records and password hashes, plus an application archive containing wp-config.php with live database credentials. Public exposure would be a credential disclosure, not merely a data leak.
- Bucket versioning was enabled. Versioning converts an overwrite or deletion into a recoverable event, which materially changes the value of the backup: without it, a corrupted backup uploaded over a good one destroys the only copy.

Default SSE-S3 server-side encryption applies, giving encryption at rest for the backup artefacts consistent with the encryption applied to the RDS storage.

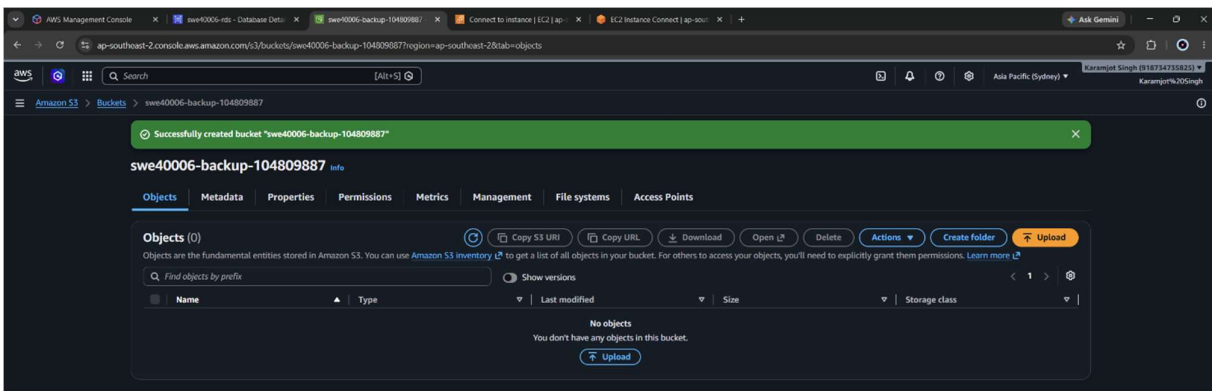


Figure 29: S3 Bucket created

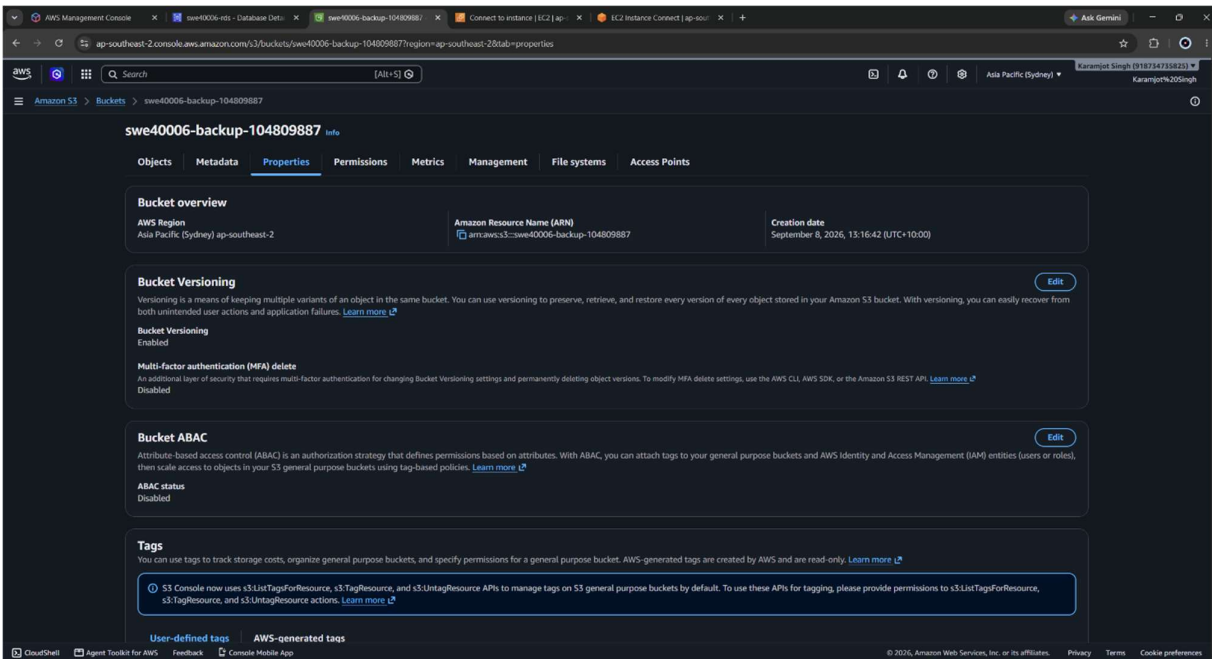


Figure 30: S3 Bucket Properties – bucket versioning on

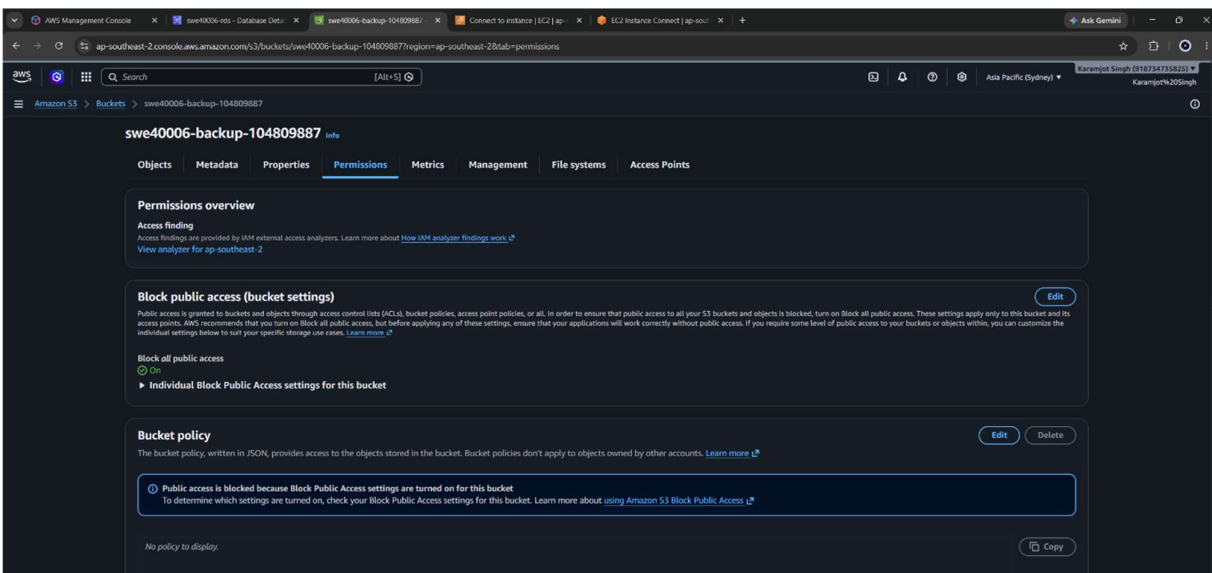


Figure 31: S3 Bucket Permissions blocking public access

4.10 IAM Role Rather Than Access Keys

Granting the instance access to S3 could have been achieved by running `aws configure` with an IAM user's access key and secret. That approach was rejected. Long-lived credentials written to disk on a web server are recoverable by anyone who obtains file access, do not rotate, and are inherited by every AMI and every instance subsequently launched from it - meaning a single key compromise would propagate across the entire Auto Scaling Group.

An IAM role was used instead. A custom least-privilege policy was authored rather than attaching the managed `AmazonS3FullAccess` policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListTargetBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3::swe40006-backup-104809887"
    },
    {
      "Sid": "ReadWriteBackupObjects",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3::swe40006-backup-104809887/*"
    }
  ]
}
```

The policy grants three actions against one bucket. It cannot delete objects, cannot access any other bucket in the account, and cannot enumerate the account's buckets. Credentials are delivered through the instance metadata service and rotated automatically by AWS, so nothing persistent is written to the filesystem. This role is extended rather than replaced at High Distinction level, when the SSM managed policy is attached to the same role.

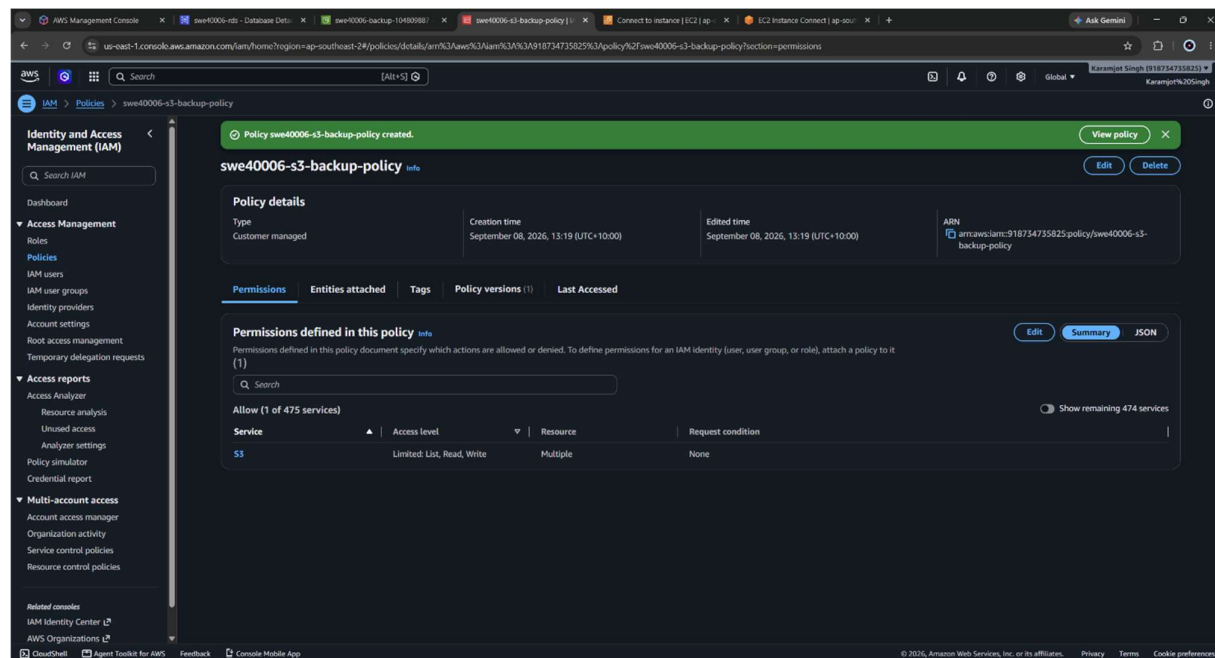


Figure 32: : IAM role `swe40006-ec2-s3-role` showing the attached least-privilege S3 policy.

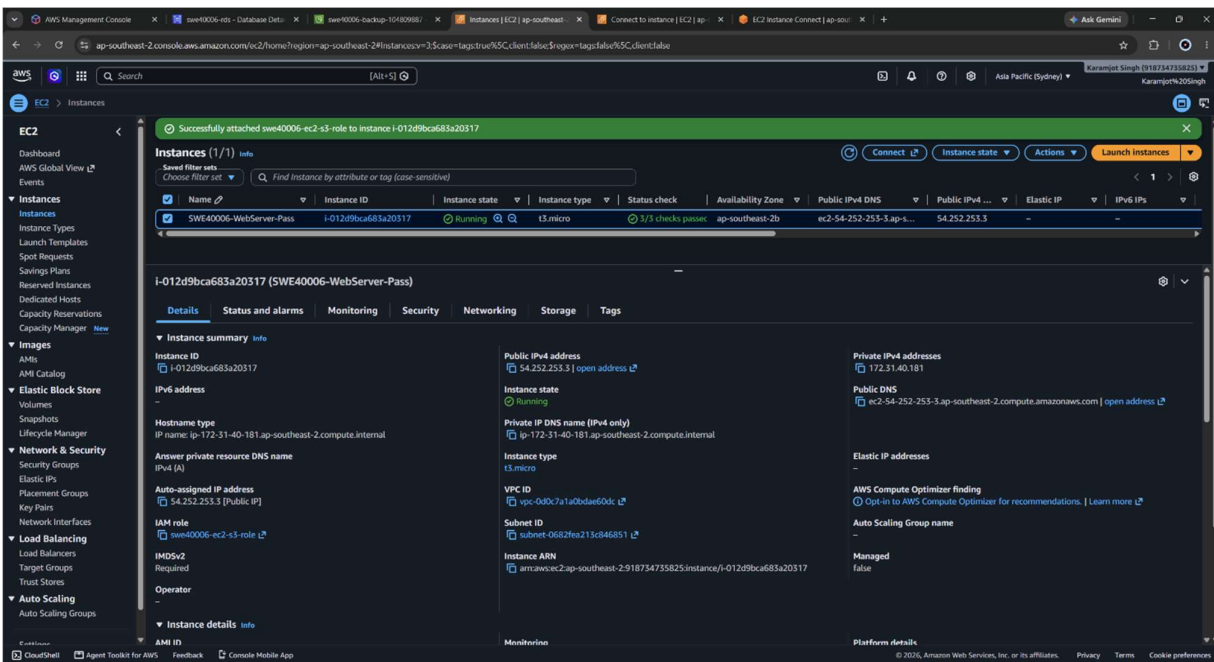


Figure 33: EC2 instance Details tab confirming the IAM role is attached to the running instance.

4.11 Backup Execution

```
BUCKET=s3://swe40006-backup-104809887
DATE=$(date +%F)
```

```
# Application files, including themes, plugins, uploads and wp-config.php
sudo tar -czf /home/ec2-user/wordpress-files-$DATE.tar.gz -C /var/www/html .
```

```
# Database dump, now taken from RDS over TLS
mysqldump -h $RDS -u wp_user -p $SSL --single-transaction \
  wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
```

```
# Confirm the role, not static keys, is providing credentials
aws sts get-caller-identity
```

```
aws s3 cp /home/ec2-user/wordpress-files-$DATE.tar.gz $BUCKET/backups/
aws s3 cp /home/ec2-user/wordpress-db-$DATE.sql $BUCKET/backups/
aws s3 ls $BUCKET/backups/ --human-readable
```

The `aws sts get-caller-identity` call returns an assumed-role ARN rather than an IAM user ARN, which is the verification that the role-based credential path is genuinely in use. Two artefacts were produced and uploaded:

Object	Size	Contents
wordpress-files-2026-09-08.tar.gz	34.6 MiB	Full document root: WordPress core, themes, plugins, uploads, wp-config.php
wordpress-db-2026-09-08.sql	1.5 MiB	Complete logical dump of <code>wordpress_db</code> taken from RDS

```

aws
[Alt+S]
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: The latest information about MariaDB is available at https://mariadb.org/.
Sep 08 01:53:32 ip-172-31-40-181.ap-southeast-2.compute.internal mariadb-prepare-db-dir[27851]: Consider joining MariaDB's strong and vibrant community:
Sep 08 01:53:33 ip-172-31-40-181.ap-southeast-2.compute.internal system[1]: Started mariadb.service - MariaDB 10.5 database server.
Sep 08 03:11:12 ip-172-31-40-181.ap-southeast-2.compute.internal system[1]: Stopping mariadb.service - MariaDB 10.5 database server...
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal system[1]: mariadb.service: Deactivated successfully.
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal system[1]: Stopped mariadb.service - MariaDB 10.5 database server.
Sep 08 03:11:13 ip-172-31-40-181.ap-southeast-2.compute.internal system[1]: mariadb.service: Consumed 1.851s CPU time.
[ec2-user@ip-172-31-40-181 tmp]$ mysql -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_date FROM wp_posts ORDER BY ID DESC LIMIT 3;"
Enter password:
ERROR 1045 (28000): Access denied for user 'wp_user'@'172.31.40.181' (using password: YES)
[ec2-user@ip-172-31-40-181 tmp]$ mysql -u wp_user -p $SSL -e "USE wordpress_db; SELECT ID, post_title, post_date FROM wp_posts ORDER BY ID DESC LIMIT 3;"
Enter password:
+----+-----+-----+
| ID | post_title | post_date |
+----+-----+-----+
| 11 | Post created after RDS migration | 2026-09-08 03:13:45 |
| 10 | Post created after RDS migration | 2026-09-08 03:13:45 |
| 9 | Post created on local Mariadb | 2026-09-08 02:14:02 |
+----+-----+-----+
[ec2-user@ip-172-31-40-181 tmp]$ BUCKET=s3://swe40006-backup-104809887
[ec2-user@ip-172-31-40-181 tmp]$ DATE=$(date +%F)
[ec2-user@ip-172-31-40-181 tmp]$ sudo tar czf /home/ec2-user/wordpress-files-$DATE.tar.gz -C /var/www/html .
[ec2-user@ip-172-31-40-181 tmp]$ mysqldump -h $RDS -u wp_user -p --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
mysqldump: Got error: 3159: "Connections using insecure transport are prohibited while --require_secure_transport=ON." when trying to connect
[ec2-user@ip-172-31-40-181 tmp]$ mysqldump -h $RDS -u wp_user -p --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
[ec2-user@ip-172-31-40-181 tmp]$ mysqldump -h $RDS -u wp_user -p --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
mysqldump: Got error: 3159: "Connections using insecure transport are prohibited while --require_secure_transport=ON." when trying to connect
[ec2-user@ip-172-31-40-181 tmp]$ mysqldump -h $RDS -u wp_user -p $SSL --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
[ec2-user@ip-172-31-40-181 tmp]$ aws sts get-caller-identity
{
  "UserId": "AKIA3L2G3HITUWQWJLIQI-012D9BCA63A20317",
  "Account": "918734735825",
  "Arn": "arn:aws:iam::918734735825:assumed-role/swe40006-ec2-s3-role/i-012d9bca63a20317"
}
[ec2-user@ip-172-31-40-181 tmp]$ aws s3 ls $BUCKET
[ec2-user@ip-172-31-40-181 tmp]$ aws s3 cp /home/ec2-user/wordpress-files-$DATE.tar.gz $BUCKET/backups/
upload: ./home/ec2-user/wordpress-files-2026-09-08.tar.gz to s3://swe40006-backup-104809887/backups/wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-40-181 tmp]$ aws s3 cp /home/ec2-user/wordpress-db-$DATE.sql $BUCKET/backups/
upload: ./home/ec2-user/wordpress-db-2026-09-08.sql to s3://swe40006-backup-104809887/backups/wordpress-db-2026-09-08.sql
[ec2-user@ip-172-31-40-181 tmp]$ aws s3 ls $BUCKET/backups/ --human-readable
2026-09-08 03:27:31 1.5 MiB wordpress-db-2026-09-08.sql
2026-09-08 03:27:20 34.6 MiB wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-40-181 tmp]$

```

Figure 34: Terminal showing tar and mysqldump producing both backup artefacts with non-zero sizes.

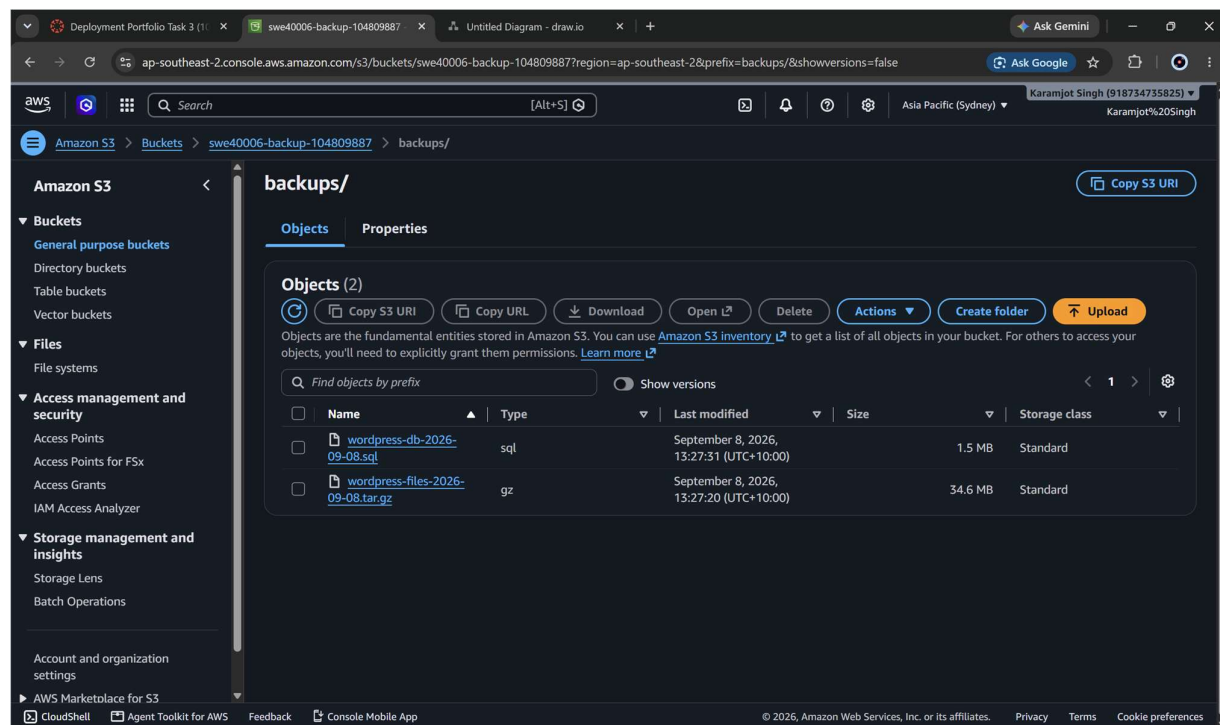


Figure 35: S3 console showing both backup objects in the backups/

4.12 Restore Demonstration

The task requires demonstrating restoration to an EC2 instance. Restoring onto the source instance would have proved little, since the files were already present there. A second instance, SWE40006-WebServer-Restore, was launched instead - a genuine bare-metal recovery scenario, and one that also validated the provisioning script later reused in the Task 3.3 launch template.

The instance was launched from a clean Amazon Linux 2023 AMI with the same security group and the same IAM role, and provisioned automatically at boot using user data:

```
#!/bin/bash
dnf update -y
dnf install -y httpd php php-mysqlnd php-gd php-xml mariadb105
systemctl enable --now httpd
```

The restore itself was performed entirely from S3:

```
BUCKET=s3://swe40006-backup-104809887

aws s3 ls $BUCKET/backups/
aws s3 cp $BUCKET/backups/wordpress-files-2026-09-08.tar.gz /tmp/
ls -lh /tmp/wordpress-files-2026-09-08.tar.gz

sudo tar -xzf /tmp/wordpress-files-2026-09-08.tar.gz -C /var/www/html
sudo chown -R apache:apache /var/www/html
sudo chmod -R 755 /var/www/html
sudo systemctl restart httpd
```

```
# The CA bundle lives outside the document root and is not in the archive
curl -o /home/ec2-user/rds-ca.pem \
https://truststore.pki.rds.amazonaws.com/ap-southeast-2/ap-southeast-2-bundle.pem
```

Because wp-config.php was inside the archive, the restored instance was already configured to reach RDS with TLS enabled - no manual reconfiguration was needed. The second instance served the same content, including the post created after the migration, because both instances read the same external database. This is the practical demonstration that the compute layer had become stateless and interchangeable, which is the precondition for the Auto Scaling Group built in Task 3.3.

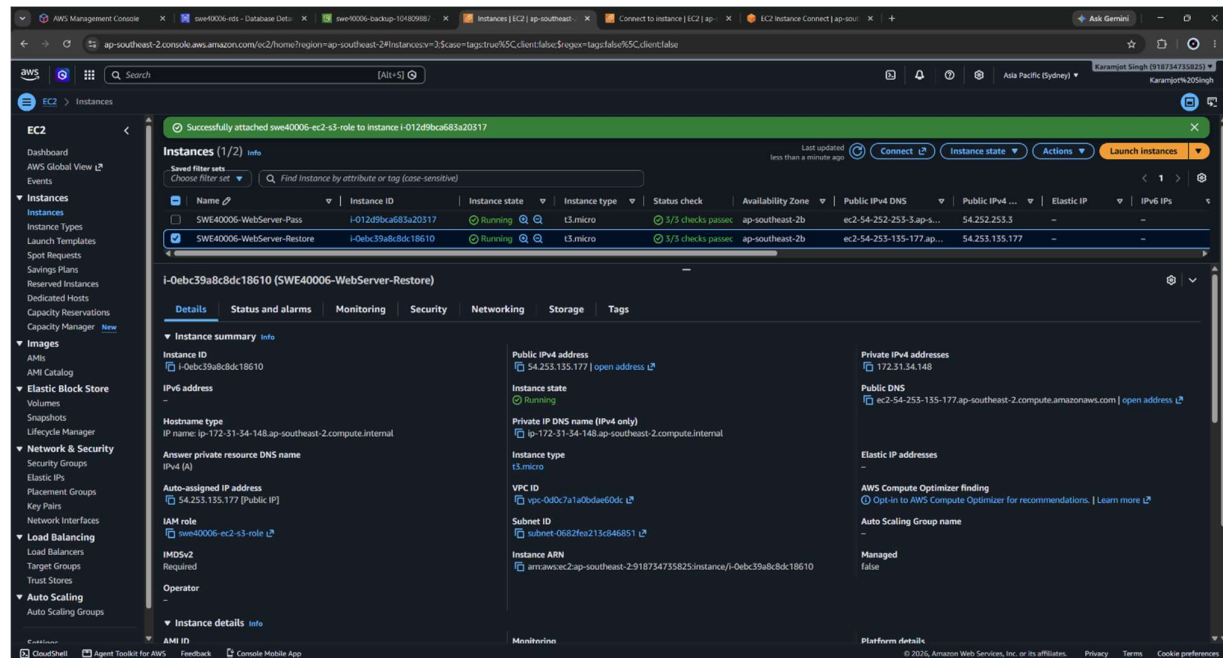


Figure 36: EC2 console showing both instances running, allowing the IP addresses in the preceding figures to be cross-referenced.

```

[ec2-user@ip-172-31-34-148 ~]$ BUCKET=s3://swe40006-backup-10480987
[ec2-user@ip-172-31-34-148 ~]$ aws s3 cp $BUCKET/backups/wordpress-files-2026-09-08.tar.gz /tmp/
download: s3://swe40006-backup-10480987/backups/wordpress-files-2026-09-08.tar.gz to ../../tmp/wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-34-148 ~]$ ls -lh /tmp/wordpress-files-2026-09-08.tar.gz
-rw-r--r-- 1 ec2-user ec2-user 30M Sep  8 03:27 /tmp/wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-34-148 ~]$ sudo tar -xzf /tmp/wordpress-files-2026-09-08.tar.gz -C /var/www/html
[ec2-user@ip-172-31-34-148 ~]$ sudo chown -R apache:apache /var/www/html
[ec2-user@ip-172-31-34-148 ~]$ sudo chmod -R 755 /var/www/html
[ec2-user@ip-172-31-34-148 ~]$ sudo systemctl restart httpd
[ec2-user@ip-172-31-34-148 ~]$ ls -l /var/www/html | head
total 252
-rwxr-xr-x. 1 apache apache 405 Sep  8 01:56 index.php
-rwxr-xr-x. 1 apache apache 10903 Sep  8 01:56 license.txt
-rwxr-xr-x. 1 apache apache 7407 Sep  8 01:56 readme.html
-rwxr-xr-x. 1 apache apache 7710 Sep  8 01:56 wp-activate.php
-rwxr-xr-x. 9 apache apache 16384 Sep  8 01:56 wp-admin
-rwxr-xr-x. 1 apache apache 351 Sep  8 01:56 wp-blog-header.php
-rwxr-xr-x. 1 apache apache 2322 Sep  8 01:56 wp-comments-post.php
-rwxr-xr-x. 1 apache apache 3339 Sep  8 01:56 wp-config-sample.php
-rwxr-xr-x. 1 apache apache 3719 Sep  8 01:09 wp-config.php
[ec2-user@ip-172-31-34-148 ~]$ sudo sed -e "s#DB_HOST#mysql_client_plugins#" /var/www/html/wp-config.php
define( 'DB_HOST', 'swe40006-rds.cnw2oc24zi.ap-southeast-3.rds.amazonaws.com' );
define( 'MYSQL_CLIENT_PLUGIN', MYSQL_CLIENT_PLUGIN );
[ec2-user@ip-172-31-34-148 ~]$

```

i-0ebc39a8c8dc18610 (SWE40006-WebServer-Restore)
Public IPs: 54.253.135.177 Private IPs: 172.31.34.148

Figure 37: Restore in progress on the second instance: aws s3 cp download and tar extraction into the document root.

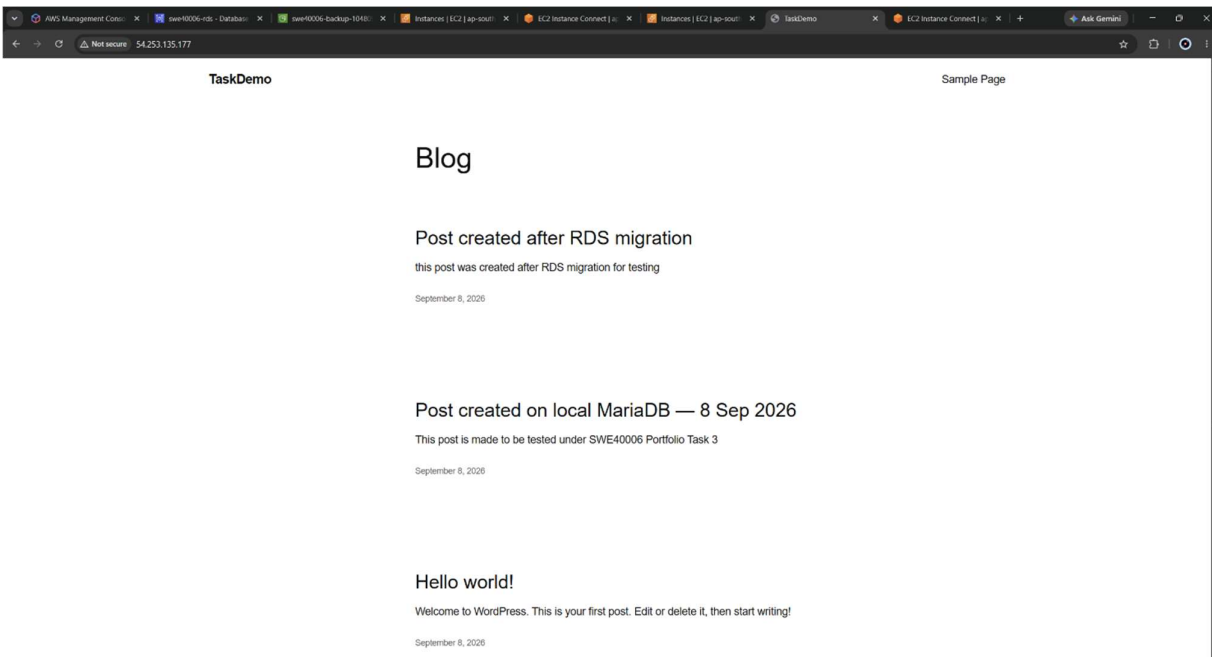


Figure 38: The restored WordPress site served from the second instance public IP, showing content identical to the original.

5. Task 3.3 - Distinction Level: Load Balancing and Auto Scaling

Objective: create an EC2 Launch Template containing the instance configuration and a user data startup script; provision an Application Load Balancer across at least two Availability Zones; and create an Auto Scaling Group attached to the ALB target group with fixed capacity across multiple zones.

5.1 Three-Tier Security Group Refactor

The default launch-wizard-1 group could not support a load-balanced design, because it opened HTTP to the entire internet. Introducing an ALB while leaving instances directly reachable would have created a bypass path around the load balancer. Three purpose-built groups were created:

Security group	Inbound	Source	Tier
swe40006-alb-sg	HTTP 80, HTTPS 443	0.0.0.0/0	Public edge
swe40006-web-sg	HTTP 80	swe40006-alb-sg	Application
swe40006-web-sg	SSH 22 (removed at 3.4)	My IP	Administrative
swe40006-rds-sg	MySQL 3306	swe40006-web-sg	Data

An additional inbound rule was added to swe40006-rds-sg permitting 3306 from swe40006-web-sg, so that Auto Scaling Group instances could reach the database. Omitting this step produces a site that loads its static assets correctly but fails with a database connection error, which is a misleading symptom.

Each rule references the security group of the tier immediately above it. The result is that no instance is reachable from the internet, and the database is reachable only from the application tier - defence in depth expressed as configuration rather than as a firewall appliance.

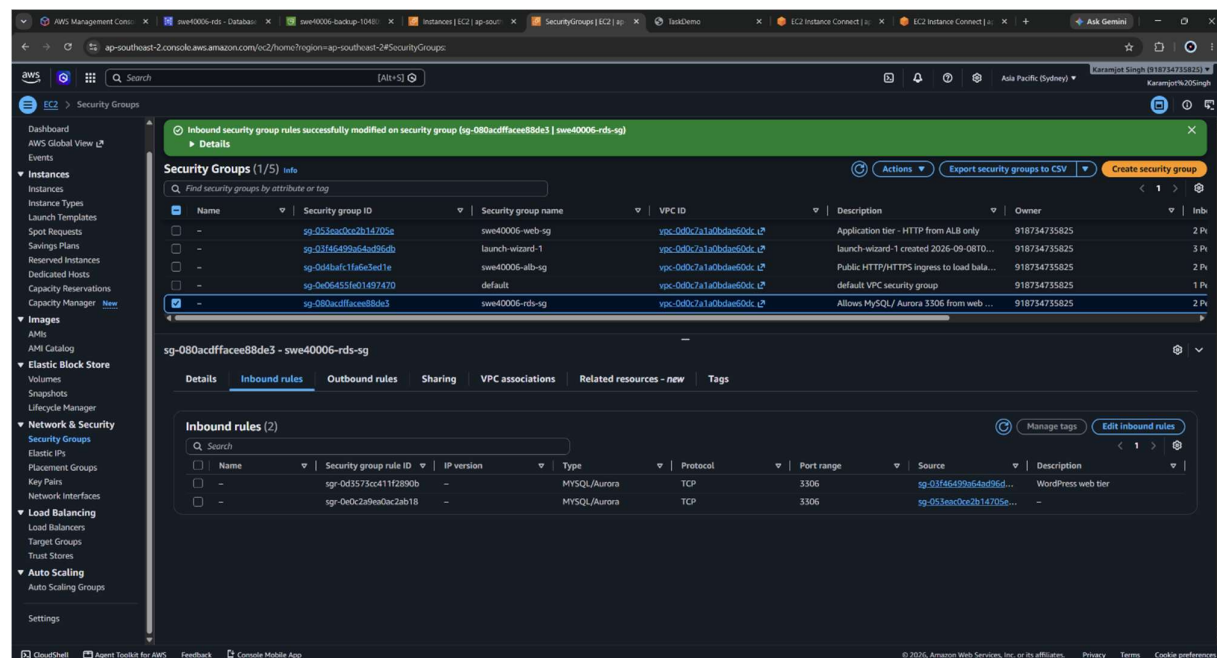


Figure 39: All three security groups listed in the EC2 console.

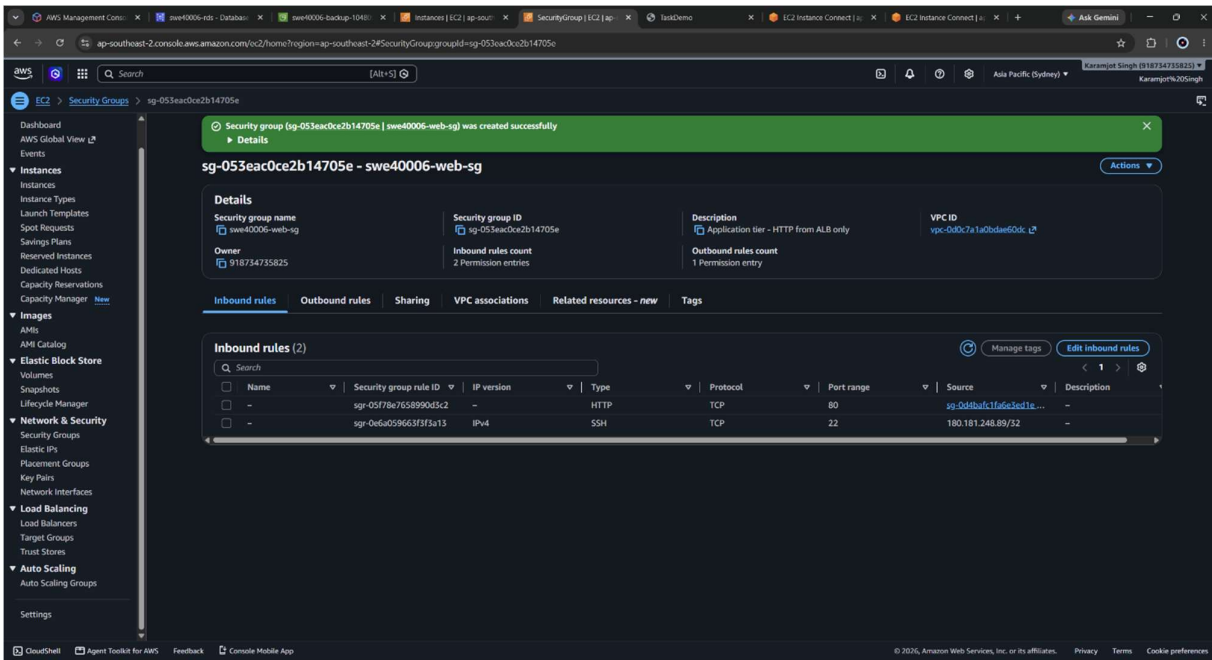


Figure 40: Inbound rules of swe40006-web-sg showing HTTP sourced from the ALB security group rather than from the internet.

5.2 Target Group and Health Check Design

A target group, swe40006-tg, was created for HTTP on port 80 with no targets registered manually. Manual registration was deliberately avoided: the Auto Scaling Group registers and deregisters targets automatically, and a manually registered instance would remain outside the group's management, never replaced on failure and permanently unhealthy once stopped.

The health check path was set to /health.html rather than the default /. This is the most consequential single setting in this section. WordPress responds to a request for / with an HTTP 302 redirect, and the target group's default success code is 200. Left at the default, every target is marked unhealthy, the Auto Scaling Group terminates and replaces them in a loop, and the ALB returns 503 to all traffic. Pointing the health check at a static file returning a flat 200 decouples infrastructure health checking from application routing behaviour.

Health check setting	Value	Rationale
Path	/health.html	Static 200 response, independent of WordPress redirect logic
Healthy threshold	2	Two consecutive successes before receiving traffic
Unhealthy threshold	2	Fast detection of a failed instance
Timeout / Interval	5 s / 10 s	Detects failure within roughly 20 seconds
Success codes	200	Explicitly excludes 3xx responses

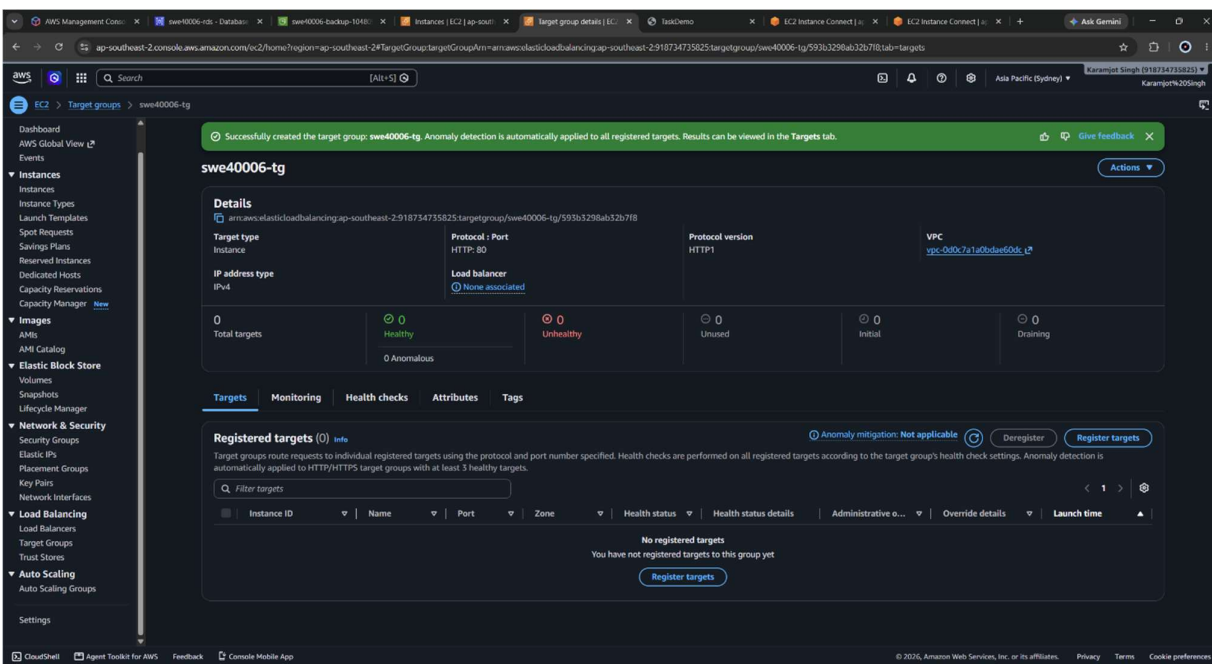


Figure 41: Created target group

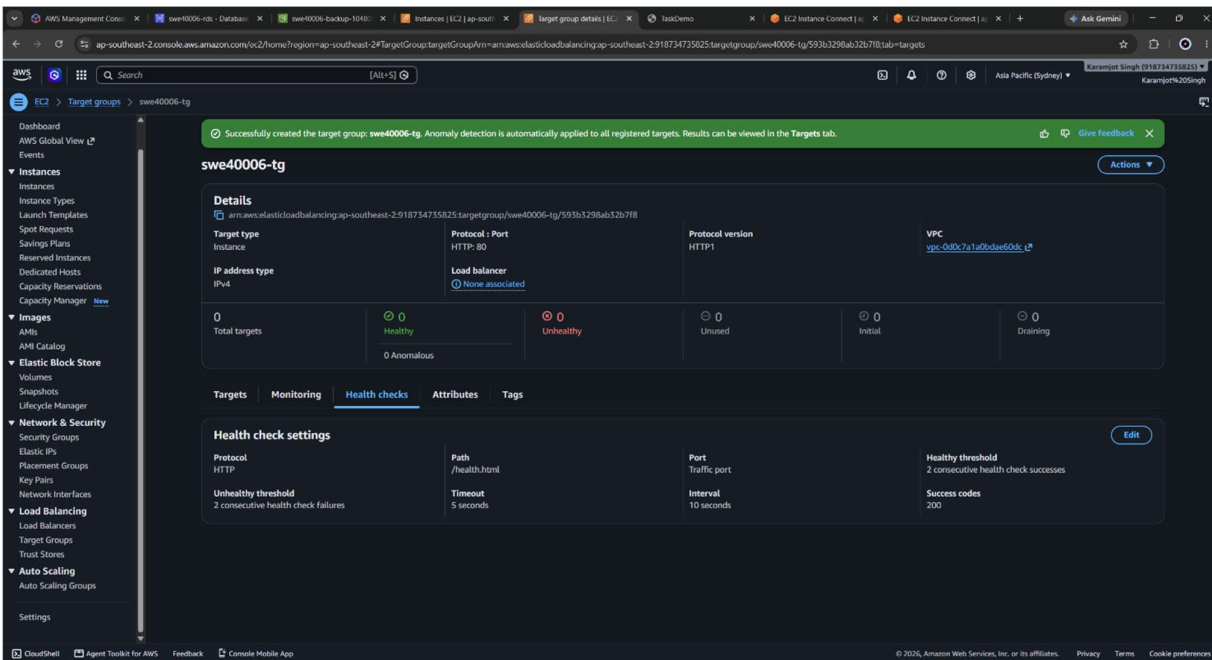


Figure 42: Target group health check configuration showing path /health.html and the threshold settings.

5.3 Application Load Balancer

An internet-facing Application Load Balancer, swe40006-alb, was provisioned across two Availability Zones (ap-southeast-2a and ap-southeast-2b) with a single HTTP listener on port 80 forwarding to swe40006-tg, and attached to swe40006-alb-sg.

Mapping the ALB across two Availability Zones is what makes the multi-zone requirement meaningful. An ALB provisions load balancer nodes in each mapped subnet; with only one zone mapped, an Availability Zone failure would take the load balancer offline regardless of how many instances were running elsewhere.

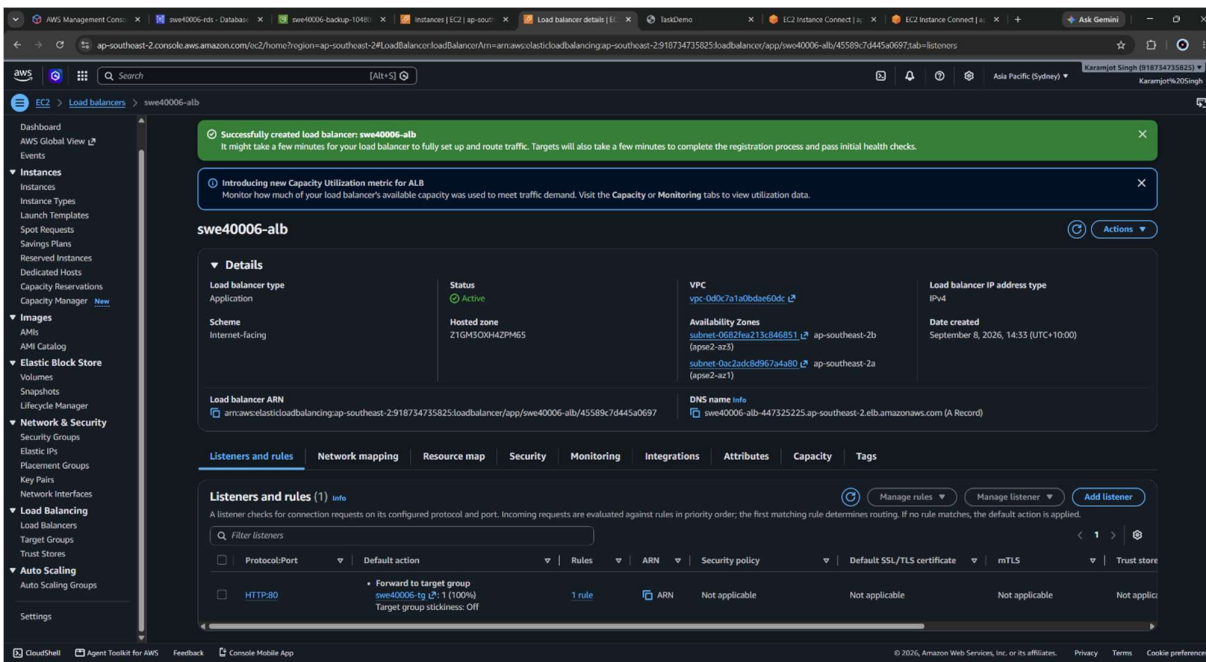


Figure 43: ALB details showing State: Active, internet-facing scheme, and both mapped Availability Zones.

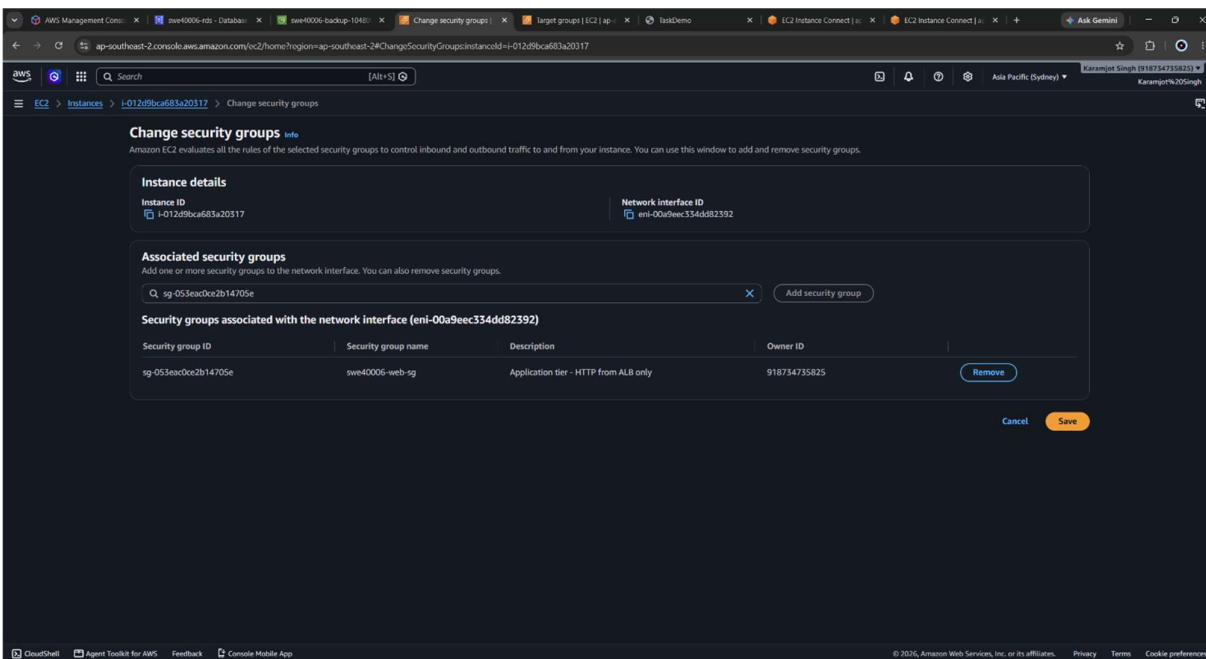


Figure 44: Associate Security Group

5.4 Preparing the Golden AMI

Before imaging, two changes were made to the source instance. The health check endpoint was created, and WordPress was reconfigured to use the ALB DNS name as its canonical address:

```
echo "OK" | sudo tee /var/www/html/health.html
curl -I http://localhost/health.html # expect HTTP/1.1 200 OK
define( 'WP_HOME', 'http://swe40006-alb-447325225.ap-southeast-2.elb.amazonaws.com' );
define( 'WP_SITEURL', 'http://swe40006-alb-447325225.ap-southeast-2.elb.amazonaws.com' );

/* Trust the protocol forwarded by the load balancer */
if ( isset( $_SERVER['HTTP_X_FORWARDED_PROTO'] ) )
```

```

    && $_SERVER['HTTP_X_FORWARDED_PROTO'] === 'https' ) {
    $_SERVER['HTTPS'] = 'on';
}

```

WordPress stores its canonical URL in the database, in the wp_options siteurl and home rows. Because every instance shares one database, a database-stored instance IP would cause all instances to redirect visitors to whichever single instance was configured last. Defining WP_HOME and WP_SITEURL in wp-config.php overrides the stored values at runtime, and because the constants live in the configuration file they are baked into the AMI and inherited identically by every instance the Auto Scaling Group launches.

The X-Forwarded-Proto handling was added in anticipation of TLS termination at the load balancer. When an ALB terminates HTTPS it forwards plain HTTP to the target, so PHP sees an insecure request and WordPress generates http:// URLs, producing mixed-content warnings. The conditional makes the application aware of the original protocol.

```

awscli@ip-172-31-40-101:~$ aws rds instance-connect describe-family=ip-4a2connType=standard3instanceid=i-012d9bca683a203177 --region=ap-southeast-2 --port=22
[ec2-user@ip-172-31-40-101 ~]$ BUCKET=s3://swe40006-backup-104809887
[ec2-user@ip-172-31-40-101 ~]$ DATE=$(date +%F)
[ec2-user@ip-172-31-40-101 ~]$ aws tar -cf /home/ec2-user/wordpress-files-$DATE.tar.gz -C /var/www/html .
[ec2-user@ip-172-31-40-101 ~]$ mysqldump -h $RDS --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
mysqldump: Got error: 3159: "Connections using insecure transport are prohibited while --require_secure_transport=ON." when trying to connect
[ec2-user@ip-172-31-40-101 ~]$ mysqldump -h $RDS -u wp_user -p --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
-bash: $'\E[200-mysqldump': command not found
[ec2-user@ip-172-31-40-101 ~]$ mysqldump -h $RDS -u wp_user -p --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
mysqldump: Got error: 3159: "Connections using insecure transport are prohibited while --require_secure_transport=ON." when trying to connect
[ec2-user@ip-172-31-40-101 ~]$ mysqldump -h $RDS -u wp_user -p SSL --single-transaction wordpress_db > /home/ec2-user/wordpress-db-$DATE.sql
Enter password:
[ec2-user@ip-172-31-40-101 ~]$ aws sts get-caller-identity
{
  "UserId": "AKIA562638H1UQVH3LPQ1:012d9bca683a203177",
  "Account": "918734735825",
  "Arn": "arn:aws:sts:918734735825:assumed-role/swe40006-ec2-s3-role/i-012d9bca683a203177"
}
[ec2-user@ip-172-31-40-101 ~]$ aws s3 ls $BUCKET
upload: ../home/ec2-user/wordpress-files-2026-09-08.tar.gz to s3://swe40006-backup-104809887/backups/wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-40-101 ~]$ aws s3 cp /home/ec2-user/wordpress-db-$DATE.sql $BUCKET/backups/
upload: ../home/ec2-user/wordpress-db-2026-09-08.sql to s3://swe40006-backup-104809887/backups/wordpress-db-2026-09-08.sql
[ec2-user@ip-172-31-40-101 ~]$ aws s3 ls $BUCKET/backups/ --human-readable
2026-09-08 03:27:31    1.5 MiB wordpress-db-2026-09-08.sql
2026-09-08 03:27:20   34.6 MiB wordpress-files-2026-09-08.tar.gz
[ec2-user@ip-172-31-40-101 ~]$ echo "OK" | sudo tee /var/www/html/health.html
OK
[ec2-user@ip-172-31-40-101 ~]$ curl -I http://localhost/health.html
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 04:36:51 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Tue, 08 Sep 2026 04:36:43 GMT
ETag: "7-63a1141dab9"
Accept-Ranges: bytes
Content-Length: 3
Content-Type: text/html; charset=UTF-8

```

Figure 45: Figure 35: curl -I returning HTTP 200 for /health.html (lost the screenshot for wp-config.php constants)

An AMI named swe40006-wordpress-ami was then created from the instance, with the reboot option left enabled to guarantee a filesystem-consistent image.

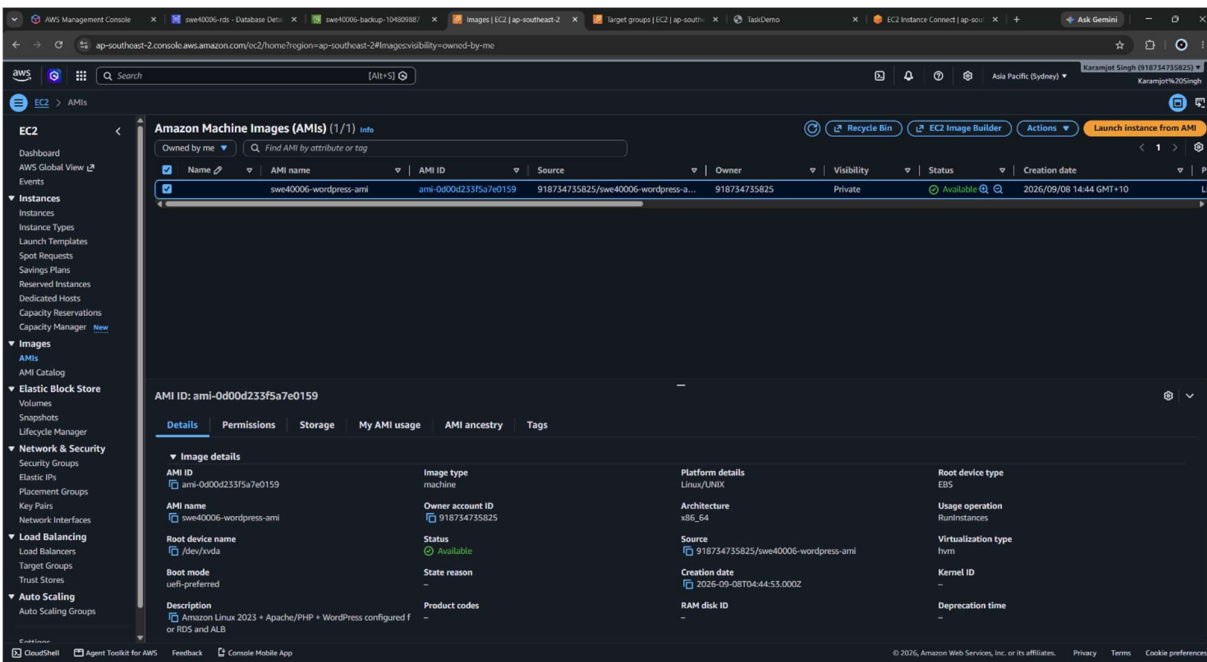


Figure 46: EC2 AMIs console showing swe40006-wordpress-ami showing AMI status Available and its AMI ID.

5.4.1 Golden AMI Versus Boot-Time Provisioning

Two provisioning strategies were available. A minimal base AMI combined with a long user data script would install Apache, PHP and WordPress at every boot. The golden AMI approach bakes the fully configured application into an immutable image.

The golden AMI was chosen. Instances boot in roughly sixty seconds rather than three to five minutes, which matters directly for scaling responsiveness and for how quickly the Auto Scaling Group can replace a failed instance. It also removes a class of failure: a boot-time script that installs packages can fail midway if a repository is unreachable, producing an instance that is running but broken. The trade-off is that the image must be rebuilt for every application change, and the image is a versioned artefact that must be managed. For a deployment of this size the trade-off favours the AMI; for a rapidly changing application it would favour boot-time provisioning or a container image.

5.5 Launch Template

A launch template, swe40006-launch-template, encapsulates the full instance specification so that the Auto Scaling Group can produce identical instances on demand.

Setting	Value
AMI	swe40006-wordpress-ami
Instance type	t3.micro
Security group	swe40006-web-sg
IAM instance profile	swe40006-ec2-s3-role
Subnet	Not specified - controlled by the Auto Scaling Group so it can balance across zones
Metadata version	IMDSv2 (token required)

The subnet was deliberately omitted from the template. Pinning a subnet in the template would tie every instance to one Availability Zone and silently defeat the multi-zone requirement, because the Auto Scaling

Group would be unable to distribute instances. IMDSv2 was enforced because token-based metadata access mitigates server-side request forgery attacks that could otherwise be used to read the instance role credentials.

The user data script performs per-boot configuration that cannot be baked into an image:

```
#!/bin/bash
systemctl enable --now httpd

# RDS TLS trust store
curl -s -o /home/ec2-user/rds-ca.pem \
  https://truststore.pki.rds.amazonaws.com/ap-southeast-2/ap-southeast-2-bundle.pem

# ALB health check endpoint
echo "OK" > /var/www/html/health.html

# Identity page, using IMDSv2 token-based metadata access
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone)

cat > /var/www/html/instance.html << EOF
<html><body style="font-family:sans-serif;padding:2rem">
<h1>SWE40006 - Served by</h1>
<p><b>Instance ID:</b> $IID</p>
<p><b>Availability Zone:</b> $AZ</p>
<p><b>Booted:</b> $(date)</p>
</body></html>
EOF

chown -R apache:apache /var/www/html
systemctl restart httpd
```

The instance.html page is a diagnostic endpoint written specifically for this task. It queries the instance metadata service at boot and renders the instance ID and Availability Zone of whichever instance served the request, which converts load distribution from an assumption into something directly observable in a browser.

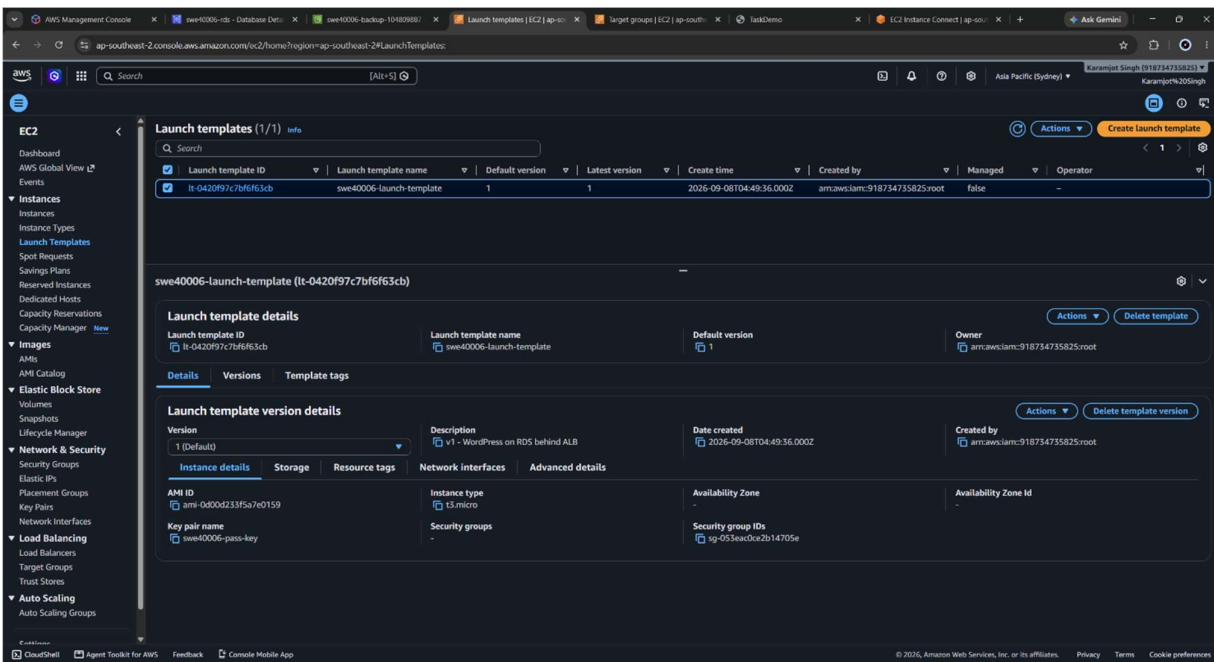


Figure 47: Launch template summary showing the AMI, instance type, security group and IAM instance profile.

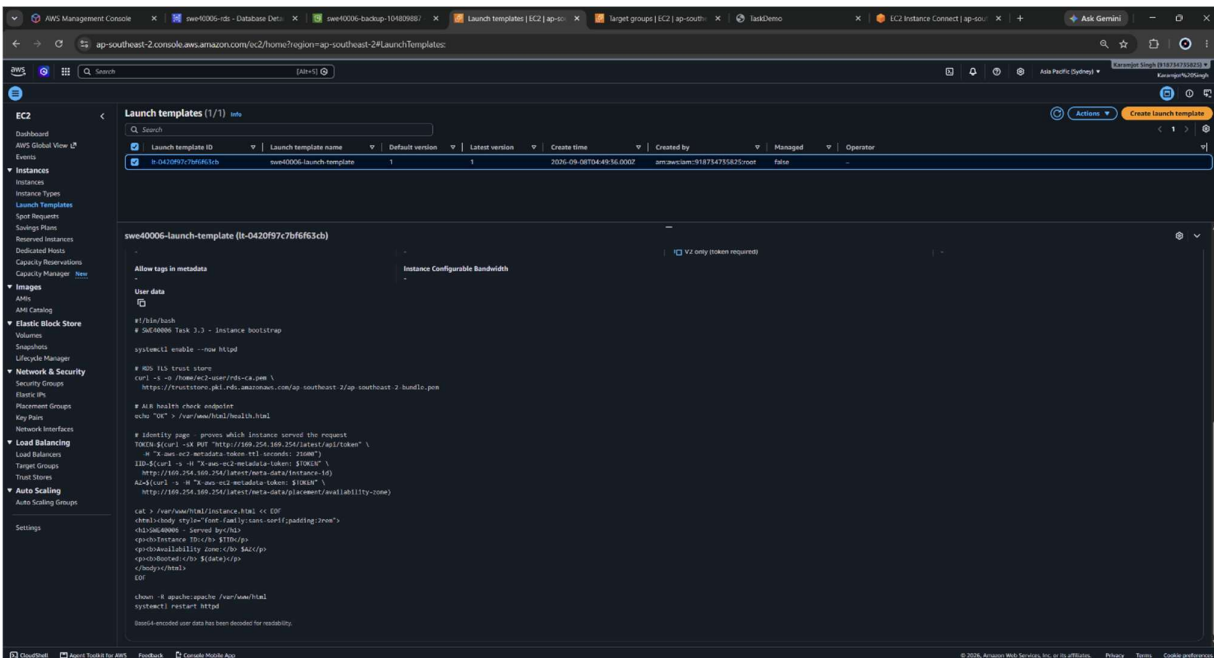


Figure 48: Launch template user data script as stored in the template version.

5.6 Auto Scaling Group

An Auto Scaling Group, swe40006-asg, was created from the launch template with the following configuration:

Setting	Value	Rationale
Availability Zones	2a and 2b	Matches the ALB zone mapping; instances distributed across both
Desired / Min / Max	2 / 2 / 4	Fixed capacity as specified by the task

Setting	Value	Rationale
Target group	swe40006-tg	Instances registered and deregistered automatically
Health check type	ELB (in addition to EC2)	Replaces instances that fail the application health check, not only those whose VM has failed
Health check grace period	300 seconds	Allows boot and Apache start before health checking begins
Scaling policies	None	Task specifies fixed capacity; automatic scaling deliberately out of scope

Enabling ELB health checks alongside the default EC2 health checks is the significant choice here. EC2 health checks assess only hypervisor-level instance status; an instance whose Apache process had crashed would pass an EC2 health check indefinitely while serving nothing. With ELB health checks enabled, the Auto Scaling Group acts on the same signal the load balancer uses, so application-level failure triggers replacement.

A minimum of two rather than one was chosen so that the group survives an instance replacement without a service gap. With a minimum of one, the interval between terminating a failed instance and its replacement passing health checks is a complete outage.

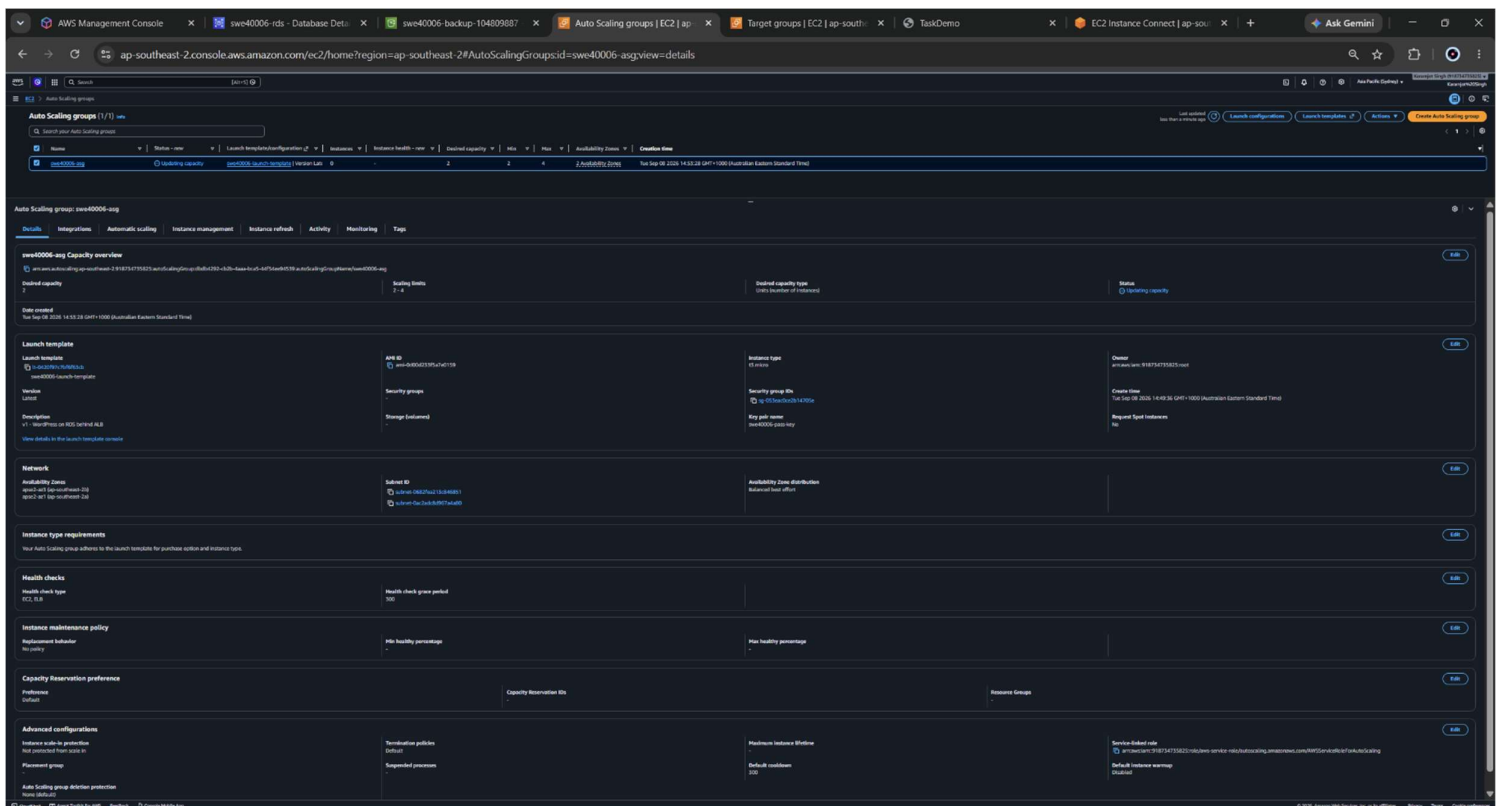


Figure 49: Auto Scaling Group details showing desired 2, minimum 2, maximum 4, and both Availability Zones.

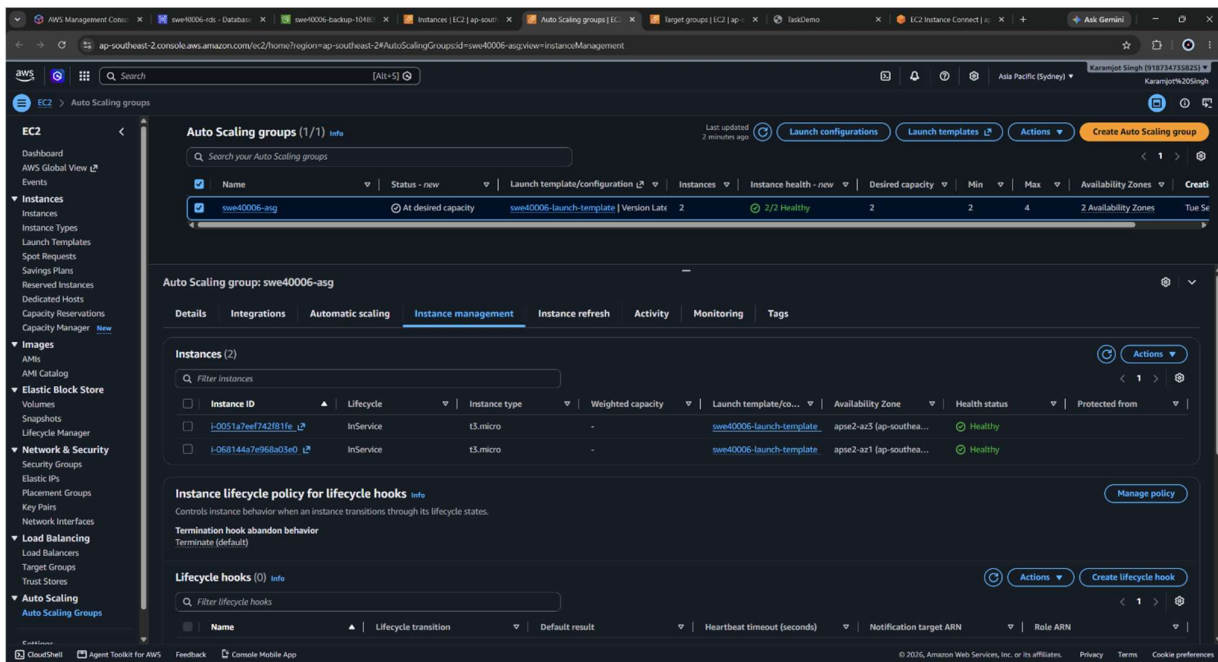


Figure 50: Two instances in Service and Healthy Status created by ASG

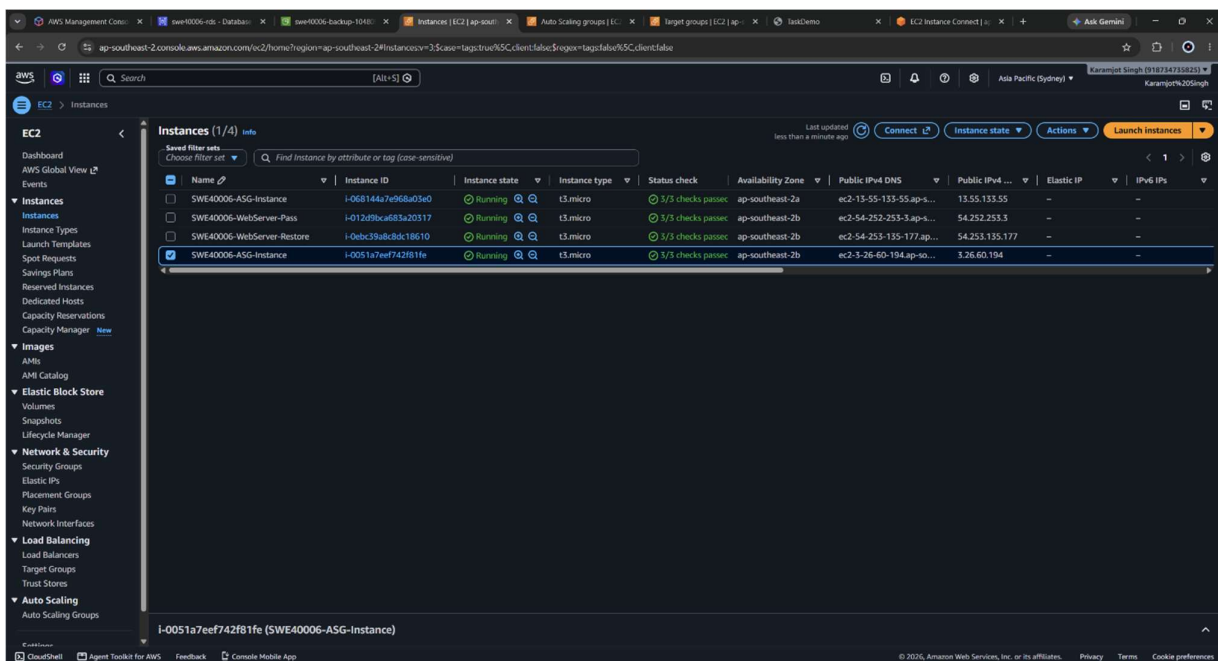


Figure 51: Both instances showing in EC2 Instances

5.7 Verification

5.7.1 Application Availability Through the Load Balancer

The WordPress site was confirmed to serve correctly through the ALB DNS name, including content created before and after the RDS migration.

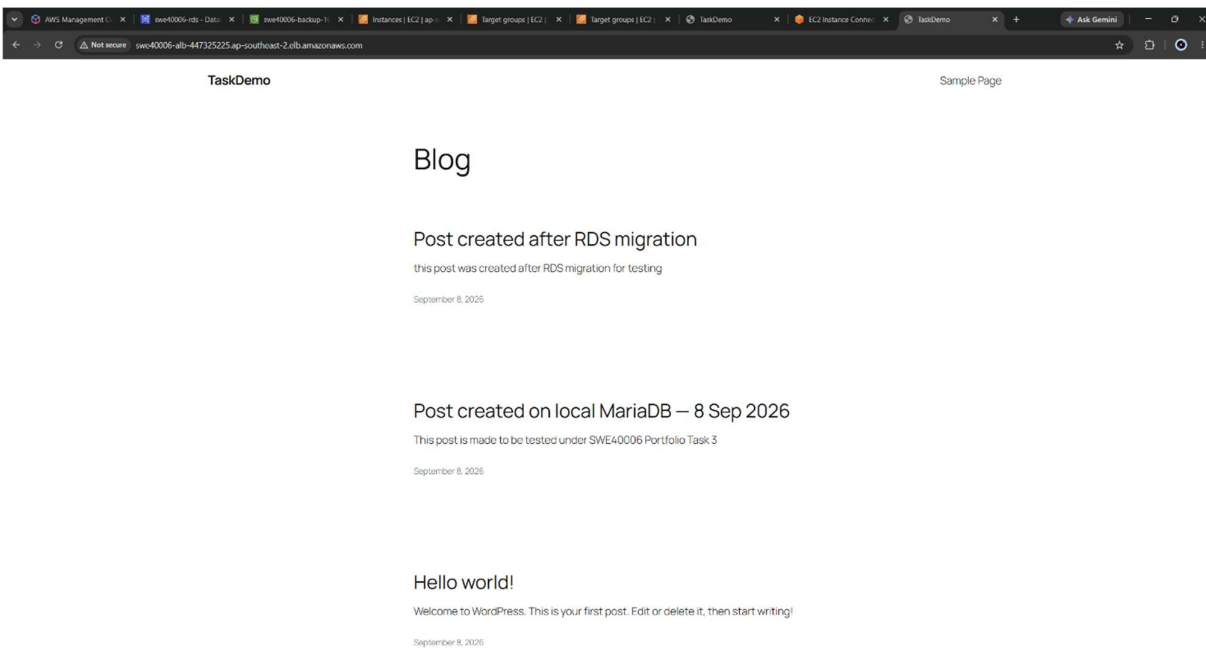


Figure 52: WordPress site loading via the ALB DNS name, visible in the browser address bar.

5.7.2 Traffic Distribution Across Instances and Zones

The instance identity endpoint was requested repeatedly to observe request distribution:

```
for i in {1..8}; do
  curl -s http://<ALB-DNS>/instance.html | grep -E "Instance ID|Zone"
  echo "---"
done
```

Responses alternated between two distinct instance IDs in two distinct Availability Zones, confirming the ALB was distributing requests across both zones rather than favouring one.

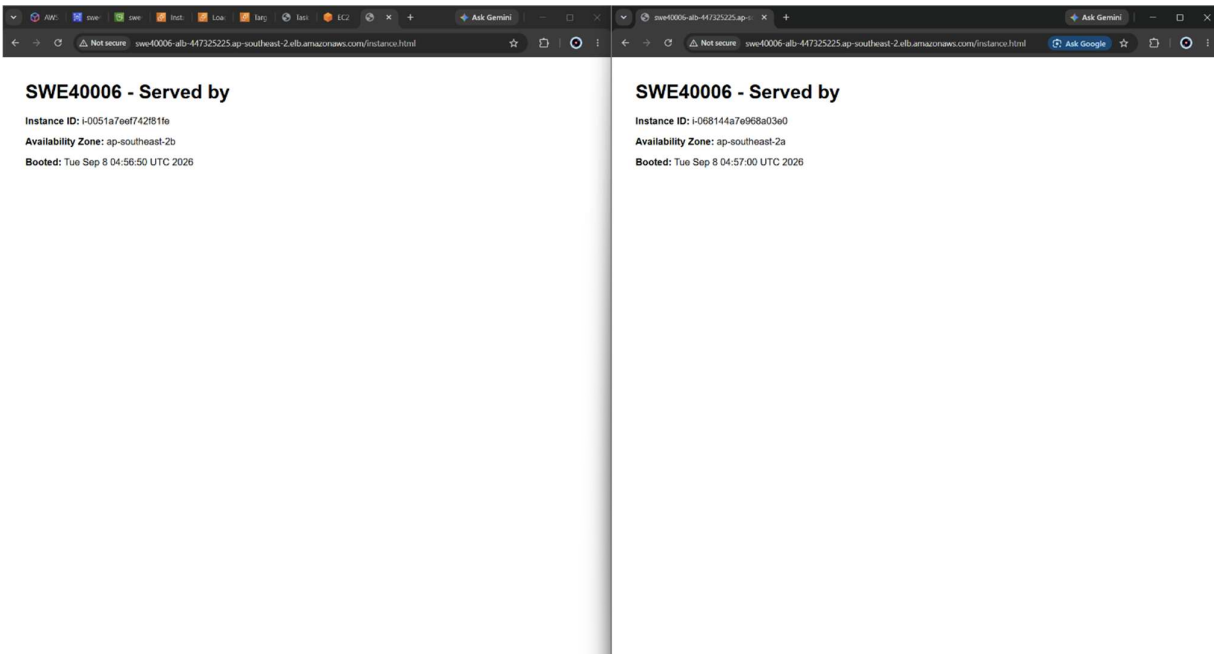


Figure 53: Repeated requests to /instance.html returning two different instance IDs in two different Availability Zones.

5.7.3 Self-Healing Under Instance Failure

An instance failure was simulated by terminating one Auto Scaling Group member directly from the EC2 console. The group detected the capacity shortfall and launched a replacement without intervention, and the site remained available throughout because the second instance continued to serve traffic.

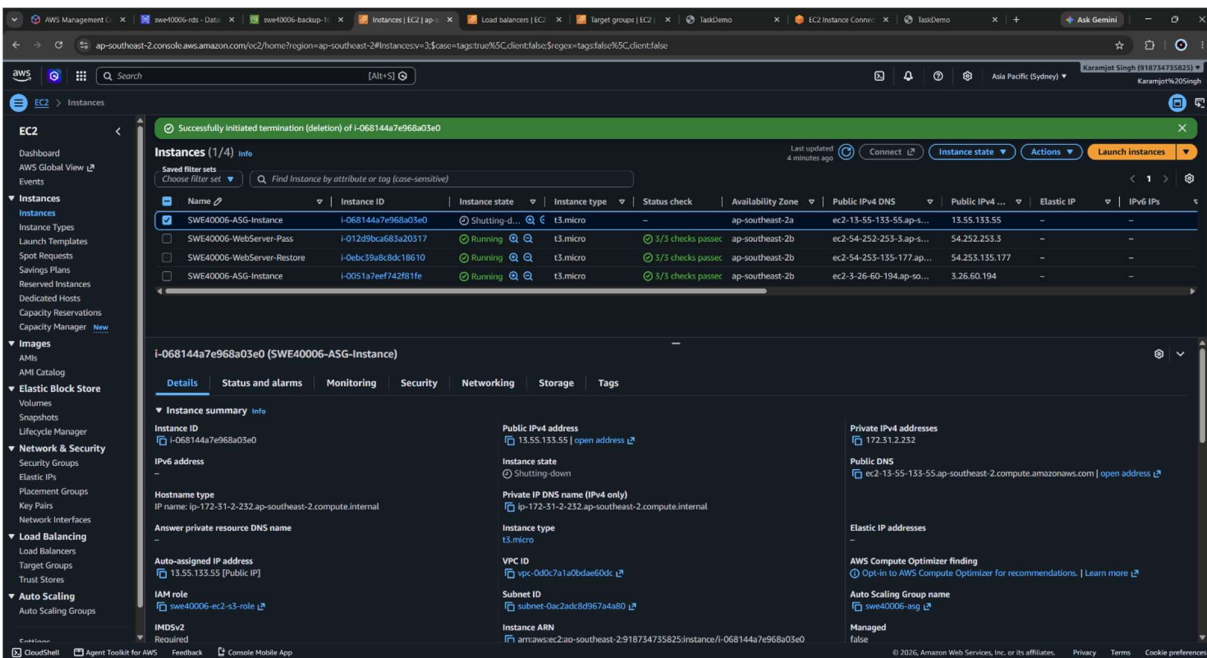


Figure 54: Instance Terminated

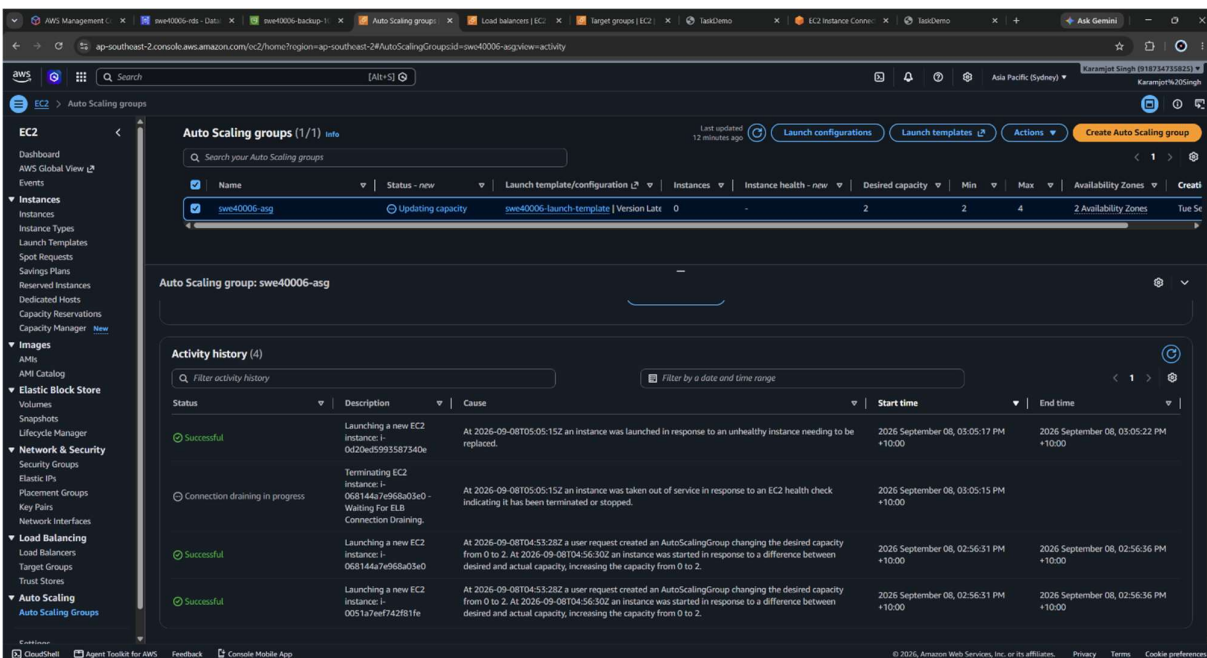


Figure 55: ASG Activity history showing the instance termination followed by an automatic launch of a replacement instance.

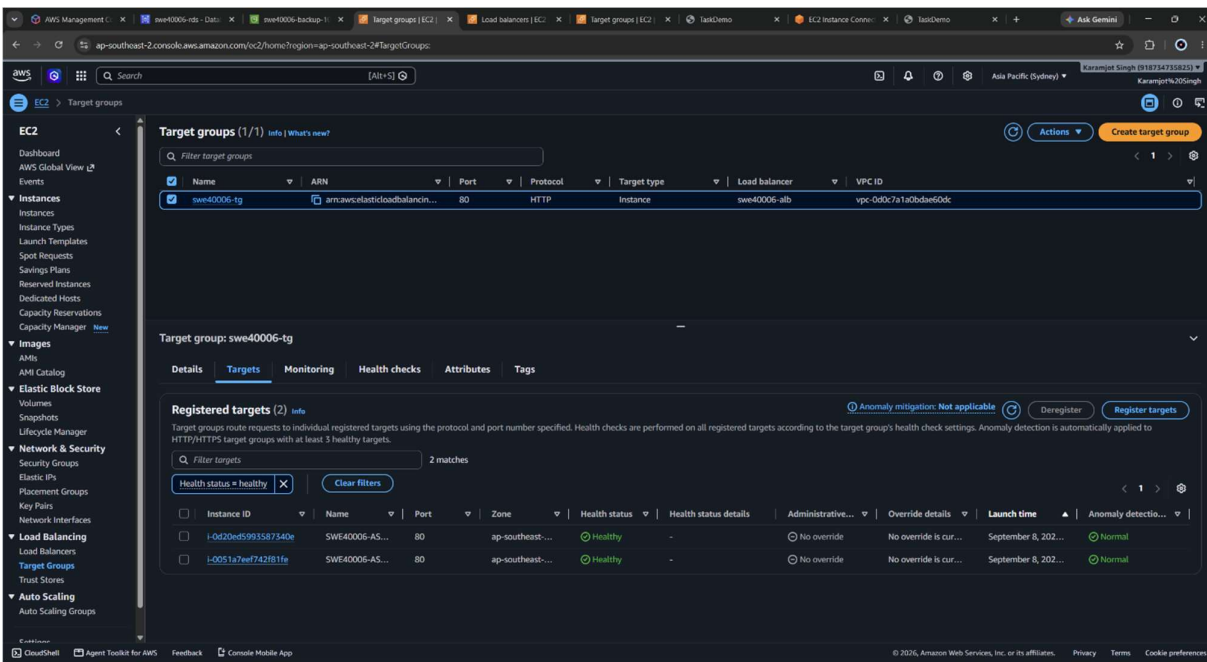


Figure 56: Target group returned to two healthy targets, one of which is a new instance ID.

5.7.4 Shared State Across Instances

A new post was published through the ALB and then observed while being served by a different instance than the one that handled the write. The post was present, because both instances read from the same RDS database. This confirms the Credit-level migration was a genuine prerequisite: with the Pass-level local database, each instance would have held its own isolated copy and content would have appeared and disappeared depending on which instance served the request.

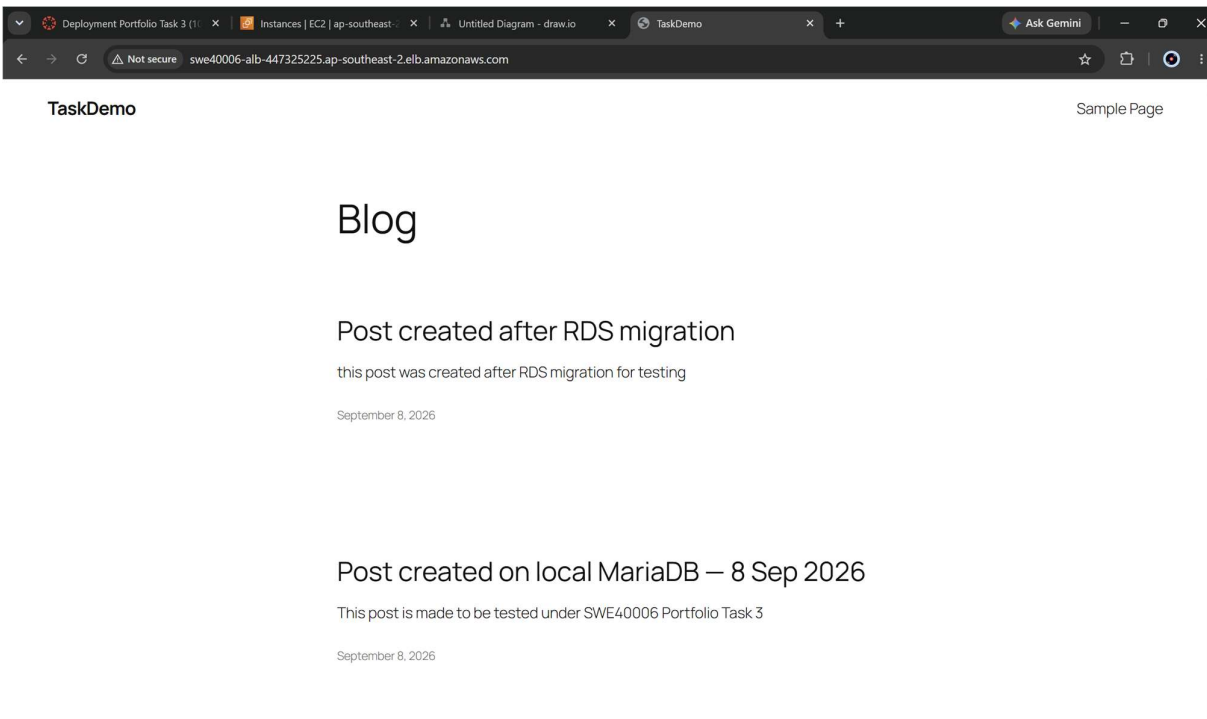


Figure 57: New post visible via the ALB while /instance.html reports a different instance ID than the one used to create it.

5.7.5 Network Isolation of Instances

Direct access to an instance public IP was tested and timed out, while the same instance responded to a request on localhost with HTTP 200. This confirms that the web server was healthy and that the failure was the security group correctly denying direct internet access, forcing all traffic through the load balancer.

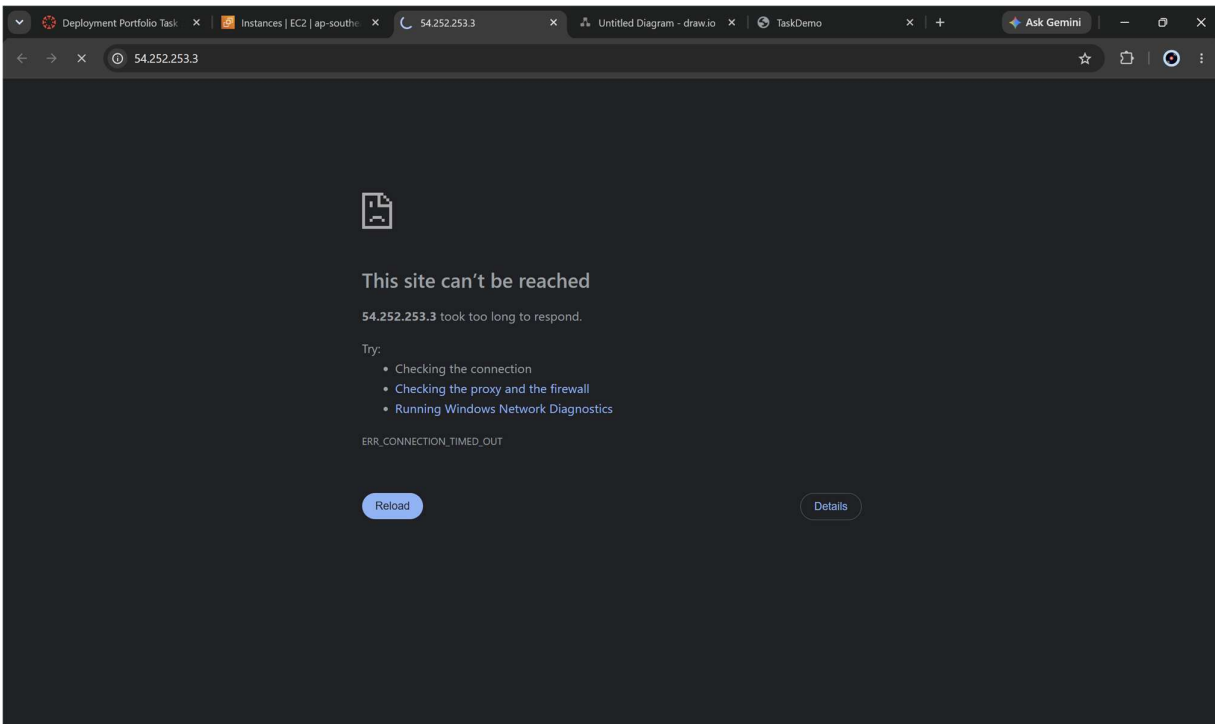


Figure 58: public IP of instance timed out

6. Task 3.4 - High Distinction Level: Monitoring and Secure Management

Objective: create a CloudWatch alarm on a simple metric such as CPU utilisation exceeding 70%, with an SNS topic delivering email notification; and enable AWS Systems Manager Session Manager for terminal access through the AWS Console without public SSH access or an open port 22.

6.1 SNS Topic and Subscription

A standard SNS topic, swe40006-alerts, was created with an email subscription. The subscription was confirmed by following the link in the AWS confirmation email; until confirmation, a subscription remains in PendingConfirmation state and silently discards all messages, which is a common cause of alarms appearing to fire without notification.

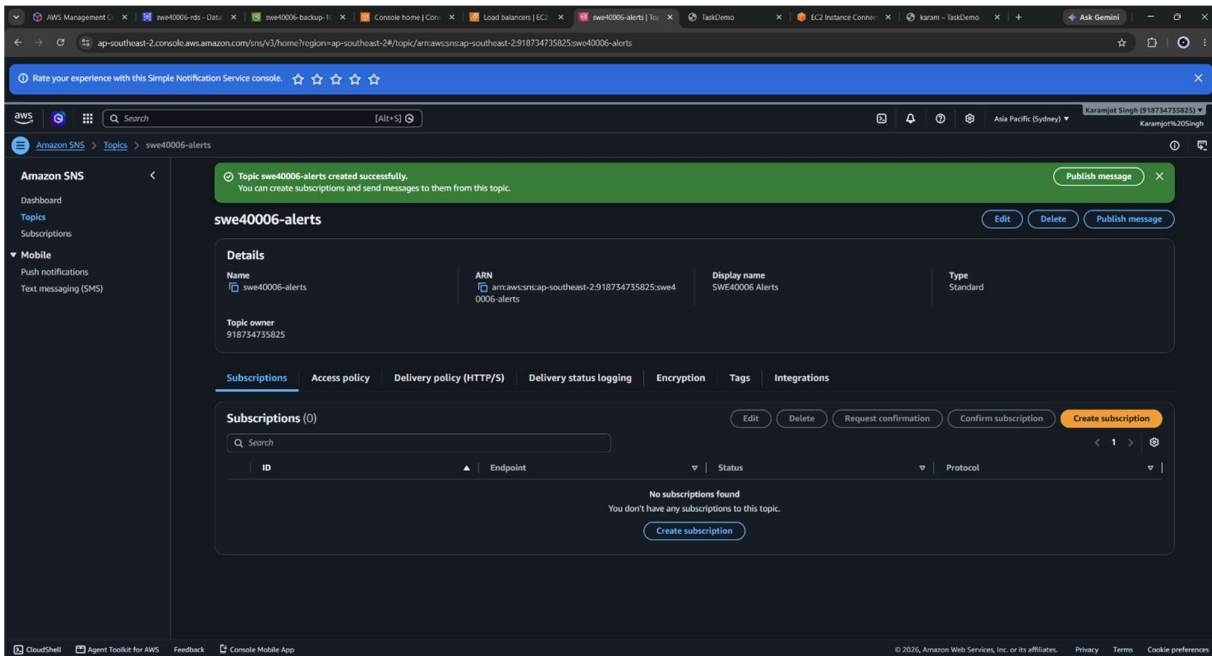


Figure 59: Created the alert in Amazon SNS portal

A test message was published to the topic before the alarm was wired to it, isolating notification delivery from alarm logic so that a later failure could be attributed unambiguously.

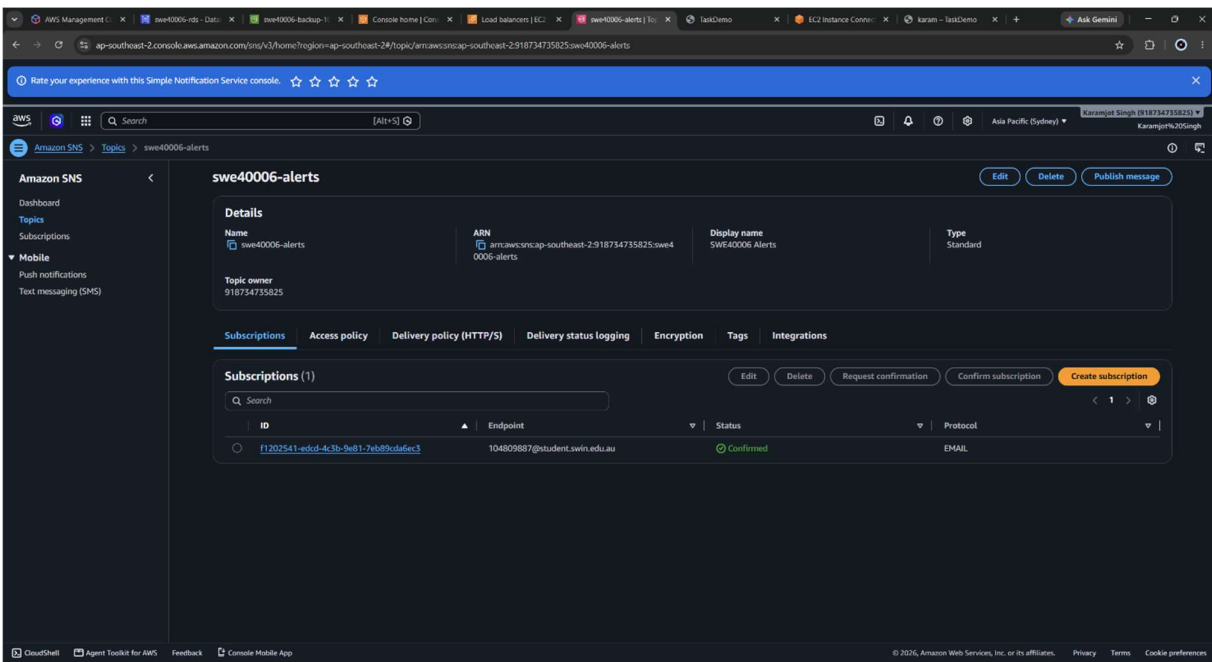


Figure 60: SNS topic swe40006-alerts showing the email subscription(to my student email) with status Confirmed.

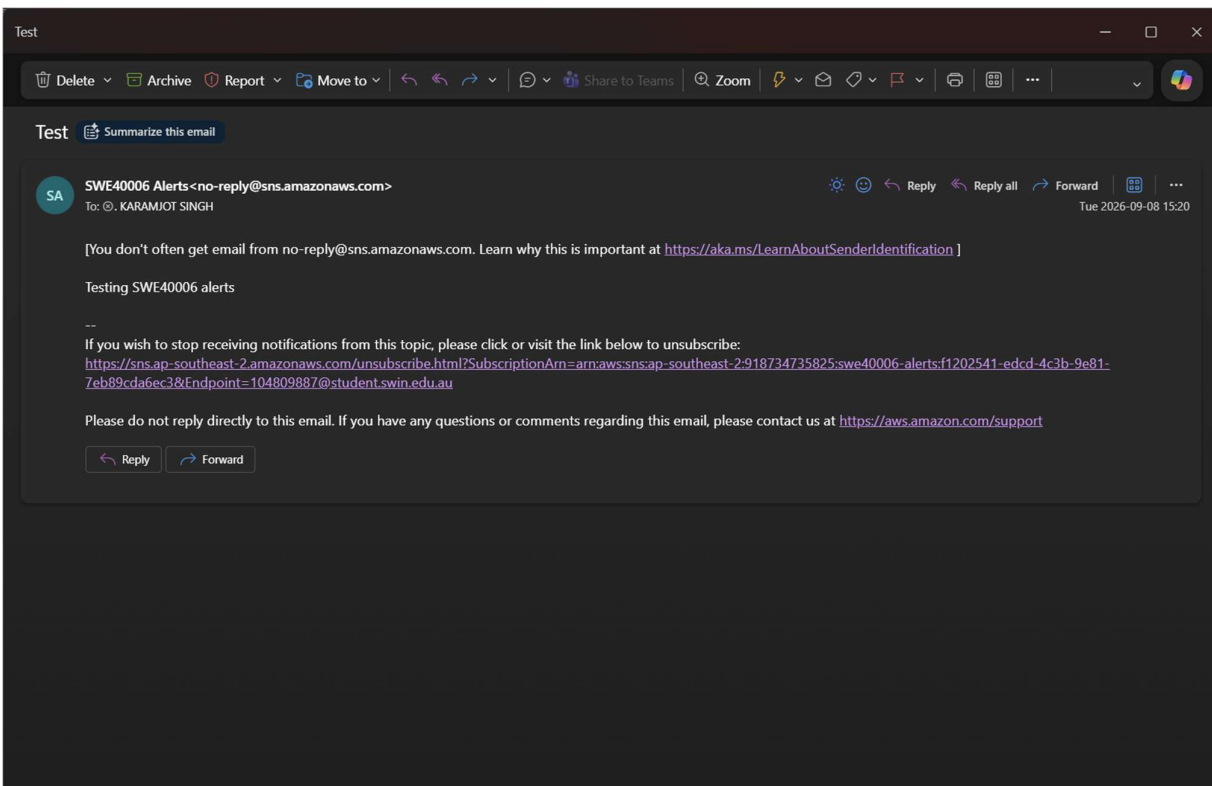


Figure 61: Tested email alert

6.2 CloudWatch Alarm Design

The alarm monitors CPUUtilization dimensioned by Auto Scaling Group rather than by instance ID.

Setting	Value	Rationale
Metric	CPUUtilization by ASG	Survives instance replacement; an instance-scoped alarm would break when the ASG replaces a member
Statistic / Period	Average / 5 minutes	Basic monitoring publishes at 5-minute intervals; 1-minute resolution requires paid detailed monitoring
Threshold	> 70%	As specified by the task
Datapoints to alarm	1 out of 1	Immediate response, accepting some sensitivity to transient spikes
Missing data	Treated as missing	Avoids spurious state changes during instance replacement
Actions	ALARM and OK → swe40006-alerts	Recovery notification included so the alarm reports resolution as well as onset

Dimensioning by Auto Scaling Group is the correct choice for an auto-scaled fleet and is a direct consequence of the Task 3.3 architecture. It has a measurable consequence for testing: because the metric is an average across the group, load applied to a single instance of two produces roughly 50% group utilisation and never crosses the threshold. Load was therefore applied to both instances simultaneously.

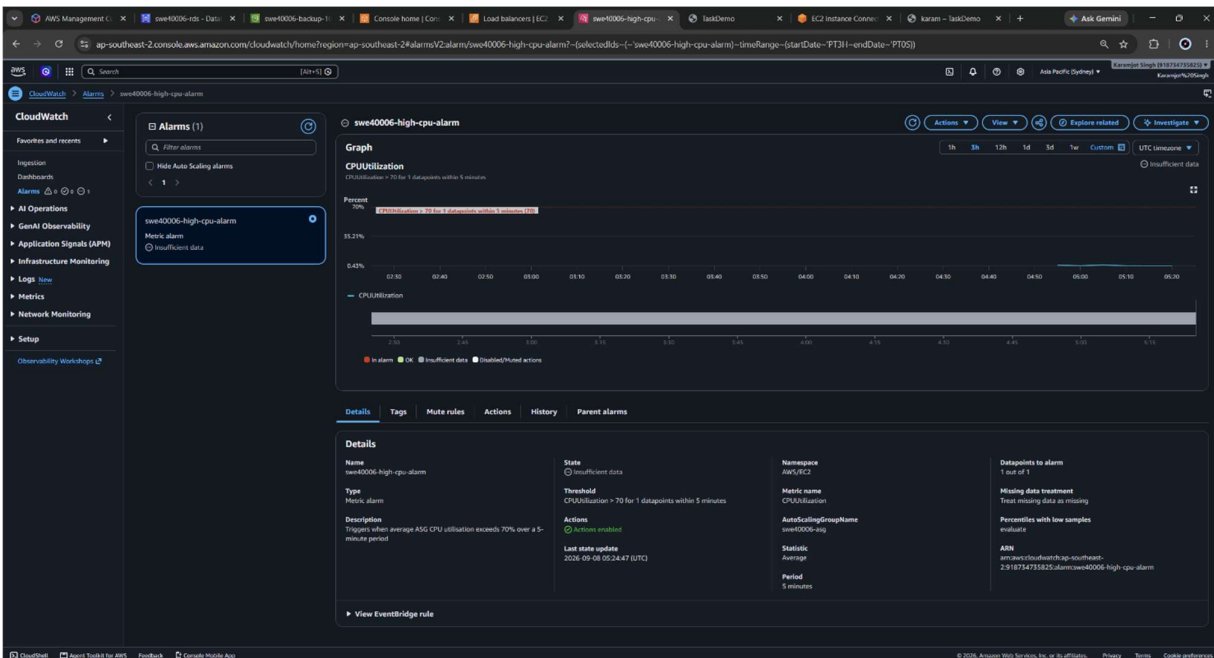


Figure 62: CloudWatch alarm configuration showing the ASG-dimensioned CPUUtilization metric and 70% threshold

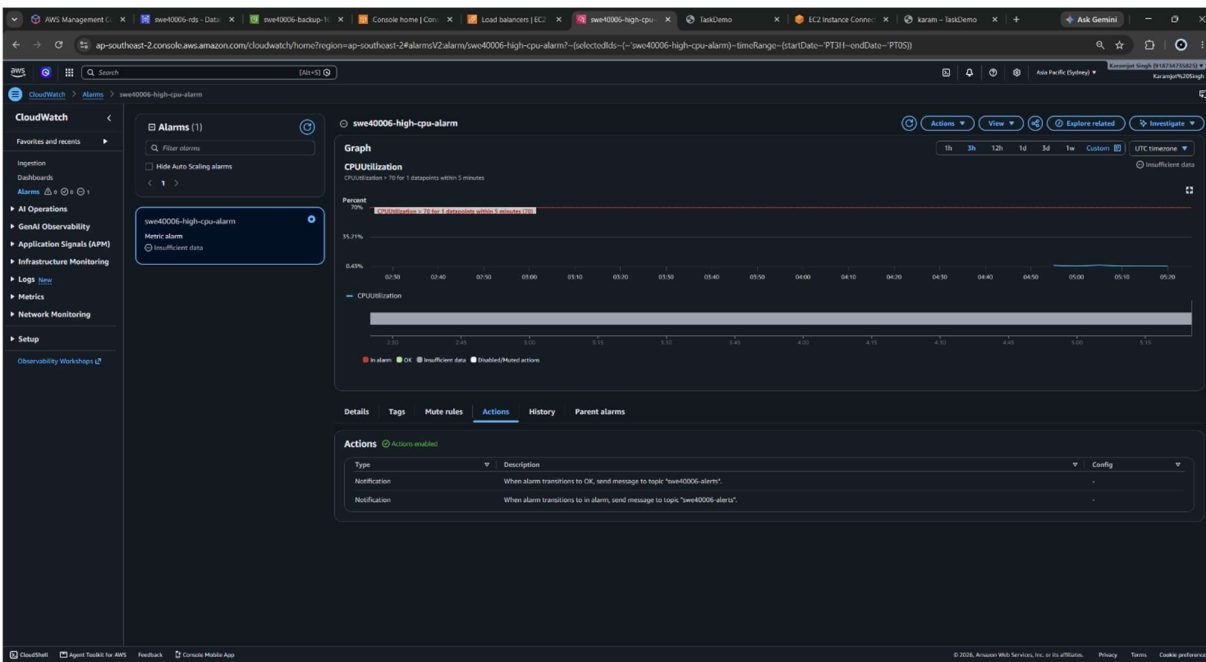


Figure 63: SNS Actions in Cloudwatch

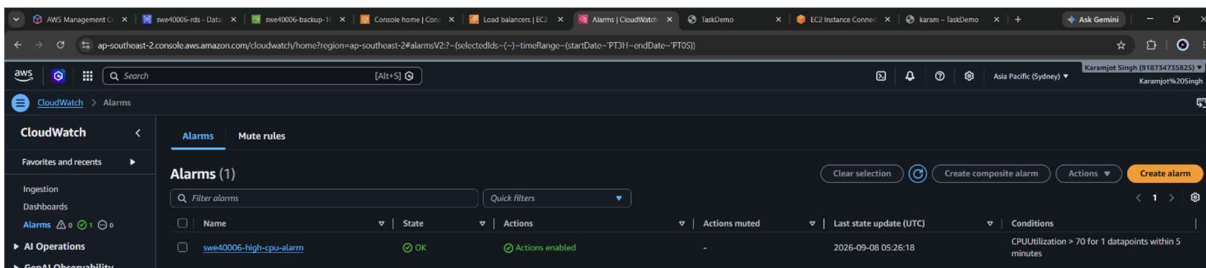


Figure 64: Alarm in the OK state prior to the load test, establishing a baseline.

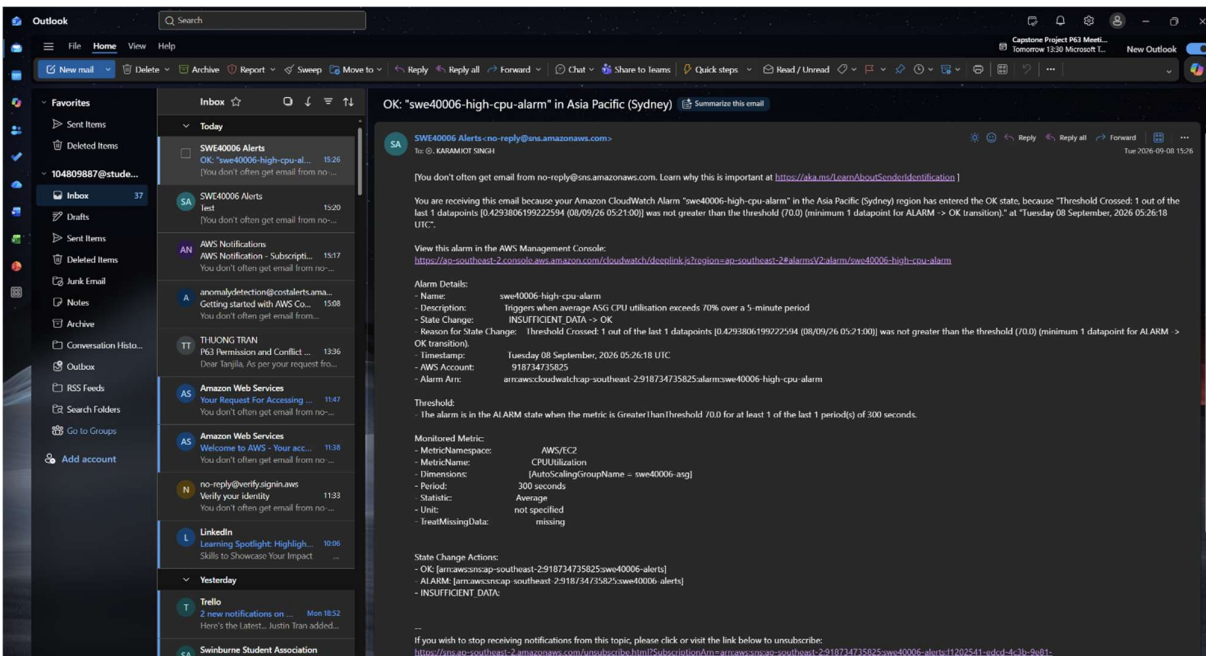


Figure 65: Received email when status is OK

6.3 Alarm Validation Under Synthetic Load

Two Session Manager sessions were opened, one per Auto Scaling Group instance, and each instance was identified from the metadata service before load was applied, to confirm the two sessions targeted different hosts:

```
whoami
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id; echo
nproc
```

Synthetic CPU load was then generated on both instances:

```
sudo dnf install -y stress-ng
cd /tmp
stress-ng --cpu 0 --timeout 900s --metrics-brief
```

--cpu 0 instructs stress-ng to use all available cores, saturating both vCPUs on each t3.micro. The alarm transitioned to ALARM approximately eight to twelve minutes after load began, consistent with the five-minute metric publication interval combined with the averaging window. Load was then stopped and the alarm observed returning to OK, with a corresponding recovery email.

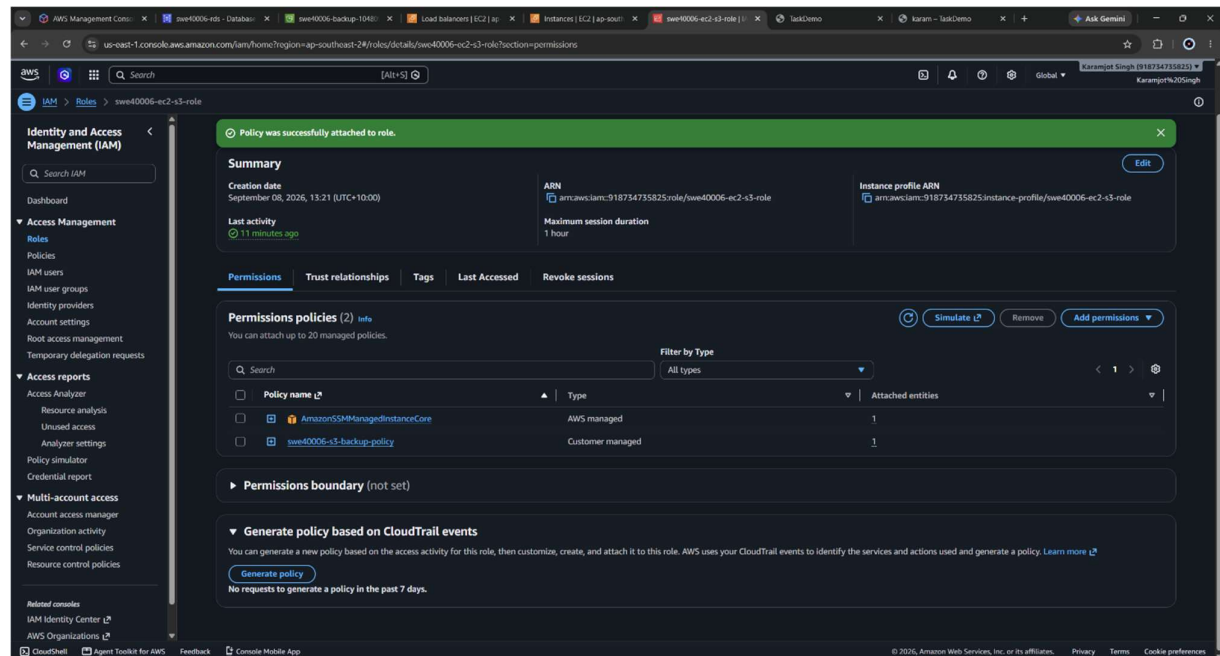


Figure 66: Attached SSM required policy to the IAM role

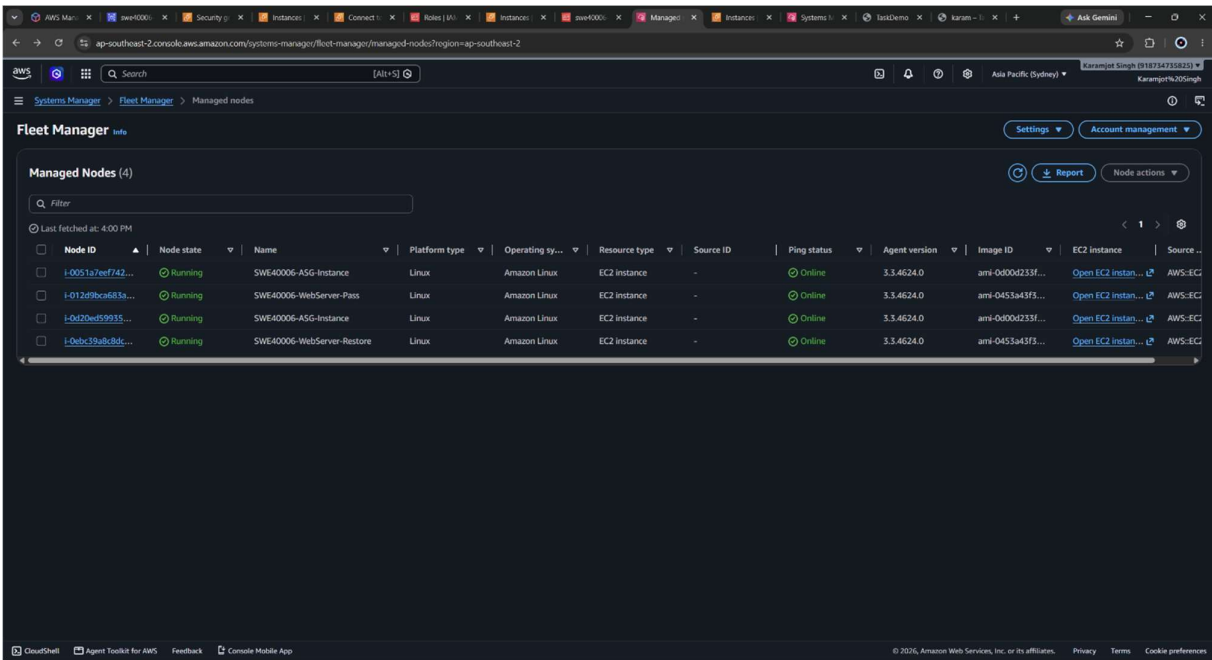


Figure 67: Fleet Manager showing all instances

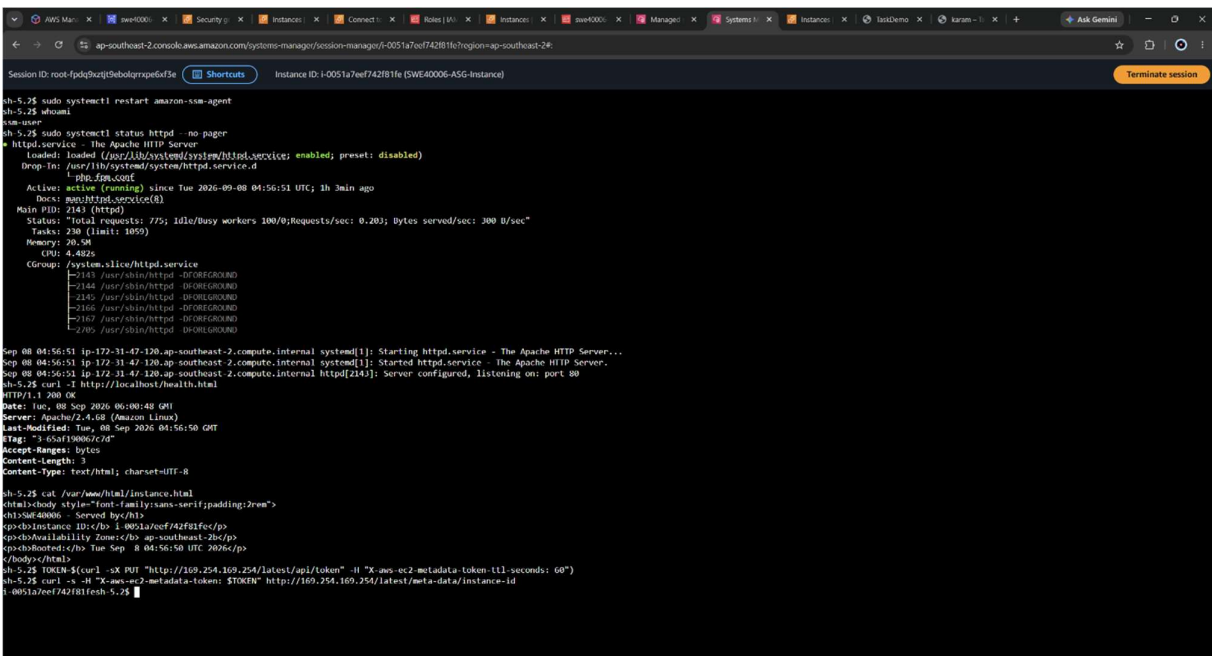


Figure 68: Session Manager terminal showing whoami returning ssm-user and the instance ID from the metadata service.

```

Session ID: root-fpdq9uzq9ebolqrxpe6af5e Instance ID: i-0051a7ee742f81fe (SWE40006-ASG-Instance)

Installed:
July-1.0.5-25.amzn2023.0.3.x86_64 libbsd-0.10.0-7.amzn2023.0.2.x86_64 lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64 stress-ng-0.15.05-1.amzn2023.x86_64

Completed
sh-5.2$ nproc
2
sh-5.2$ stress-ng --cpu 2 --timeout 600s --metrics-brief
aborting: temp-path "." must be readable and writable
sh-5.2$ for i in $(seq 1 $(nproc)); do (while ;; do ;; done) & done
[1] 6394
[2] 6395
sh-5.2$ kill $(jobs -p)
sh-5.2$ sudo dnf install -y stress-ng
Last metadata expiration check: 4:12:47 ago on Tue Sep  8 01:51:18 2026.
Package stress-ng-0.15.05-1.amzn2023.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Completed
[1]+ Terminated (while ;; do
done)
[2]+ Terminated (while ;; do
done)
sh-5.2$ nproc
2
sh-5.2$ stress-ng --cpu 2 --timeout 600s --metrics-brief
aborting: temp-path "." must be readable and writable
sh-5.2$ cd /tmp
stress-ng --cpu 0 --timeout 900s --metrics-brief
stress-ng: info: [227] setting to a 900 second (15 mins, 0.00 secs) run per stressor
stress-ng: info: [228] dispatching hogs: 2 cpu

```

Figure 69: Running stress test (on both instances)

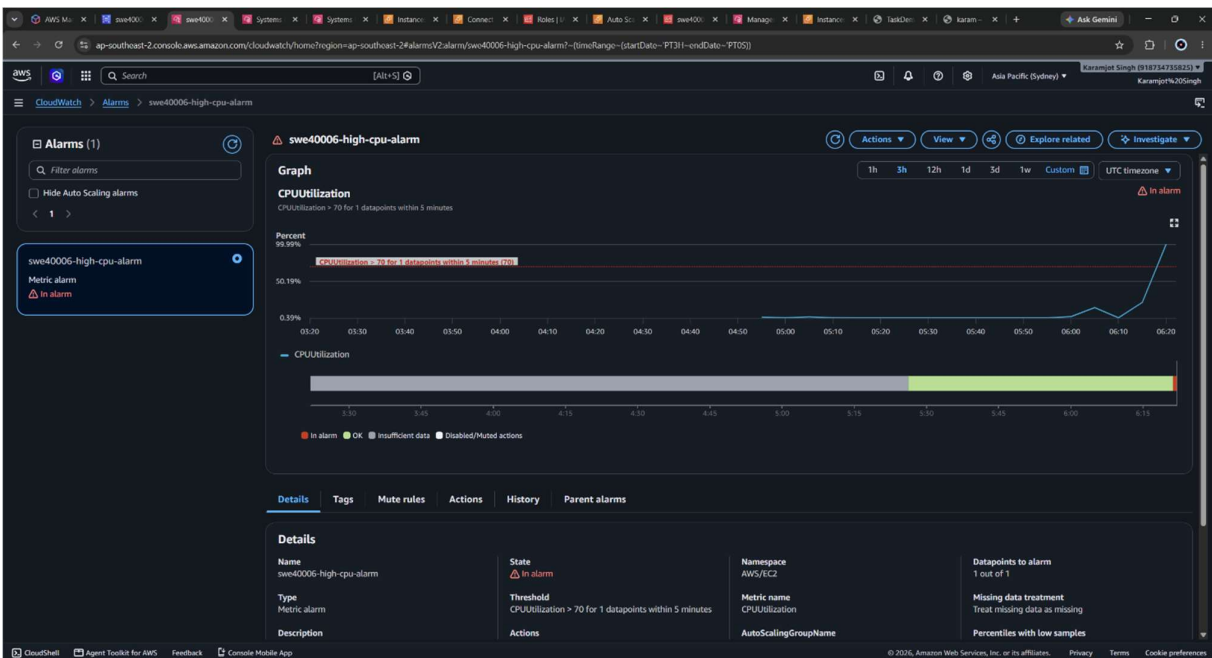


Figure 70: CloudWatch metric graph showing average ASG CPU utilisation rising above the 70% threshold line. Alarm in alarm state

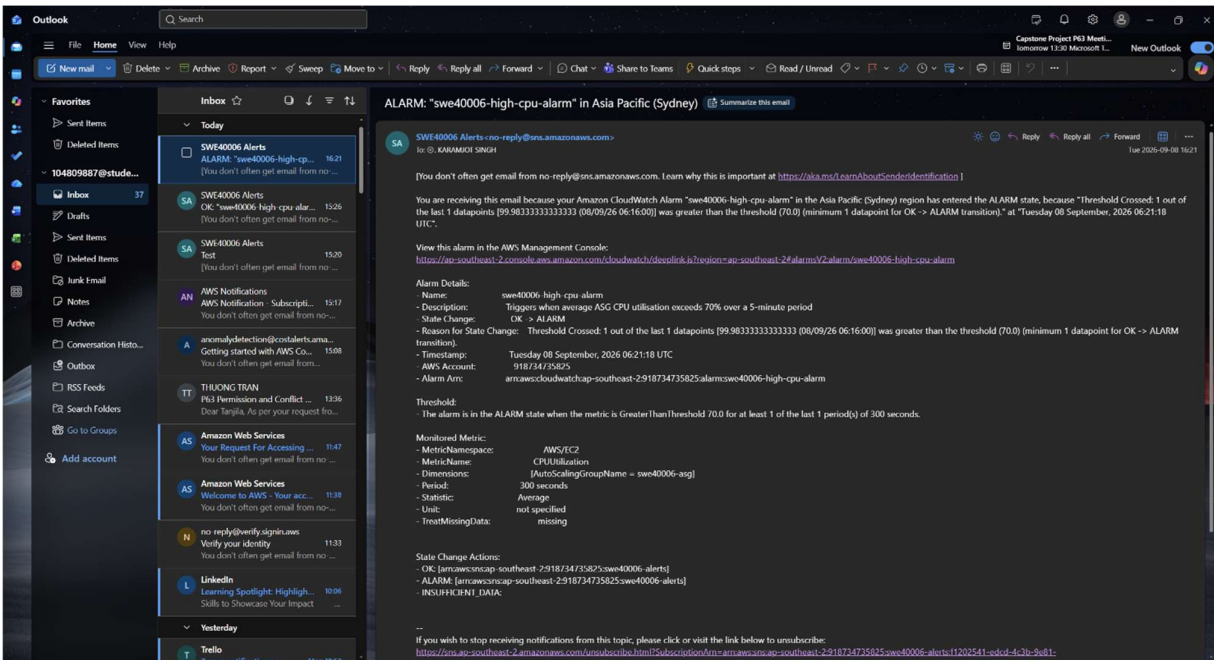


Figure 71: Email notification received from SNS with subject line ALARM: "swe40006-high-cpu-alarm" in Asia Pacific (Sydney).

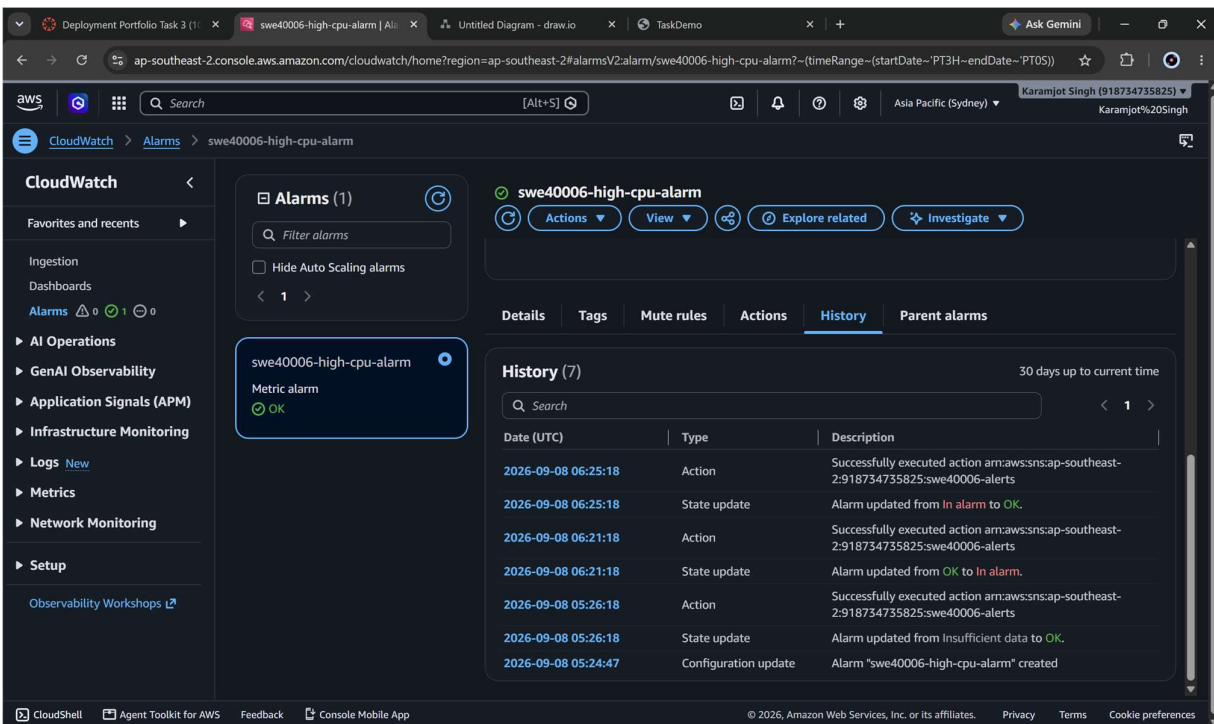


Figure 72: History tab showing the state transitions

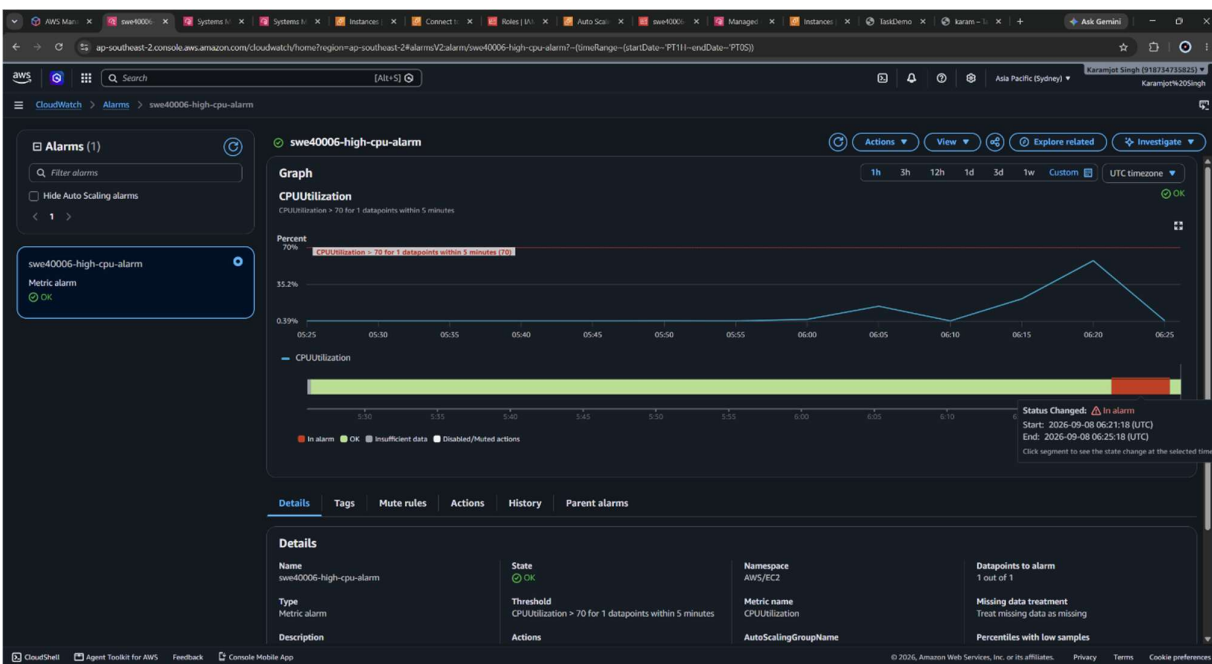


Figure 73: Alarm returned to the OK state after load ceased

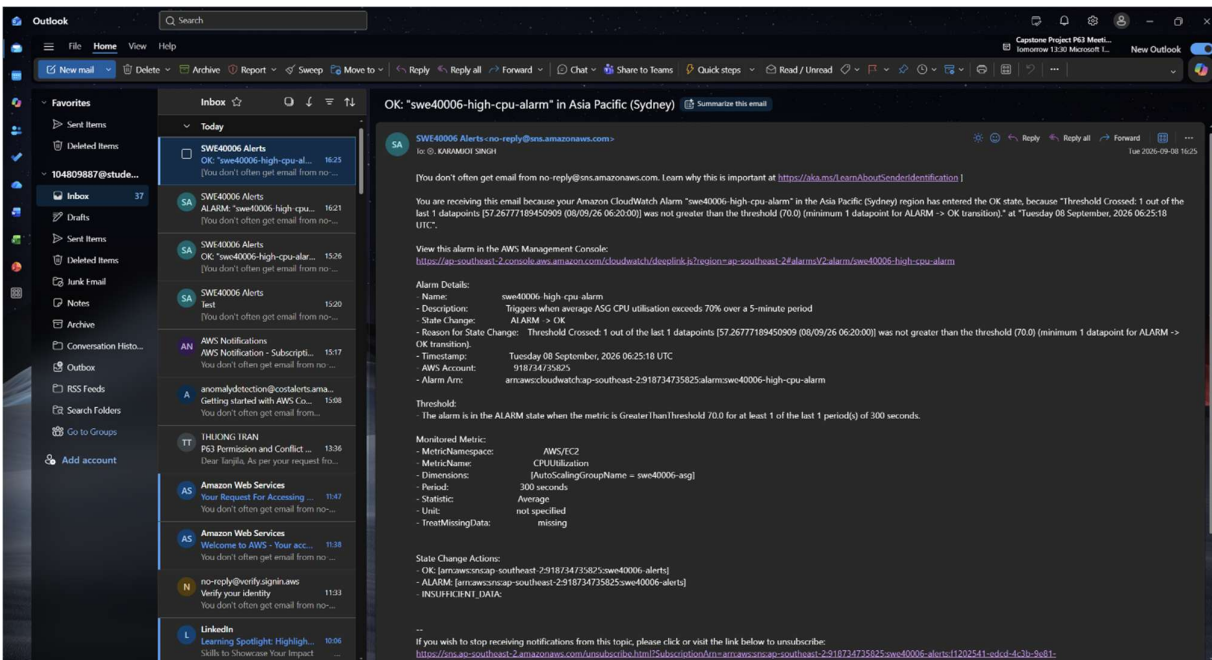


Figure 74: the recovery email - demonstrating the complete ALARM to OK cycle.

6.4 Systems Manager Session Manager

6.4.1 Permission Model

Session Manager requires no inbound network access. The SSM Agent, preinstalled on Amazon Linux 2023, establishes an outbound HTTPS connection to the Systems Manager service and holds it open; session traffic is delivered over that existing outbound channel. Authorisation is therefore an IAM problem rather than a firewall problem.

The AWS managed policy AmazonSSMManagedInstanceCore was attached to the existing swe40006-ec2-s3-role. No new role was created and no instance was rebuilt, because the launch template already attached this role to every instance.

AmazonSSMManagedInstanceCore was chosen in preference to AmazonSSMFullAccess. The former grants only the API calls the agent itself requires - instance information updates, the message delivery channels, and limited S3 access for agent updates - and grants no ability to act on other resources in the account. AmazonSSMFullAccess would permit the instance to manage Systems Manager across the account, which is excessive for a credential held on an internet-facing web server.

6.4.2 Registration and Connection

Instances appeared in Systems Manager Fleet Manager with a ping status of Online. A session was then opened from the console and the target verified from within the session by comparing the instance ID reported by the metadata service against the console.

Sessions run as the ssm-user account, which is granted passwordless sudo. This is a meaningful difference from SSH: the identity performing the session is an IAM principal recorded in CloudTrail, whereas an SSH login is authenticated by possession of a key file and recorded only in the instance-local system log - a log that disappears when the Auto Scaling Group replaces the instance.

6.5 Elimination of Public SSH Access

With Session Manager verified, the SSH rule was deleted from swe40006-web-sg, leaving a single inbound rule permitting HTTP 80 from the ALB security group. No instance in the account now accepts inbound SSH from any source. Both halves of the claim were then verified rather than asserted. A TCP connection test to port 22 on an instance public IP was attempted from the my workstation and failed; a Session Manager session to the same instance succeeded immediately afterwards; and the application continued to serve through the ALB, confirming that removing SSH had no effect on availability.

```
# From the local Windows workstation
Test-NetConnection -ComputerName <INSTANCE_PUBLIC_IP> -Port 22
# Expected: TcpTestSucceeded : False
```

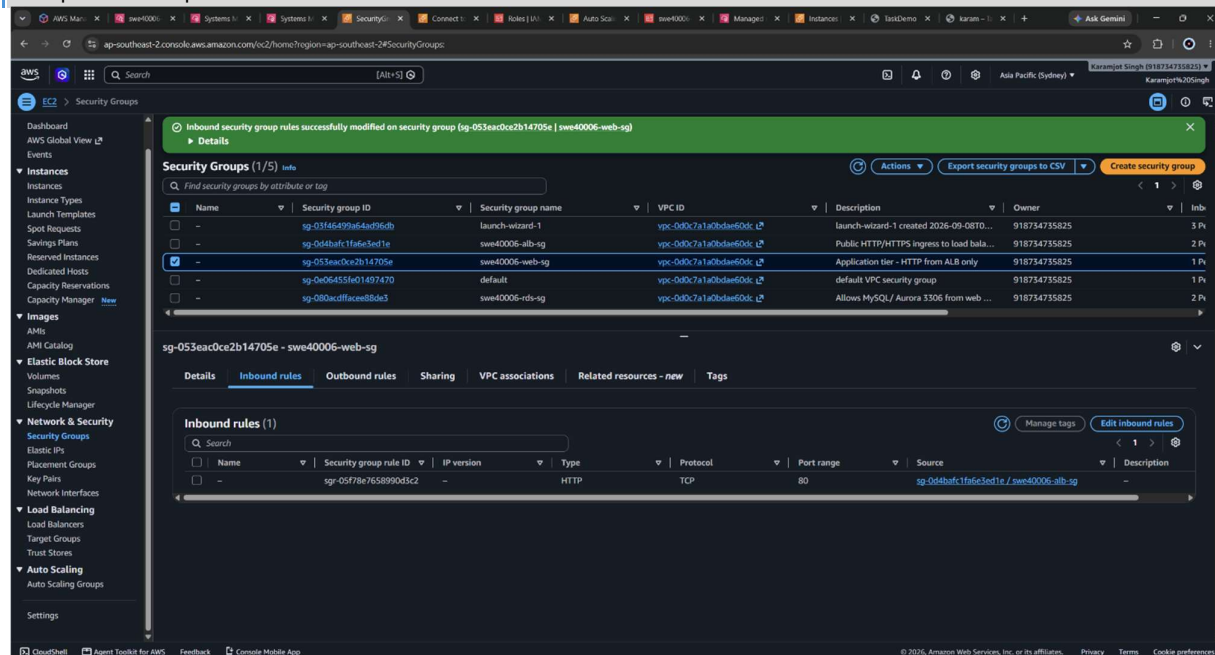


Figure 75: Inbound rules of swe40006-web-sg after the change, containing no port 22 entry.

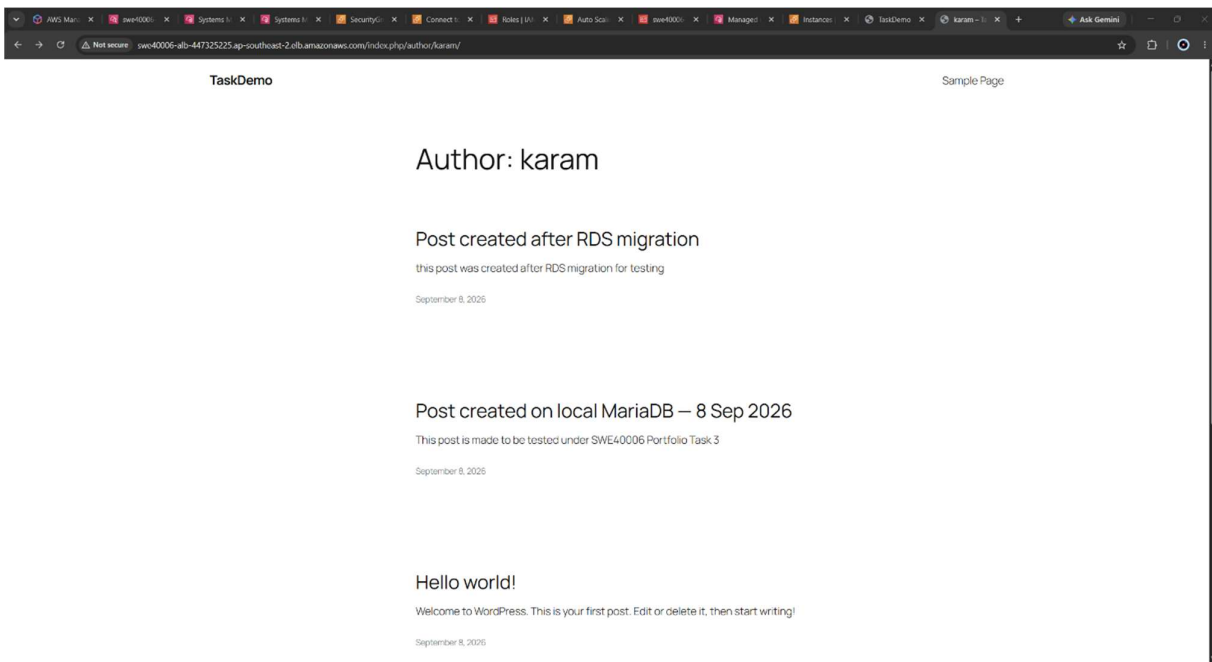


Figure 76: WordPress site still serving correctly via the ALB after port 22 removal, confirming no availability impact

6.6 Security Posture Across the Four Levels

Property	3.1 Pass	3.3 Distinction	3.4 High Distinction
Inbound SSH	Port 22 open to 0.0.0.0/0	Port 22 restricted to a single IP	Port 22 absent
Administrative authentication	Possession of a .ppk private key file	Possession of a .ppk private key file	IAM principal identity
Revoking access	Rotate keys on every instance	Rotate keys on every instance	Detach an IAM policy - immediate and central
Audit trail	Instance-local /var/log/secure, lost on termination	Instance-local, lost on termination	CloudTrail, plus optional S3 session logging
Database exposure	Same host as the web server	Private, TLS enforced, SG-restricted	Unchanged
Failure detection	None	ALB health checks	Health checks plus CloudWatch alarm with email alerting

7. Self-Troubleshooting Log and Analysis

Eight substantive faults were encountered and resolved during the task. Each is recorded with the observed symptom, the diagnosis, the resolution applied, and the generalisable lesson. Where an easier but weaker resolution existed, it is stated along with the reason it was rejected.

7.1 MySQL Error 2005 - Unknown Server Host

```
ERROR 2005 (HY000): Unknown MySQL server host
'swe40006-rds.<placeholder>.ap-southeast-2.rds.amazonaws.com' (-2)
```

Symptom: the first connection attempt to RDS failed immediately, with no delay.

Diagnosis: the (-2) suffix corresponds to the getaddrinfo return code for a name resolution failure. The failure was therefore in DNS, before any network connection was attempted - which ruled out both the security group and the credentials. The endpoint in use was a documentation placeholder rather than the account-specific hostname; the middle component of an RDS endpoint is a randomly generated identifier unique to each account.

Resolution: the real endpoint was copied from the RDS console Connectivity & security panel. Resolution was then confirmed with nslookup, which returned a private 172.31.x.x address - itself a useful confirmation that Publicly accessible: No had been applied correctly.

Lesson: the failure mode encodes the failing layer. An immediate resolution failure, a delayed timeout and an access-denied response indicate DNS, firewall and authentication respectively, and distinguishing between them prevents time being spent on the wrong layer.

7.2 MySQL Error 3159 - Insecure Transport Prohibited

```
ERROR 3159 (08004): Connections using insecure transport are prohibited
while --require_secure_transport=ON.
```

Symptom: with a valid endpoint and correct credentials, the connection was refused by the database engine.

Diagnosis: current RDS default parameter groups set require_secure_transport = ON, rejecting any unencrypted client connection. The error is in fact positive evidence that DNS, routing, the security group and authentication were all functioning, since the server was reached and responded.

Resolution: the ap-southeast-2 certificate authority bundle was downloaded from the AWS trust store and supplied to all client connections via --ssl-ca. Encryption was then confirmed rather than assumed by querying Ssl_cipher, which returned a populated TLS cipher suite.

Rejected alternative: a custom DB parameter group with require_secure_transport = 0 would have resolved the error in minutes. It was rejected because it disables encryption in transit for the entire database to avoid a two-line client configuration change, weakening the security posture that this task exists to demonstrate.

Consequence discovered later: the same requirement applies to the application. WordPress connects via mysqli, which does not negotiate TLS by default, and required the MYSQL_CLIENT_FLAGS constant documented in Section 4.7. Had the parameter been disabled instead, this dependency would never have surfaced.

7.3 MySQL Error 1045 - Access Denied

Symptom: authentication failure when connecting as wp_user.

Diagnosis: two distinct accounts exist with independent credentials - the RDS master user admin, set during instance creation, and the application user wp_user, created afterwards. Additionally, the wp_user account on the local MariaDB instance is an entirely separate account on a separate server and does not transfer with the data. The dump contains the wordpress_db schema, not the server's mysql user table.

Resolution: the master user connection was tested first to isolate which credential was failing, and the account list queried directly:

```
mysql -h $RDS -u admin -p $SSL -e \
"SELECT User, Host FROM mysql.user WHERE User IN ('admin','wp_user');"
```

Lesson: when two credentials could explain one failure, test the one that is independently verifiable first. This narrows the fault in a single step rather than guessing.

7.4 Shell Metacharacter Misinterpretation

```
$ aws s3 cp $BUCKET/backups/wordpress-files-<DATE>.tar.gz /tmp/
-bash: DATE: No such file or directory

$ sudo tar -xzf /tmp/wordpress-files-2026-09-08.tar.gz -C /var/www/html
tar (child): Cannot open: No such file or directory
```

Symptom: a command copied from documentation failed, and the subsequent command failed with an apparently unrelated error.

Diagnosis: the angle brackets in the documented placeholder <DATE> are shell redirection operators. Bash interpreted < as an input redirection and attempted to open a file named DATE, so the aws command never executed. The tar failure was a consequence rather than a separate fault: the archive had never been downloaded, so /tmp/ was empty.

Resolution: the placeholder was replaced with the actual filename. The download was then verified with ls -lh before extraction was attempted.

Lesson: cascading failures should be traced to the earliest error in the sequence rather than debugged individually. Verifying that each step produced its expected artefact, rather than only that it produced no error, prevents the cascade.

7.5 Security Group Change Causing Loss of Access

Symptom: immediately after moving the instance from launch-wizard-1 to swe40006-web-sg, the EC2 Instance Connect session was lost and the site became unreachable at the instance public IP.

Diagnosis: both symptoms were the new security group behaving exactly as designed. Port 80 now accepts traffic only from the ALB security group, so direct browser access is correctly refused; the browser hang is the expected symptom of a silently dropped packet rather than a rejected connection. EC2 Instance Connect originates from an AWS service address range rather than from the my workstation, so it did not match the SSH rule restricted to My IP.

Resolution: the AWS managed prefix list com.amazonaws.ap-southeast-2.ec2-instance-connect was added as the source of the SSH rule, restoring browser-based access without opening port 22 to the internet. Application health was verified from inside the instance using curl to localhost, which bypasses the security group entirely and confirmed Apache was serving correctly while the instance was externally unreachable.

Lesson: a security control functioning correctly and a fault present an identical symptom. Distinguishing them requires a test path that is not subject to the control - in this case localhost. This incident also produced Figure 47, which is direct evidence of the network isolation the design intends.

7.6 WordPress Canonical URL Redirection

Symptom: the restored second instance redirected visitors to the original instance IP address rather than serving its own content.

Diagnosis: WordPress stores its canonical address in the database in the `wp_options` `siteurl` and `home` rows. Because both instances share one RDS database, both inherited the address configured during the original installation. This is not a restore fault; it is a design property of a shared-database deployment that would have affected every Auto Scaling Group instance.

Resolution: `WP_HOME` and `WP_SITEURL` were defined in `wp-config.php`, which overrides the database values at runtime. At Distinction level these were permanently set to the ALB DNS name before the golden AMI was created, so all instances present a single consistent canonical address.

Lesson: state stored in a shared database becomes a coupling between otherwise independent instances. Identifying this at Credit level, when it affected two instances, avoided rediscovering it at Distinction level when it would have affected the whole fleet.

7.7 stress-ng Temporary Path Failure

```
aborting: temp-path '.' must be readable and writeable
```

Symptom: `stress-ng` refused to start when launched from a Session Manager session, but the same command worked over SSH.

Diagnosis: Session Manager starts sessions as `ssm-user` with a working directory that is not writable by that account, whereas an SSH session starts in the `ec2-user` home directory. `stress-ng` requires a writable working directory for its temporary files.

Resolution: the working directory was changed to `/tmp` before invocation.

Lesson: Session Manager is not a drop-in replacement for SSH in every respect. The identity, the working directory and the environment differ, and scripts that make assumptions about any of these need review before an SSH-based workflow is retired.

8. Limitations and Future Work

The following limitations are known and were accepted within the scope and cost constraints of this task. They are stated explicitly because an accurate account of what a deployment does not do is as informative as an account of what it does.

8.1 Uploaded Media Is Not Shared Between Instances

The wp-content/uploads directory resides on each instance's local EBS volume. A media file uploaded through the WordPress admin interface is written to whichever instance served that request and returns HTTP 404 when a subsequent request is routed to another instance. The file is also lost permanently when the Auto Scaling Group replaces that instance.

This is the same class of problem the database migration solved, applied to a different kind of state. The appropriate remedies are an Amazon EFS file system mounted at wp-content/uploads and shared by all instances, or an S3 offload plugin that stores media in the existing bucket and rewrites URLs. EFS was not implemented because it falls outside the free tier for sustained use.

8.2 No HTTPS Termination

The ALB listens on HTTP port 80 only. Terminating TLS requires an ACM certificate, which requires a domain name that can be validated - neither an ALB DNS name nor an IP address is eligible. The X-Forwarded-Proto handling is already present in wp-config.php, so adding an HTTPS listener and an HTTP-to-HTTPS redirect rule would require no application change. Until then, WordPress admin credentials traverse the internet unencrypted, which is the most significant outstanding security weakness in the deployment.

8.3 Fixed Capacity Rather Than Elastic Scaling

The Auto Scaling Group is configured with a fixed capacity of two, as specified by the task. It therefore provides high availability but not elasticity: it will not add capacity under load or remove it when idle. The CloudWatch alarm built at High Distinction level already produces exactly the signal a target tracking or step scaling policy would consume, so converting the deployment to scale automatically would be a configuration change rather than a redesign.

8.4 Single-AZ Database

RDS was deployed as a single instance without Multi-AZ standby, because Multi-AZ is not free tier eligible. The database is consequently the remaining single point of failure in an otherwise multi-zone architecture: the loss of its Availability Zone would take the application offline despite the load balancer and Auto Scaling Group remaining healthy. Automated backups with seven-day retention limit data loss but do not prevent an outage.

8.5 Manual Backups and Manual Deployment

Backups are executed manually and are not scheduled. A cron entry, an EventBridge rule, or AWS Backup would automate them; S3 lifecycle policies would then manage retention and archival to a lower-cost storage class. Similarly, application changes require rebuilding the AMI and refreshing the Auto Scaling Group by hand - there is no CI/CD pipeline.

8.6 Infrastructure Defined Through the Console

All resources were created through the AWS Management Console. The configuration is therefore documented rather than declared, and reproducing the environment requires repeating the steps in this report. Expressing the stack as CloudFormation or CDK would make it version-controlled, reviewable and repeatable, and would eliminate the class of configuration drift that manual changes introduce. The parameter values recorded throughout this report were captured with that conversion in mind, and the repository accompanying this submission stores the scripts and policy documents in a form suitable for it.

8.7 Monitoring Covers Compute Only

CPU utilisation is a proxy metric and a weak one for a PHP application, whose failures more often manifest as memory pressure, database connection exhaustion or elevated response time. Memory and disk metrics require the CloudWatch Agent, which was not installed. There is no log aggregation - Apache and PHP error logs remain on each instance and are lost on replacement - and the alarm notifies without remediating.

9. External Code and Script Repository

In accordance with the submission guidelines, no full source files, database dumps or private keys are reproduced in this report. The scripts, policy documents and configuration templates produced for this task are published in a public version-controlled repository.

Repository contents:

Path	Purpose
README.md	Repository overview, architecture summary and usage notes
scripts/user-data.sh	Launch template boot script - health endpoint, TLS trust store, instance identity page
scripts/provision-lamp.sh	LAMP stack installation used on the restore-target instance
scripts/backup-to-s3.sh	Creates the application and database backups and uploads them to S3
scripts/restore-from-s3.sh	Downloads and restores an application backup onto a fresh instance
iam/s3-backup-policy.json	Least-privilege IAM policy attached to the instance role
config/wp-config-snippet.php	The added wp-config.php directives, with all secrets redacted
config/health.html	Static ALB health check endpoint
docs/architecture.md	Resource inventory, security group matrix and network topology
docs/teardown.md	Ordered decommissioning procedure

No credential, key material or database content is present in the repository. The wp-config.php snippet contains redaction markers in place of the database password, and the endpoint hostnames are given as placeholders.

10. Generative AI Declaration

This declaration is provided in accordance with Swinburne University's policy on the use of generative artificial intelligence in assessment.

10.1 Tool Used and Scope of Use

A generative AI assistant (Anthropic Claude) was used during this task. Its use fell into three categories:

- Procedural guidance. Task 3.1 was accompanied by a unit-provided walkthrough; Tasks 3.2 to 3.4 were not, and the assessment brief explicitly directs students to undertake self-directed research for those levels. The AI assistant was used as one research aid among the AWS documentation cited in Section 11, principally to establish the correct order of operations - for example, that a security group should be created before the RDS instance it will be attached to, and that the ALB should be provisioned before the golden AMI is built so its DNS name can be written into the application configuration.
- Error diagnosis. When a command or console operation failed, the error output was described to the assistant to help identify which layer of the stack the fault occupied. Each of the eight faults recorded in Section 7 was investigated this way, then verified independently against the actual system state before any change was applied.
- Report drafting assistance. The assistant was used to help structure this document and to review written explanations for clarity.

10.2 What Was Performed by me

Every action taken against the AWS account was performed by me. The AI assistant had no access to the AWS account, no ability to execute commands, and no visibility of the console at any point. Specifically, I personally:

- Created the AWS account and selected and maintained the working region.
- Generated the key pair and launched, configured and connected to every EC2 instance.
- Installed and configured Apache, MariaDB, PHP and WordPress, and executed every command recorded in this report.
- Designed and created all four security groups, including the decision to authorise by security group reference rather than by CIDR range.
- Provisioned the RDS instance, exported and imported the database, created the application user, and reconfigured the application.
- Created the S3 bucket, authored the IAM policy, created and attached the role, and performed the backup and restore.
- Built the AMI, launch template, target group, load balancer and Auto Scaling Group, and performed all verification testing including the instance termination test.
- Created the SNS topic and CloudWatch alarm, generated the load that triggered it, and configured Session Manager and removed public SSH access.
- Captured every screenshot in this report from my own AWS account.

10.3 Verification and Independent Judgement

AI-supplied guidance was treated as a starting point requiring verification, not as authoritative instruction. Several instances of this are documented in the body of this report, and are stated here explicitly because they evidence understanding rather than transcription:

- A suggested command contained the documentation placeholder <DATE>, which bash interprets as a redirection operator (Section 7.4). I identified the shell-level cause and corrected the command rather than reporting it as an AWS fault.
- A suggested endpoint hostname was a generic example rather than the account-specific value (Section 7.1). I recognised the DNS-layer failure signature and retrieved the correct endpoint from the console.
- When the TLS requirement was encountered, disabling `require_secure_transport` via a custom parameter group was the faster remedy. I rejected it as a weakening of the security posture and implemented the correct client-side configuration instead, which subsequently revealed the related `mysql` requirement in the application (Section 7.2).
- The order of operations was altered from the guidance received: Systems Manager was configured before the CloudWatch load test, so that the load could be generated through Session Manager rather than by reopening SSH access that was about to be permanently removed.
- Where guidance offered alternatives - access keys versus an IAM role, `AmazonS3FullAccess` versus a scoped custom policy, boot-time provisioning versus a golden AMI - I selected among them on stated technical grounds and has documented the reasoning and the trade-offs in the relevant sections.

10.4 Statement

I understand the architecture described in this report, the function and configuration of every AWS service used, and the reasoning behind each design decision recorded. All configuration was implemented, tested and verified by me. The analysis, troubleshooting reasoning and evaluation of limitations in this report represents my own understanding. I accept full responsibility for the accuracy of the content of this submission.

Field	Detail
Signed	Karamjot Singh
Student ID	104809887
Date	09/09/2026

11. References

All AWS documentation was accessed during September 2026.

- [1] Amazon Web Services, "Amazon EC2 User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/ec2/>. [Accessed: Sep. 2026].
- [2] Amazon Web Services, "Amazon RDS User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>. [Accessed: Sep. 2026].
- [3] Amazon Web Services, "Using SSL/TLS to encrypt a connection to a DB instance," Amazon RDS User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDS.SSL.html>. [Accessed: Sep. 2026].
- [4] Amazon Web Services, "Amazon S3 User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>. [Accessed: Sep. 2026].
- [5] Amazon Web Services, "IAM roles for Amazon EC2," AWS Identity and Access Management User Guide, 2026. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_switch-role-ec2.html. [Accessed: Sep. 2026].
- [6] Amazon Web Services, "Security best practices in IAM," AWS Identity and Access Management User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>. [Accessed: Sep. 2026].
- [7] Amazon Web Services, "Application Load Balancer User Guide," Elastic Load Balancing Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/>. [Accessed: Sep. 2026].
- [8] Amazon Web Services, "Amazon EC2 Auto Scaling User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/>. [Accessed: Sep. 2026].
- [9] Amazon Web Services, "Launch templates," Amazon EC2 Auto Scaling User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-templates.html>. [Accessed: Sep. 2026].
- [10] Amazon Web Services, "Amazon CloudWatch User Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>. [Accessed: Sep. 2026].
- [11] Amazon Web Services, "Amazon Simple Notification Service Developer Guide," AWS Documentation, 2026. [Online]. Available: <https://docs.aws.amazon.com/sns/latest/dg/>. [Accessed: Sep. 2026].
- [12] Amazon Web Services, "AWS Systems Manager Session Manager," AWS Systems Manager User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>. [Accessed: Sep. 2026].
- [13] Amazon Web Services, "AWS Well-Architected Framework: Security Pillar," AWS Whitepaper, 2026. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/>. [Accessed: Sep. 2026].
- [14] Amazon Web Services, "AWS Well-Architected Framework: Reliability Pillar," AWS Whitepaper, 2026. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/>. [Accessed: Sep. 2026].
- [15] Amazon Web Services, "Use IMDSv2," Amazon EC2 User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html>. [Accessed: Sep. 2026].
- [16] WordPress.org, "Editing wp-config.php," WordPress Developer Resources, 2026. [Online]. Available: <https://developer.wordpress.org/advanced-administration/wordpress/wp-config/>. [Accessed: Sep. 2026].
- [17] MariaDB Foundation, "mysqldump," MariaDB Knowledge Base, 2026. [Online]. Available: <https://mariadb.com/kb/en/mysqldump/>. [Accessed: Sep. 2026].
- [18] The PHP Group, "mysqli::real_connect - MYSQLI_CLIENT_SSL," PHP Manual, 2026. [Online]. Available: <https://www.php.net/manual/en/mysqli.real-connect.php>. [Accessed: Sep. 2026].